

TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT TP.HCM

KHOA ĐIỆN ĐIỆN TỬ

BỘ MÔN KỸ THUẬT MÁY TÍNH – VIỄN THÔNG



HCMUTE

ĐỒ ÁN MÔN HỌC 2

**THIẾT KẾ VÀ THI CÔNG HỆ THỐNG GIÁM SÁT
NHÀ THÔNG MINH**

NGÀNH CÔNG NGHỆ KỸ THUẬT ĐIỆN TỬ - TRUYỀN THÔNG

Sinh viên: **NGUYỄN TÀI ANH TUẤN**

MSSV: 22161203

TRẦN CAO MINH KHÔI

MSSV: 22161142

GVHD: **ThS. NGUYỄN NGÔ LÂM**

TP. HỒ CHÍ MINH – 05/2025

LỜI CẢM ƠN

Trước hết, em xin được gửi lời tri ân chân thành và sâu sắc đến Ban Giám hiệu Trường Đại học Sư phạm Kỹ thuật TP.HCM, cùng quý thầy cô trong Khoa Điện – Điện Tử, đặc biệt là Bộ môn Kỹ thuật Máy tính – Viễn thông đã luôn tạo điều kiện thuận lợi, cung cấp môi trường học tập và nghiên cứu chuyên nghiệp, hiện đại để em có thể tiếp thu kiến thức và hoàn thành tốt đồ án môn học.

Em đặc biệt xin gửi lời cảm ơn trân trọng đến Thạc sĩ Nguyễn Ngô Lâm – giảng viên hướng dẫn của em, người đã luôn tận tâm chỉ dẫn, theo sát quá trình thực hiện đồ án, không ngại dành thời gian đóng góp ý kiến, định hướng và hỗ trợ em vượt qua những khó khăn về kỹ thuật cũng như định hình tư duy nghiên cứu khoa học. Sự tận tụy và tinh thần trách nhiệm của thầy là nguồn động lực to lớn giúp em hoàn thiện đồ án một cách tốt nhất.

Cuối cùng, em xin cảm ơn gia đình và bạn bè đã luôn bên cạnh động viên, cổ vũ tinh thần để em vững bước trong quá trình học tập và hoàn thành đồ án. Em xem đây là bước khởi đầu quý giá để tích lũy kinh nghiệm, rèn luyện bản thân, và sẵn sàng cho những thử thách trong hành trình nghề nghiệp phía trước.

TP.Hồ Chí Minh, tháng 05 năm 2025

Sinh viên thực hiện

Nguyễn Tài Anh Tuấn

Trần Cao Minh Khôi

TÓM TẮT

Tại các hộ gia đình hiện đại, việc triển khai hệ thống giám sát và điều khiển thông minh thường gặp trở ngại do chi phí cao và yêu cầu kết nối ổn định. Nhằm giải quyết vấn đề này, đồ án đã xây dựng một hệ thống nhà thông minh sử dụng kiến trúc phân tán, hoạt động dựa trên hai vi điều khiển ESP32, trong đó một vi điều khiển đóng vai trò ESP32 Slave, còn lại là ESP32 Master. Hệ thống thực hiện các chức năng chính như giám sát môi trường, xác thực người dùng, điều khiển thiết bị và đảm bảo an ninh, được kết nối nội bộ với nền tảng đám mây thông qua WiFi.

ESP32 Slave phụ trách thu thập dữ liệu môi trường bao gồm ánh sáng, nhiệt độ, độ ẩm và khí gas, sau đó hiển thị lên màn hình Oled thông qua giao tiếp I2C. Đồng thời, ESP32 Master đảm nhiệm việc xác thực người dùng thông qua các phương thức như mật khẩu bàn phím, thẻ RFID và cảm biến vân tay. Khi xác thực thành công, hệ thống sẽ hiển thị trạng thái lên màn hình LCD, đồng thời điều khiển servo để mở cửa tự động, đảm bảo sự an toàn và tiện lợi cho người sử dụng. Cả 2 ESP32 đều đồng bộ dữ liệu lên nền tảng Firebase và hiển thị trực quan thông qua Web Dashboard, giúp người dùng giám sát hệ thống từ xa theo thời gian thực.

Toàn bộ hệ thống được thiết kế với mục tiêu tiết kiệm chi phí, dễ mở rộng, kết nối linh hoạt, góp phần nâng cao chất lượng sống, tăng cường tính bảo mật và hiện đại hóa không gian sống trong các ngôi nhà thông minh.

MỤC LỤC

LỜI CẢM ƠN.....	i
TÓM TẮT	ii
MỤC LỤC	iii
DANH MỤC CÁC TỪ VIẾT TẮT.....	vi
DANH MỤC BẢNG BIỂU	viii
DANH MỤC HÌNH ẢNH.....	ix
CHƯƠNG 1: TỔNG QUAN.....	1
1.1. GIỚI THIỆU CHUNG.....	1
1.2. MỤC TIÊU ĐỀ TÀI	1
1.3. TÌNH HÌNH NGHIÊN CỨU TRONG VÀ NGOÀI NƯỚC.....	1
1.3.1. Tình hình nghiên cứu ở nước ngoài	1
1.3.2. Tình hình nghiên cứu ở trong nước.....	2
1.4. PHƯƠNG PHÁP NGHIÊN CỨU	3
1.5. BỐ CỤC BÀI BÁO CÁO.....	4
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT	6
2.1. TỔNG QUAN VỀ HỆ THỐNG GIÁM NHÀ THÔNG MINH.....	6
2.2. KIẾN TRÚC HỆ THỐNG MASTER - SLAVE TRONG NHÀ THÔNG MINH.....	7
2.2.1. Nguyên lý hoạt động Master – Slave	7
2.2.2. Vi điều khiển ESP32	8
2.3. CÁC MODULE CẢM BIẾN, ĐIỀU KHIỂN, HIỂN THỊ	11
2.3.1. Cảm biến nhiệt độ, độ ẩm.....	11
2.3.2. Cảm biến cường độ ánh sáng	13
2.3.3. Cảm biến khí GAS	15
2.3.4. Module RFID RC522	17
2.3.5. Cảm biến vân tay AS608.....	19
2.3.6. Keypad 4x4.....	21
2.3.7. Module Relay 4 kênh	23

2.3.8. Động cơ Servo	26
2.3.9. Màn hình LCD 20X4 giao tiếp I2C.....	28
2.9.10. Màn hình OLED giao tiếp I2C	30
2.4. CÁC CHUẨN TRUYỀN THÔNG SỬ DỤNG.....	32
2.4.1. Giao tiếp UART	32
2.4.2. Giao tiếp I2C	34
2.4.3. Giao tiếp SPI	35
2.4.4. Giao tiếp WiFi	37
2.5. LƯU TRỮ DỮ LIỆU VÀ GIAO DIỆN NGƯỜI DÙNG	39
2.5.1. Giới thiệu về Firebase Realtime Database	39
2.5.2. Các bước thiết lập Firebase	40
2.5.3. Giới thiệu Web Dashboard giám sát và điều khiển.....	42
2.6. TỔNG QUAN VỀ CÁC CÔNG CỤ HỖ TRỢ XÂY DỰNG VÀ LẬP TRÌNH	43
2.6.1. Phần mềm Arduino IDE	43
2.6.2. Phần mềm Easy EDA	44
2.6.3. Phần mềm Visual Studio Code.....	45
CHƯƠNG 3: THIẾT KẾ HỆ THỐNG	47
3.1. YÊU CẦU THIẾT KẾ HỆ THỐNG	47
3.2. ĐẶC TẢ HỆ THỐNG	48
3.2.1. Chức năng hệ thống tổng thể.....	48
3.2.2. Chức năng ESP32 – Master.....	48
3.2.3. Chức năng ESP32 – Slave.....	49
3.2.4. Sơ đồ khối hệ thống.....	51
3.3. THIẾT KẾ PHẦN CỨNG	52
3.3.1. Thiết kế khối Master.....	52
3.3.2. Thiết kế khối Slave.....	62
3.3.3. Thiết kế khối nguồn.....	71
3.3.4. Sơ đồ nguyên lý toàn bộ hệ thống.....	74
3.4. THIẾT KẾ PHẦN MỀM.....	75
3.4.1. Lưu đồ giải thuật cho ESP32 – Master	75

3.4.2. Lưu đồ giải thuật cho ESP32 – Slave.....	85
CHƯƠNG 4: KẾT QUẢ VÀ ĐÁNH GIÁ.....	91
4.1. KẾT QUẢ PHẦN CỨNG	91
4.1.1. Khối Master	91
4.1.2. Khối Slave	99
4.2. GIAO DIỆN PHẦN MỀM	108
4.3. ĐÁNH GIÁ VÀ NHẬN XÉT HỆ THỐNG	110
4.3.1. Đánh giá hiệu suất	111
4.3.2. Ưu điểm và những hạn chế của hệ thống.....	111
CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	113
5.1. KẾT LUẬN	113
5.2. HƯỚNG PHÁT TRIỂN	113
TÀI LIỆU THAM KHẢO	115

DANH MỤC CÁC TỪ VIẾT TẮT

Từ viết tắt	Từ viết đầy đủ	Nghĩa tiếng việt
ADC	Analog-to-Digital Converter	Bộ chuyển đổi tương tự sang số
AI	Artificial Intelligence	Trí tuệ nhân tạo
BLE	Bluetooth Low Energy	Bluetooth năng lượng thấp
DAC	Digital-to-Analog Converter	Bộ chuyển đổi số sang tương tự
DSP	Digital Signal Processing	Xử lý tín hiệu số
FPU	Floating Point Unit	Đơn vị xử lý số thực
FreeRTOS	Free Real-Time Operating System	Hệ điều hành thời gian thực
GPIO	General Purpose Input/Output	Ngõ vào/ra
HTTPS	HyperText Transfer Protocol Secure	Giao thức truyền tải siêu văn bản bảo mật
I2C	Inter-Integrated Circuit	Giao tiếp giữa các mạch tích hợp
IoT	Internet of Things	Internet kết nối vạn vật
LCD	Liquid Crystal Display	Màn hình tinh thể lỏng
MQTT	Message Queuing Telemetry Transport	Giao thức truyền thông điệp
PCB	Printed Circuit Board	Bảng mạch in
PWM	Pulse Width Modulation	Điều chế độ rộng xung

RFID	Radio Frequency Identification	Nhận diện tần số vô tuyến
SPI	Serial Peripheral Interface	Giao diện ngoại vi nối tiếp
SRAM	Static Random-Access Memory	Bộ nhớ truy cập ngẫu nhiên tĩnh
UART	Universal Asynchronous Receiver - Transmitter	Bộ thu phát không đồng bộ tổng hợp
Wi-Fi	Wireless Fidelity	Tín hiệu không dây

DANH MỤC BẢNG BIỂU

Bảng 2.1: Bảng thông số kỹ thuật chính của ESP32-S	8
Bảng 2.2: Thông số kỹ thuật DHT22	11
Bảng 2.3: Tên và chức năng từng chân DHT22.....	12
Bảng 2.4: Thông số kỹ thuật BH1750.....	13
Bảng 2.5: Tên và chức năng từng chân BH1750	14
Bảng 2.6: Thông số kỹ thuật MQ-2	15
Bảng 2.7: Tên và chức năng từng chân MQ-2	16
Bảng 2.8: Thông số kỹ thuật RFID	17
Bảng 2.9: Tên và chức năng từng chân RFID.....	18
Bảng 2.10: Thông số kỹ thuật AS608	19
Bảng 2.11: Tên và chức năng từng chân AS608.....	20
Bảng 2.12: Thông số kỹ thuật Keypad 4x4.....	22
Bảng 2.13: Tên và chức năng từng chân Keypad 4x4	22
Bảng 2.14: Thông số kỹ thuật Module Relay	24
Bảng 2.15: Tên và chức năng từng chân Module Relay 4 kênh	25
Bảng 2.16: Thông số kỹ thuật Servo.....	26
Bảng 2.17: Tên và chức năng từng chân Servo	27
Bảng 2.18: Thông số kỹ thuật LCD20x4	28
Bảng 2.19: Tên và chức năng từng chân LCD20x4 I2C.....	29
Bảng 2.20: Thông số kỹ thuật OLED	30
Bảng 2.21: Tên và chức năng từng chân OLED I2C	31
Bảng 3.1: Bảng kết nối chân giữa Keypad và ESP32 – Master.....	54
Bảng 3.2: Bảng kết nối chân giữa RFID với ESP32 – Master.....	55
Bảng 3.3: Bảng kết nối chân giữa AS608 với ESP32 MASTER	57
Bảng 3.4: Bảng kết nối chân giữa Servo và ESP32 – Master	59
Bảng 3.5: Bảng kết nối chân giữa LCD và ESP32 – Master	60
Bảng 3.6: Bảng kết nối chân giữ các module với ESP32 Master	61
Bảng 3.7: Bảng kết nối chân giữa cảm biến và ESP32 – Slave	64
Bảng 3.8: Bảng kết nối chân MQ2 với ESP32 Slave.....	66
Bảng 3.9: Bảng kết nối chân BH1750 với ESP32 Slave	67
Bảng 3.10: Bảng kết nối chân giữa module relay với ESP32 – Slave	68
Bảng 3.11: Bảng kết nối chân giữa OLED và ESP32 - Slave	69
Bảng 3.12: Thông số kỹ thuật của khối nguồn	72
Bảng 3.13: Bảng thông số điện áp, dòng và công suất tiêu thụ của hệ thống.....	72

DANH MỤC HÌNH ẢNH

Hình 2.1: Vi điều khiển ESP32-S	8
Hình 2.2: ESP32 và chức năng cơ bản của từng chân	10
Hình 2.3: DHT22 và vị trí của từng chân	12
Hình 2.4: Hình ảnh ký hiệu DHT22.....	12
Hình 2.5: BH1750 và vị trí từng chân.....	14
Hình 2.6: Hình ảnh ký hiệu BH1750	14
Hình 2.7: MQ-2 và vị trí từng chân.....	16
Hình 2.8: Hình ảnh ký hiệu MQ-2	16
Hình 2.9: RFID và vị trí từng chân	18
Hình 2.10: Hình ảnh ký hiệu RFID	18
Hình 2.11: AS608 và vị trí từng chân	20
Hình 2.12: Hình ảnh ký hiệu AS608.....	20
Hình 2.13: Keypad 4x4 và vị trí từng chân.....	22
Hình 2.14: Hình ảnh ký hiệu Keypad	22
Hình 2.15: Module Relay 4 kênh và vị trí từng chân.....	25
Hình 2.16: Hình ảnh ký hiệu Module Relay	25
Hình 2.17: Servo MG90S và vị trí từng chân	27
Hình 2.18: Hình ảnh ký hiệu Servo.....	27
Hình 2.19: Module I2C PC8574	29
Hình 2.20: LCD 20x4.....	29
Hình 2.21: Hình ảnh ký hiệu LCD	29
Hình 2.22: OLED và vị trí từng chân.....	31
Hình 2.23: Hình ảnh ký hiệu OLED	31
Hình 2.24: Giao tiếp UART giữa hai thiết bị.....	32
Hình 2.25: Tín hiệu truyền của UART	33
Hình 2.26: Giao tiếp I2C giữa hai thiết bị.....	34
Hình 2.27: Tín hiệu truyền I2C	35
Hình 2.28: Giao tiếp SPI giữa hai thiết bị.....	36
Hình 2.29: Tín hiệu truyền SPI	37
Hình 2.30: Giao thức WiFi.....	38
Hình 2.31: ESP giao tiếp WiFi.....	38
Hình 2.32: Hình ảnh minh họa giao diện Realtime Database.....	40
Hình 2.33: Phần mềm Arduino IDE.....	43
Hình 2.34: Phần mềm Easy EDA.....	44
Hình 2.35: Phần mềm Visual Studio Code	46
 Hình 3.1: Sơ đồ khối hệ thống	 51

Hình 3.2: Module bàn phím kết nối với Master.....	53
Hình 3.3: Module RFID kết nối với Master.....	55
Hình 3.4: Module vân tay kết nối với MASTER.....	57
Hình 3.5: Khối điều khiển cửa kết nối với Master.....	58
Hình 3.6: Khối hiển thị LCD kết nối với Master	59
Hình 3.7: Khối xử lý trung tâm Master.....	60
Hình 3.8: Sơ đồ nguyên lý ESP32 – Master với các khối.....	61
Hình 3.9: Module nhiệt độ kết nối với Slave	64
Hình 3.10: Module khí gas kết nối với Slave.....	65
Hình 3.11: Module cảm biến ánh sáng kết nối với Slave	66
Hình 3.12: Khối điều khiển relay kết nối với Slave.....	68
Hình 3.13: Khối hiển thị OLED kết nối với Slave.....	69
Hình 3.14: Khối xử lý trung tâm Slave.....	70
Hình 3.15: Sơ đồ nguyên lý ESP32 – Slave với các khối.....	71
Hình 3.16: Khối nguồn.....	72
Hình 3.17: Sơ đồ nguyên lý toàn bộ hệ thống.....	74
Hình 3.18: Lưu đồ giải thuật chính ESP32 – Master	75
Hình 3.19: Lưu đồ giải thuật con chế độ xác thực	76
Hình 3.20: Lưu đồ giải thuật con chọn xác thực.....	77
Hình 3.21: Lưu đồ giải thuật con hiển thị menu Slection.....	78
Hình 3.22: Lưu đồ giải thuật con chế độ cài đặt.....	80
Hình 3.23: Lưu đồ giải thuật con hiển thị setup Selection.....	81
Hình 3.24: : Lưu đồ giải thuật con điều khiển servo.....	83
Hình 3.25: Lưu đồ giải thuật con giao tiếp Firebase với Master	84
Hình 3.26: Lưu đồ giải thuật ESP – Slave	85
Hình 3.27: Lưu đồ giải thuật con đọc giá trị cảm biến	86
Hình 3.28: Lưu đồ giải thuật con điều khiển relay	87
Hình 3.29: Lưu đồ giải thuật con hiển thị OLED	88
Hình 3.30: Lưu đồ giải thuật giao tiếp Firebase với Slave	89
Hình 4.1: Mạch PCB của khối Master	91
Hình 4.2: LCD hiển thị thông tin khi cấp nguồn	92
Hình 4.3: Chế độ xác thực.....	93
Hình 4.4: Phương thức mật khẩu.....	93
Hình 4.5: Xác thực mật khẩu thành công.....	94
Hình 4.6: Xác thực mật khẩu sai.....	94
Hình 4.7: Phương thức thẻ RFID	95
Hình 4.8: Xác thực thẻ RFID thành công	95
Hình 4.9: Xác thực thẻ RFID sai.....	96

Hình 4.10: Phương thức vân tay	96
Hình 4.11: Xác thực vân tay thành công.....	97
Hình 4.12: Xác thực vân tay sai	97
Hình 4.13: Chế độ cài đặt.....	98
Hình 4.14: Dữ liệu xác thực được lưu trữ trên Firebase	98
Hình 4.15: Dữ liệu xác thực lưu trữ trên Dashboard	99
Hình 4.16: Kết nối WiFi.....	101
Hình 4.17: Kết nối Firebase	101
Hình 4.18: Test relay.....	101
Hình 4.19: OLED hiển thị thông số cảm biến.....	101
Hình 4.20: OLED hiển thị trạng thái thiết bị	102
Hình 4.21: OLED hiển thị chế độ điều khiển	102
Hình 4.22: Điều khiển từ xa thông qua Dashboard.....	103
Hình 4.23: LED 1 và LED 2 ở chế độ từ xa	103
Hình 4.24: Quạt ở hai chế độ từ và từ động.....	103
Hình 4.25: Quạt ở hai chế độ	104
Hình 4.26: LED 4 ở hai chế độ từ xa và tự động	104
Hình 4.27: Đèn ở hai chế độ	105
Hình 4.28: Trạng thái điều khiển thiết bị trên Firebase	105
Hình 4.29: Dữ liệu cảm biến trên Firebase	105
Hình 4.30: Thông số nhiệt độ.....	106
Hình 4.31: Thông số độ ẩm.....	106
Hình 4.32: Thông số khí gas	106
Hình 4.33: Thông số ánh sáng	106
Hình 4.34: Biểu đồ thông số nhiệt độ	106
Hình 4.35: Biểu đồ thông số độ ẩm	107
Hình 4.36: Biểu đồ thông số ánh sáng	107
Hình 4.37: Biểu đồ thông số khí gas.....	108
Hình 4.38: Trang tổng quan	108
Hình 4.39: Trang cảm biến	109
Hình 4.40: Trang thiết bị.....	109
Hình 4.41: Trang bảo mật	110
Hình 4.42: Trang lịch sử	110

CHƯƠNG 1: TỔNG QUAN

1.1. GIỚI THIỆU CHUNG

Trong thời đại công nghệ phát triển mạnh mẽ, sự bùng nổ của Internet of Things (IoT) và trí tuệ nhân tạo (AI) đã mở ra một kỷ nguyên mới, nơi khái niệm Smart Home trở thành xu hướng tất yếu, định hình lại cách con người sống và tương tác với không gian sống. Hệ thống nhà thông minh không chỉ mang lại sự tiện nghi vượt trội thông qua tự động hóa các công việc hàng ngày mà còn đóng vai trò quan trọng trong việc nâng cao an ninh, tối ưu hóa quản lý năng lượng và cải thiện chất lượng cuộc sống trong các hộ gia đình hiện đại. Tuy nhiên, khi nhu cầu về một môi trường sống an toàn, tiện nghi và tiết kiệm ngày càng gia tăng, các giải pháp giám sát và điều khiển truyền thống dần bộc lộ những hạn chế nghiêm trọng, bao gồm khả năng truy cập từ xa hạn chế, thiếu tính linh hoạt trong điều khiển tự động và khó khăn trong việc tích hợp đa dạng các chức năng. Những bất cập này đặt ra yêu cầu cấp thiết về việc phát triển một hệ thống nhà thông minh toàn diện, có khả năng khắc phục các rào cản hiện tại để đáp ứng nhu cầu thực tế.

1.2. MỤC TIÊU ĐỀ TÀI

Đề tài xây dựng một hệ thống nhà thông minh, với mục tiêu thu thập xử lý dữ liệu nhiệt độ, độ ẩm, nồng độ khí gas và ánh sáng trong không gian sống. Hệ thống cũng hướng đến việc nâng cao bảo mật và hỗ trợ điều khiển từ xa thông qua tích hợp các phương thức xác thực bảo mật an toàn, đồng thời sử dụng giao thức Wi-Fi để truyền dữ liệu, cho phép người dùng giám sát và quản lý thiết bị từ xa thông qua giao diện web. Hệ thống sẽ được thử nghiệm để đánh giá hiệu suất, bao gồm độ tin cậy của cảm biến, tốc độ truyền dữ liệu qua Wi-Fi và hiệu quả của phương thức xác thực, từ đó sẽ đưa ra các giải pháp cải tiến hoặc mở rộng trong tương lai.

1.3. TÌNH HÌNH NGHIÊN CỨU TRONG VÀ NGOÀI NƯỚC

1.3.1. Tình hình nghiên cứu ở nước ngoài

Lĩnh vực nhà thông minh đã và đang phát triển mạnh mẽ trên phạm vi toàn cầu, trở thành một trong những ứng dụng trọng tâm của IoT. Các nghiên cứu quốc

tế không ngừng khám phá ra các phương pháp mới để nâng cao tính tiện nghi, hiệu quả năng lượng, bảo mật và khả năng tương tác của hệ thống giám sát và điều khiển nhà ở.

Trong bài báo khảo sát [1], Bài báo này tập trung vào các giải pháp công nghệ tiên tiến cho phép kiểm soát và tự động hóa các thiết bị trong nhà thông qua một trung tâm điều khiển trung tâm, điện thoại thông minh, lệnh thoại và IoT. Nghiên cứu này đã phân tích các công nghệ giao tiếp đa dạng như Wi-Fi, Bluetooth, Zigbee cùng với các giao diện điều khiển như ứng dụng di động, trợ lý giọng nói. Ngoài ra, còn đề cập đến các tính năng bảo mật quan trọng nhưng lại có thách thức chính về khả năng tương tác và chi phí cao.

Trong nghiên cứu [2], cung cấp một cái nhìn tổng quan về các hoạt động và chức năng cơ bản của nhà thông minh từ giai đoạn đầu phát triển. Bài báo này đã hệ thống hóa các khái niệm, ứng dụng và công nghệ nền tảng được sử dụng trong các hệ thống nhà thông minh tại thời điểm đó, bao gồm các phương pháp quản lý năng lượng và điều khiển thiết bị. Mặc dù là một công trình mang tính nền tảng, các công nghệ và xu hướng mới nhất như trí tuệ nhân tạo, học máy hay giao thức IoT chuyên biệt sau này vẫn chưa được đề cập tới.

Các nghiên cứu trên quốc tế đã đạt được nhiều thành tựu trong việc phát triển các hệ thống nhà thông minh tiên tiến với nhiều tính năng đa dạng. Tuy nhiên, cũng có nhiều thách thức về mặt chi phí tối ưu, khả năng tích hợp giữa các nền tảng khác nhau và việc xây dựng hệ thống đủ mạnh để đáp ứng mọi nhu cầu phức tạp vẫn là thách thức lớn.

1.3.2. Tình hình nghiên cứu ở trong nước

Tại Việt Nam, lĩnh vực nhà thông minh cũng đang có những bước phát triển tích cực, đặc biệt trong bối cảnh cuộc cách mạng công nghiệp 4.0 và chuyển đổi số được đẩy mạnh. Các nghiên cứu trong nước thường tập trung vào việc ứng dụng các công nghệ IoT hiện có để xây dựng các giải pháp thiết thực, phù hợp với điều kiện và nguồn lực trong nước.

Trong nghiên cứu [3], thiết kế và triển khai một hệ thống nhà thông minh cho phép điều khiển và giám sát các thiết bị từ xa thông qua chuẩn không dây Zigbee. Hệ thống cung cấp giao diện trực quan trên ứng dụng di động, cho phép người dùng điều khiển thủ công và giám sát trạng thái của thiết bị. Mặc dù đã đạt được khả năng điều khiển và giám sát từ xa, nhưng dự án này vẫn còn một số hạn chế như có độ trễ cao khi điều khiển qua ứng dụng, chỉ hỗ trợ trên Android và chưa đạt được mức tự động hóa hoàn toàn.

Trong nghiên cứu [4], đóng góp đáng kể vào lĩnh vực này bằng cách phát triển một mô hình nhà thông minh tích hợp phần cứng và phần mềm dựa trên công nghệ IoT. Hệ thống này tập trung vào việc mô phỏng các chức năng cơ bản như thu thập dữ liệu môi trường, điều khiển thiết bị qua giao diện Web Dashboard và cung cấp khả năng giám sát thời gian thực. Điểm nổi bật là việc sử dụng các cảm biến môi trường và giao thức truyền thông để tạo ra một mô hình thử nghiệm, minh chứng cho tính khả thi của việc áp dụng IoT trong điều kiện nội địa. Tuy nhiên, nghiên cứu này cũng gặp phải những hạn chế tương tự, bao gồm việc kết nối còn mang tính thủ công và thiếu các kịch bản điều khiển phức tạp, đồng thời mô hình chỉ dừng ở giai đoạn mô phỏng mà chưa được triển khai rộng rãi trong thực tế.

Nhìn chung, các nghiên cứu trong nước đã cho thấy sự phát triển tích cực của các hệ thống nhà thông minh dựa trên IoT với nhiều giải pháp thiết thực đã được triển khai. Tuy nhiên, vẫn còn những thách thức đáng kể về khả năng tự động hóa hoàn toàn, sự đa dạng và linh hoạt trong kết nối không dây, cũng như việc tối ưu hóa các kịch bản điều khiển phức tạp để đáp ứng toàn diện nhu cầu của người dùng.

1.4. PHƯƠNG PHÁP NGHIÊN CỨU

Để thực hiện đề tài thiết kế hệ thống nhà thông minh, quá trình nghiên cứu được tiến hành một cách có hệ thống, bắt đầu bằng việc thiết kế kiến trúc hệ thống theo mô hình phân tán với các khối chính, xác định luồng dữ liệu và giao thức giao tiếp giữa chúng, đồng thời xây dựng sơ đồ khối phần cứng để thể hiện kết nối tổng thể. Tiếp theo, lập trình được thực hiện để phát triển các chức năng cho từng khối trên nền tảng lập trình phù hợp, thiết lập giao tiếp hai chiều giữa các khối, kết nối

với nền tảng lưu trữ đám mây để hiển thị dữ liệu thời gian thực và lập trình cơ chế xác thực đa phương thức. Về giao diện, hệ thống được tích hợp với bảng điều khiển trực tuyến để theo dõi và điều khiển từ xa, đồng thời hiển thị thông tin tại chỗ trên màn hình để người dùng tương tác. Cuối cùng, quá trình thử nghiệm bao gồm kiểm tra từng thành phần riêng lẻ, tích hợp toàn bộ hệ thống để kiểm tra đồng bộ, và đo lường các thông số như độ ổn định, độ trễ truyền dữ liệu, cùng khả năng hoạt động trong môi trường thực tế. Các phương pháp này được thực hiện tuần tự và kết hợp chặt chẽ, đảm bảo hệ thống đạt hiệu quả và đáp ứng yêu cầu đề ra.

1.5. BỐ CỤC BÀI BÁO CÁO

Bố cục của bài báo cáo gồm những thành phần sau:

Chương 1: Tổng quan

Trình bày lý do chọn đề tài, mục tiêu nghiên cứu, phạm vi nghiên cứu, phương pháp nghiên cứu và bố cục tổng thể của báo cáo.

Chương 2: Cơ sở lý thuyết

Trình bày các kiến thức nền tảng liên quan đến đề tài, bao gồm khái niệm về hệ thống nhà thông minh, giao tiếp vi điều khiển, công nghệ kết nối không dây, nền tảng lưu trữ đám mây, cùng các thiết bị cảm biến và xác thực được sử dụng trong hệ thống.

Chương 3: Thiết kế hệ thống

Mô tả chức năng tổng thể của hệ thống, thiết kế phần cứng, phần mềm và giao diện giám sát, bao gồm sơ đồ khối, lưu đồ thuật toán và cách các thành phần tương tác với nhau để đảm bảo vận hành hiệu quả.

Chương 4: Kết quả thực nghiệm và đánh giá

Trình bày kết quả triển khai thực tế, bao gồm mô hình phần cứng, phần mềm vận hành, kết quả hiển thị dữ liệu trên giao diện và tại chỗ, cùng với đánh giá hiệu suất và độ tin cậy của hệ thống.

Chương 5: Kết luận và hướng phát triển

Dựa vào kết quả đạt được từ chương 4, đưa ra kết luận về những việc đã làm được và chưa làm được của đề tài. Từ đó đưa ra các hướng để cải thiện đề tài và nâng cao tính ứng dụng để đưa dự án vào thực tiễn.

Với bố cục gồm 5 chương, đề tài được triển khai một cách tuần tự và logic, đi từ cơ sở lý thuyết, thiết kế, thực nghiệm, đến đánh giá và định hướng phát triển. Cách trình bày chặt chẽ này đảm bảo tính rõ ràng, khoa học và thuận tiện cho quá trình nghiên cứu, triển khai, cũng như tham khảo sau này.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1. TỔNG QUAN VỀ HỆ THỐNG GIÁM NHÀ THÔNG MINH

Hệ thống nhà thông minh là một bước tiến công nghệ nhằm nâng cao chất lượng cuộc sống, tối ưu hóa hiệu quả năng lượng và tăng cường an ninh cho ngôi nhà. Theo tài liệu *Build Your Own Smart Home* của Elsenpeter và Velte (2003), nhà thông minh tích hợp các hệ thống tự động hóa để quản lý các chức năng như chiếu sáng, an ninh, điều hòa nhiệt độ, liên lạc và giải trí thông qua các thiết bị kết nối. Hệ thống này không chỉ mang lại sự tiện nghi mà còn tạo ra một môi trường sống thông minh, đáp ứng linh hoạt nhu cầu của gia đình [5].

Cốt lõi của nhà thông minh là các giao thức như X10, cho phép thiết bị giao tiếp qua hệ thống điện hiện có hoặc mạng không dây, giảm thiểu chi phí và độ phức tạp khi triển khai. Điều này giúp việc cài đặt trở nên đơn giản mà không cần thay đổi lớn về cơ sở hạ tầng. Các thành phần chính bao gồm:

- **Hệ thống an ninh:** Gồm cảm biến chuyển động, camera, cảm biến nước và khói, giúp giám sát và bảo vệ ngôi nhà khỏi các nguy cơ như trộm cắp, hỏa hoạn hoặc rò rỉ nước.
- **Hệ thống chiếu sáng:** Sử dụng mô-đun X10 để điều khiển đèn, hỗ trợ bật/tắt hoặc điều chỉnh độ sáng theo lịch trình hoặc từ xa.
- **Hệ thống HVAC:** Quản lý nhiệt độ và sưởi ấm, đảm bảo môi trường sống thoải mái và tiết kiệm năng lượng.
- **Hệ thống liên lạc:** Bao gồm mạng LAN và điện thoại, hỗ trợ kết nối nội bộ và bên ngoài.
- **Hệ thống giải trí:** Phân phối âm thanh và video đến nhiều khu vực trong nhà, nâng cao trải nghiệm giải trí.

Hệ thống nhà thông minh có thể được lập trình và điều khiển thông qua bảng điều khiển, máy tính hoặc thiết bị di động, mang lại sự linh hoạt và tiện lợi. Về chi phí, hệ thống này cho phép bắt đầu với các dự án nhỏ như mô-đun đèn X10 (chi phí 30-100 USD) và mở rộng dần theo nhu cầu và ngân sách. Việc sử dụng các tiêu chuẩn như cáp Cat 5 hoặc RG-6 đảm bảo khả năng tương thích và dễ dàng

nâng cấp. Ngoài ra, nhà thông minh không chỉ cải thiện chất lượng cuộc sống mà còn tăng giá trị bất động sản nhờ tích hợp công nghệ hiện đại.

2.2. KIẾN TRÚC HỆ THỐNG MASTER - SLAVE TRONG NHÀ THÔNG MINH

2.2.1. Nguyên lý hoạt động Master – Slave

Kiến trúc Master – Slave là một mô hình tổ chức hệ thống trong đó các node được phân tách vai trò rõ ràng để tối ưu hóa hiệu quả xử lý và quản lý dữ liệu. Trong mô hình này, node Master đóng vai trò trung tâm, điều phối các tác vụ ưu tiên cao, trong khi node Slave thực hiện các tác vụ hỗ trợ, hoạt động độc lập và đồng bộ hóa thông qua một nền tảng trung gian. Mô hình này được thiết kế để đảm bảo tính linh hoạt, giảm tải xử lý và hỗ trợ mở rộng trong các hệ thống IoT, đặc biệt trong các ứng dụng nhà thông minh.

Cốt lõi của nguyên lý Master – Slave nằm ở việc phân chia chức năng theo mức độ ưu tiên và độc lập hóa hoạt động của các node. Node Master chịu trách nhiệm quản lý các tác vụ yêu cầu ra quyết định, chẳng hạn như xử lý xác thực an ninh hoặc điều khiển các cơ cấu quan trọng. Ngược lại, node Slave tập trung vào thu thập và xử lý dữ liệu từ môi trường hoặc điều khiển các thiết bị đơn giản. Thay vì giao tiếp trực tiếp, Master và Slave sử dụng một nền tảng đám mây làm trung gian, với mỗi node kết nối qua WiFi để gửi và nhận dữ liệu.

Mô hình Master-Slave mang lại lợi ích cốt lõi là khả năng mở rộng và giảm độ phức tạp trong quản lý hệ thống. Bằng cách cho phép các node hoạt động độc lập và đồng bộ qua đám mây, kiến trúc này hỗ trợ thêm nhiều node Slave mà không làm tăng tải cho Master, đồng thời giảm yêu cầu về phần cứng giao tiếp trực tiếp. Tuy nhiên, thách thức chính là sự phụ thuộc vào kết nối mạng ổn định để đảm bảo đồng bộ dữ liệu. Giải pháp là sử dụng cơ sở dữ liệu thời gian thực với cơ chế lưu trữ tạm thời (caching) để duy trì tính toàn vẹn dữ liệu khi mạng gián đoạn. Nguyên lý Master-Slave vì thế là nền tảng lý thuyết quan trọng, cung cấp cơ sở cho việc thiết kế các hệ thống IoT hiệu quả và đáng tin cậy.

2.2.2. Vi điều khiển ESP32



Hình 2.1: Vi điều khiển ESP32-S

Hình 2.1 là vi điều khiển ESP32 có tích hợp khả năng giao tiếp không dây, được dùng để kết nối mạng và truyền dữ liệu thời gian thực trong hệ thống giám sát nhà thông minh [6]. Với lõi xử lý ESP32-D0WDQ6 với hai nhân Xtensa 32-bit LX6 hoạt động ở tần số tối đa là 240MHz. ESP32 hỗ trợ xử lý đa nhiệm hiệu quả, cho phép thực hiện đồng thời các tác vụ như thu thập dữ liệu và gửi yêu cầu mạng mà không gây chậm trễ. Bộ nhớ flash 4MB và SRAM 520KB đáp ứng tốt nhu cầu lưu trữ chương trình và dữ liệu tạm thời, đảm bảo cho hệ thống hoạt động mượt mà. Điểm nổi bật khác đó là khả năng kết nối Wi-Fi theo chuẩn 802.11 b/g/n, cho phép truyền dữ liệu an toàn qua giao thức HTTPS đến nền tảng đám mây như Firebase, hỗ trợ giám sát và điều khiển từ xa. Ngoài ra, ESP32 cũng hỗ trợ các giao thức như UART, I2C và SPI để kết nối với các module khác.

Để có cái nhìn tổng quan hơn về các thông số kỹ thuật của ESP32, thì dưới đây là Bảng 2.1 tóm tắt thông số kỹ thuật của vi điều khiển này.

Bảng 2.1: Bảng thông số kỹ thuật chính của ESP32-S

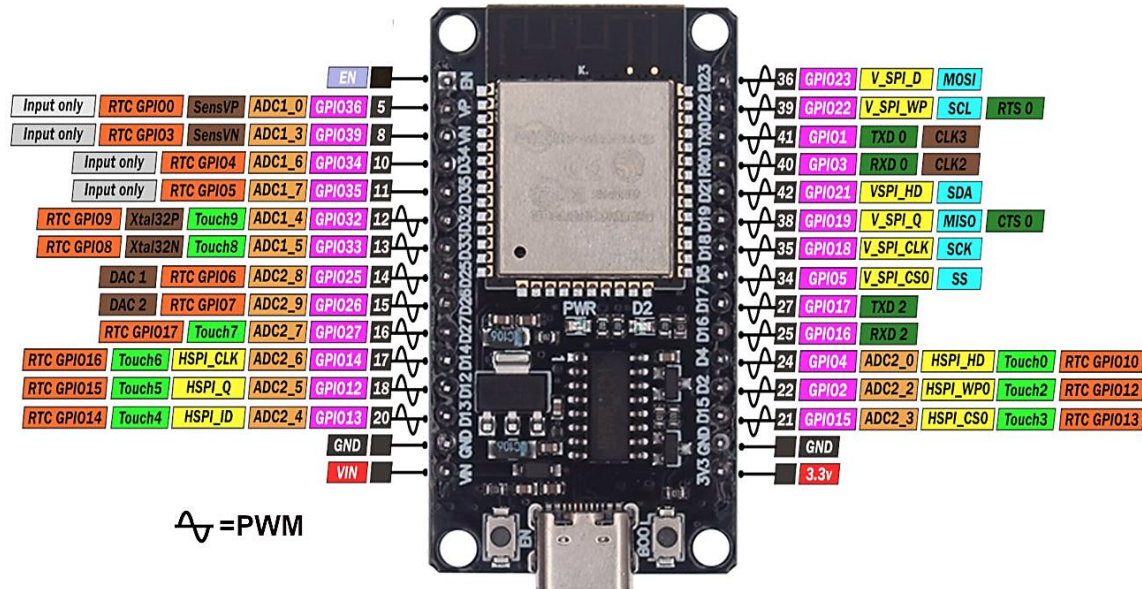
STT	Thông số	Mô tả
1	Kiến trúc	Dual-core Tensilica Xtensa LX6
2	Tần số hoạt động	Tối đa 240MHz
3	Bộ nhớ flash	4MB
4	Bộ nhớ SRAM	520KB

5	Giao thức hỗ trợ	UART, I2C, SPI, ADC, DAC, PWM
6	Kết nối không dây	Wi-Fi 802.11 b/g/n (2.4GHz), Bluetooth v4.2 BR/EDR & BLE.
7	Số chân GPIO	Có 38 GPIO
8	Nguồn hoạt động	3V

Hình 2.2, mô tả sơ đồ trực quan của vi điều khiển này, mỗi chân GPIO trên ESP32 có thể hoạt động đa chức năng không chỉ là hoạt động như ngõ vào và ngõ ra như thông thường mà còn có thể cấu hình để đảm nhiệm nhiều vai trò chức năng khác nhau như:

- Ngõ vào (Input) và ngõ ra (Output): Cấu hình làm ngõ vào và ngõ ra để đọc trạng thái mức HIGH và LOW hoặc điều khiển thiết bị.
- ADC: Một số chân có khả năng đọc tín hiệu analog từ cảm biến và chuyển đổi sang tín hiệu số.
- DAC: Một số chân có thể tạo ra tín hiệu analog sau đó chuyển qua tín hiệu số.
- PWM: Hầu hết các chân GPIO đều có thể dùng để tạo tín hiệu PWM, rất hiệu quả trong việc điều khiển động cơ.
- UART: Giao tiếp truyền thông nối tiếp không đồng bộ với các thiết bị khác.
- I2C: Giao tiếp hai dây, thường dùng để kết nối với màn hình LCD.
- SPI: Giao tiếp nối tiếp tốc độ cao, thường dùng cho RFID, màn hình TFT, thẻ nhớ hoặc các module khác.
- Cảm biến chạm (Touch Pins): Một số chân có hỗ trợ tính năng cảm ứng, cho phép phát hiện sự thay đổi khi có sự tiếp xúc, hữu ích cho việc tạo các nút cảm ứng.

- GPIO của RTC (Real-Time Clock): Một số chân có thể được sử dụng khi ESP32 ở chế độ tiết kiệm năng lượng (Deep Sleep), giúp duy trì trạng thái hoặc kích hoạt chip khi từ trạng thái ngủ.



Hình 2.2: ESP32 và chức năng cơ bản của từng chân

ESP32 thường được lập trình bằng môi trường Arduino IDE hoặc ESP-IDF (Espressif IoT Development Framework), với sự hỗ trợ của nhiều thư viện để đơn giản hóa việc phát triển. Một số thư viện phổ biến bao gồm:

- **WiFi.h**: Thư viện này cho phép ESP32 kết nối với mạng Wi-Fi.
- **HTTPClient.h**: Dùng để gửi các yêu cầu HTTP/HTTPS đến máy chủ đám mây như Firebase.
- **Wire.h** và **SPI.h**: Hỗ trợ giao tiếp I2C và SPI với các cảm biến hoặc mô-đun. Ví dụ, sử dụng Wire.h để đọc dữ liệu từ cảm biến I2C...

Cách sử dụng: Để sử dụng các thư viện này, trước tiên cần cài đặt Arduino IDE hoặc ESP-IDF, sau đó thêm các thư viện thông qua trình quản lý thư viện (Library Manager) hoặc tải từ GitHub. Khi lập trình, bạn cần khai báo thư viện, khởi tạo các đối tượng cần thiết, và cấu hình các chân GPIO hoặc giao thức phù hợp với ứng dụng. Ví dụ, để điều khiển đèn qua Wi-Fi, bạn có thể kết hợp **WiFi.h** để kết nối mạng và **digitalWrite** để điều khiển chân GPIO. Các thư viện này giúp

ESP32 dễ dàng tích hợp vào hệ thống nhà thông minh, từ việc giám sát cảm biến đến điều khiển thiết bị từ xa thông qua các nền tảng như Firebase hoặc HomeSeer.

2.3. CÁC MODULE CẢM BIẾN, ĐIỀU KHIỂN, HIỂN THỊ

2.3.1. Cảm biến nhiệt độ, độ ẩm

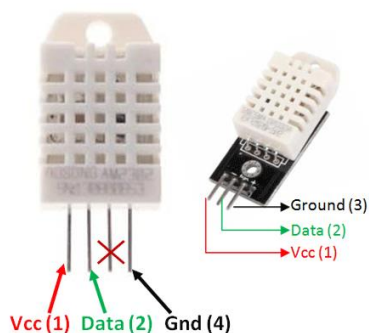
DTH22 là một cảm biến kỹ thuật số được thiết kế để đo nhiệt độ và độ ẩm, phổ biến trong các dự án IoT và hệ thống giám sát môi trường và được sản xuất bởi Aosong Electronics [7]. Cảm biến này sử dụng một thermistor để đo nhiệt độ và một cảm biến độ ẩm điện dung để đo độ ẩm, kết hợp với mạch xử lý tích hợp để chuyển đổi tín hiệu analog thành tín hiệu số. Dữ liệu được truyền qua giao thức giao tiếp một dây (single-wire protocol), cho phép kết nối đơn giản với các vi điều khiển hoặc hệ thống xử lý tín hiệu.

Các thông số kỹ thuật chi tiết được trình bày trong Bảng 2.2 sau đây, cung cấp cái nhìn tổng quan về đặc điểm kỹ thuật của DHT22:

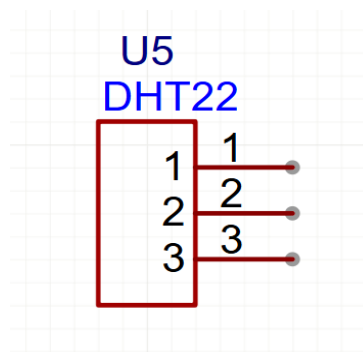
Bảng 2.2: Thông số kỹ thuật DHT22

STT	Thông số	Chi tiết
1	Phạm vi đo nhiệt độ	-40°C đến 80°C
2	Độ chính xác nhiệt độ	$\pm 0.5^{\circ}\text{C}$
3	Phạm vi đo độ ẩm	0% đến 100% RH (Relative Humidity)
4	Độ chính xác độ ẩm	$\pm 2\% \text{ RH}$
5	Dòng tiêu thụ	Trung bình 1mA, tối đa 2.5mA
6	Thời gian phản hồi	2 giây
7	Giao tiếp	Giao thức một dây (single-bus)
8	Nguồn	3.3V đến 5V

DHT22 được thiết kế với 4 chân, trong đó 1 chân là không sử dụng (NC-No Connection). Sơ đồ chân của cảm biến được minh họa như hình sau:



Hình 2.3: DHT22 và vị trí của từng chân



Hình 2.4: Hình ảnh ký hiệu DHT22

Hình 2.3, giúp người dùng dễ dàng nhận biết được vị trí và chức năng của từng chân để kết nối với vi điều khiển. Còn hình 2.4, minh họa về ký hiệu của DHT22 trong phần mềm mô phỏng. Chi tiết về tên và chức năng của các chân được trình bày trong bảng sau:

Bảng 2.3: Tên và chức năng từng chân DHT22

STT	Tên	Chức năng
1	VCC	3.3V đến 5V
2	GND	Nối với mass (đất)
3	DATA	Chân dữ liệu
4	NC	Không kết nối

Dựa trên Bảng 2.3, có thể thấy mỗi chân của DHT22 đảm nhiệm một vai trò cụ thể trong việc vận hành và kết nối cảm biến. Chân VCC cung cấp nguồn điện cho cảm biến, hoạt động ổn định với điện áp từ 3.3V đến 5V, thường được khuyến nghị sử dụng 5V để đảm bảo hiệu suất tối ưu. Chân DATA chịu trách nhiệm truyền dữ liệu số (nhiệt độ và độ ẩm) dưới dạng tín hiệu xung theo giao thức một dây, cần kết nối thêm một điện trở kéo lên (pull-up resistor) từ 4.7kΩ đến 10kΩ với nguồn VCC để đảm bảo tín hiệu ổn định. Chân GND đóng vai trò tham chiếu điện áp, được nối với điểm chung của hệ thống. Chân NC không được sử dụng trong các kết nối thông thường.

Thư viện sử dụng cho DHT22. Để giao tiếp với cảm biến DHT22 trên ESP32, thư viện **DHT.h** từ Adafruit là lựa chọn phổ biến nhất, thường được sử dụng cùng với thư viện **Adafruit_Sensor** để đảm bảo tương thích. Các thư viện này có thể được cài đặt thông qua Arduino IDE Library Manager hoặc từ kho GitHub của Adafruit. Thư viện **DHT.h** hỗ trợ cả DHT11 và DHT22, giúp đơn giản hóa việc đọc dữ liệu nhiệt độ và độ ẩm.

2.3.2. Cảm biến cường độ ánh sáng

Cảm biến cường độ ánh sáng là một linh kiện được thiết kế để đo mức độ ánh sáng trong môi trường, thường được dùng trong các ứng dụng cần điều chỉnh ánh sáng hoặc giám sát điều kiện chiếu sáng. Điển hình là cảm biến BH1750, một cảm biến kỹ thuật số được phát triển bởi ROHM Semiconducto [8], nổi bật nhờ khả năng đo ánh sáng chính xác và tích hợp dễ dàng với các vi điều khiển.

BH1750 sử dụng một cảm biến quang nhạy với ánh sáng khả kiến, có khả năng đo cường độ ánh sáng trong dải từ 0 đến 65.535 lux, tương đương với ánh sáng từ ánh trăng mờ đến ánh sáng mặt trời trực tiếp. Cảm biến này chuyển đổi tín hiệu ánh sáng thành tín hiệu số thông qua bộ chuyển đổi analog sang số và truyền dữ liệu qua giao thức I2C.

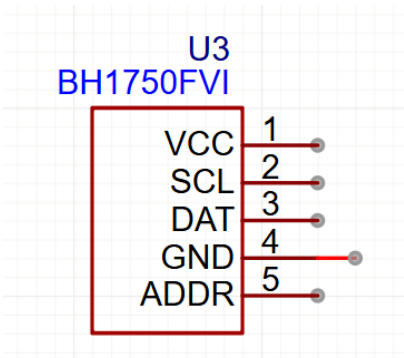
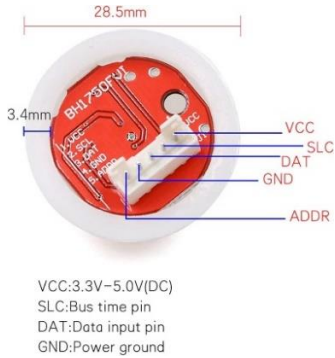
Các thông số kỹ thuật chi tiết được trình bày trong Bảng 2.4 sau đây, cung cấp cái nhìn tổng quan về đặc điểm kỹ thuật của BH1750:

Bảng 2.4: Thông số kỹ thuật BH1750

STT	Thông số	Chi tiết
1	Phạm vi đo	0 đến 65535 lux
2	Độ phân giải	1 lux
3	Độ chính xác	$\pm 3\%$ trong điều kiện ánh sáng ổn định.
4	Nguồn	3.3V
5	Dòng tiêu thụ	Trung bình 120 μA , chế độ ngủ 0.1 μA
6	Thời gian	24ms đến 400ms

7	Giao tiếp	I2C, địa chỉ mặc định 0x23.
---	-----------	-----------------------------

BH1750 được thiết kế với 5 chân, trong đó một số chân đảm nhiệm các vai trò cụ thể để vận hành và kết nối cảm biến. Sơ đồ chân của cảm biến được minh họa như hình sau:



Hình 2.5: BH1750 và vị trí từng chân Hình 2.6: Hình ảnh ký hiệu BH1750

Hình 2.5, giúp người dùng dễ dàng nhận biết được vị trí và chức năng của từng chân để kết nối với vi điều khiển. Còn hình 2.6, minh họa về ký hiệu của BH1750 trong phần mềm mô phỏng. Chi tiết về tên và chức năng của các chân được trình bày trong bảng sau:

Bảng 2.5: Tên và chức năng từng chân BH1750

STT	Tên	Chức năng
1	VCC	3.3V
2	GND	Nối với mass (đất)
3	SCL	Chân xung Clock I2C
4	SDA	Chân dữ liệu I2C
5	ADDR	Chân chọn địa chỉ I2C, nối với GND (0x23) hoặc VCC (0x5C).

Dựa trên Bảng 2.5, có thể thấy mỗi chân của BH1750 đảm nhiệm một vai trò cụ thể trong việc vận hành và kết nối cảm biến. Chân VCC cung cấp nguồn 3.3V để cảm biến hoạt động ổn định.. Chân GND đóng vai trò là tham chiếu điện áp, được nối với điểm chung của hệ thống. Chân SCL và SDA là các chân giao tiếp I2C, lần lượt đảm nhiệm vai trò truyền xung Clock và dữ liệu số, cần kết nối

thêm điện trở kéo lên (pull-up) với nguồn VCC để đảm bảo tín hiệu ổn định, Chân ADDR cho phép thay đổi địa chỉ I2C của cảm biến.

Để giao tiếp với cảm biến BH1750 trên ESP32, thư viện **BH1750** là lựa chọn phổ biến, thường được sử dụng cùng với thư viện **Wire.h** (tích hợp sẵn trong Arduino IDE) để hỗ trợ giao tiếp I2C. Thư viện này đơn giản hóa việc đọc dữ liệu ánh sáng và cung cấp các chế độ đo như độ phân giải cao (High Resolution) hoặc độ phân giải thấp (Low Resolution).

2.3.3. Cảm biến khí GAS

Cảm biến khí gas là một loại linh kiện được thiết kế để phát hiện và đo các loại khí khác nhau trong môi trường, thường được sử dụng trong các ứng dụng giám sát chất lượng không khí hoặc phát hiện rò rỉ khí. Điển hình là cảm biến MQ-2, được sản xuất bởi Hanwei Electronics [9], nổi bật với khả năng phát hiện nhiều loại khí như metan (CH_4), khí butan (C_4H_{10}), LPG (gas hóa lỏng) và khói.

Các thông số kỹ thuật chi tiết được trình bày trong Bảng 2.6 sau đây, cung cấp cái nhìn tổng quan về đặc điểm kỹ thuật của MQ-2:

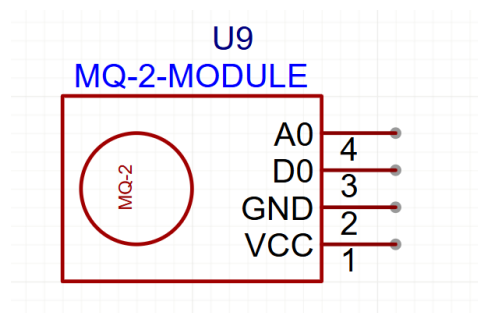
Bảng 2.6: Thông số kỹ thuật MQ-2

STT	Thông số	Chi tiết
1	Phạm vi đo	200 đến 10000 ppm
2	Độ nhạy	$\pm 5\%$
3	Nguồn	5V
4	Dòng tiêu thụ	150 mA.
5	Thời gian phản hồi	Dưới 10 giây
6	Giao tiếp	Analog (qua ADC), không cần giao thức số (Digital).

MQ-2 được thiết kế với 4 chân, trong đó một số chân đảm nhiệm các vai trò cụ thể để vận hành và kết nối cảm biến. Sơ đồ chân của cảm biến được minh họa như hình sau:



Hình 2.7: MQ-2 và vị trí từng chân



Hình 2.8: Hình ảnh ký hiệu MQ-2

Hình 2.7, giúp người dùng dễ dàng nhận biết được vị trí và chức năng của từng chân để kết nối với vi điều khiển. Còn hình 2.8, minh họa về ký hiệu của MQ – 2 trong phần mềm mô phỏng. Chi tiết về tên và chức năng của các chân được trình bày trong bảng sau:

Bảng 2.7: Tên và chức năng từng chân MQ-2

STT	Tên	Chức năng
1	VCC	5V
2	GND	Nối với mass
3	AO	Chân tín hiệu Analog
4	DO	Chân tín hiệu Digital

Dựa trên Bảng 2.7, có thể thấy mỗi chân của MQ-2 đảm nhiệm một vai trò cụ thể trong việc vận hành và kết nối cảm biến. Chân VCC cung cấp nguồn 5V để hoạt động ổn định. Chân GND đóng vai trò là tham chiếu điện áp, được nối với điểm chung của hệ thống. Chân AO cung cấp tín hiệu analog tương ứng với nồng độ khí, có thể được kết nối với chân ADC của vi điều khiển để đọc giá trị. Chân DO là đầu ra kỹ thuật số, thường được điều chỉnh bởi một biến trở để thiết lập ngưỡng khí.

Để giao tiếp với cảm biến MQ2 trên ESP32, có thể sử dụng các thư viện sau:

- **Không cần thư viện đặc biệt (dùng ADC tích hợp):** Đối với các ứng dụng đơn giản, bạn có thể đọc trực tiếp tín hiệu analog từ chân AO của MQ2 bằng hàm `analogRead()` của Arduino IDE, không cần thư viện bổ sung.
- **MQUnifiedsensor:** Đây là thư viện chuyên dụng cho các cảm biến MQ (bao gồm MQ2), hỗ trợ tính toán nồng độ khí (ppm) dựa trên đặc tính của cảm biến. Thư viện này cung cấp các phương pháp hiệu chỉnh (calibration) và chuyển đổi tín hiệu analog thành nồng độ khí cụ thể (LPG, CO, khói, v.v.). Thư viện có thể được tải từ GitHub: MQUnifiedsensor Library.

Blynk hoặc WiFi.h (kèm HTTPClient.h): Nếu tích hợp MQ2 với hệ thống IoT (như Blynk hoặc Firebase), các thư viện **BlynkSimpleEsp32.h** hoặc **WiFi.h** và **HTTPClient.h** được sử dụng để gửi dữ liệu khí đến nền tảng đám mây.

2.3.4. Module RFID RC522

Module RFID RC522 là một linh kiện được thiết kế để thực hiện giao tiếp không dây dựa trên công nghệ RFID, thường được sử dụng để đọc và ghi dữ liệu trên thẻ hoặc thẻ từ thông qua sóng radio. Module này sử dụng chip MFRC522 do NXP Semiconductors sản xuất [10], phù hợp cho các ứng dụng như kiểm soát truy cập hoặc quản lý tài sản. MFRC522 hoạt động ở tần số 13.56MHz, sử dụng giao tiếp SPI để kết nối với các vi điều khiển.

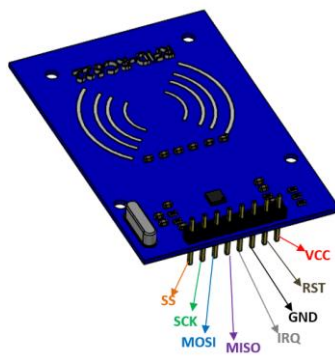
Các thông số kỹ thuật chi tiết được trình bày trong Bảng 2.8 sau đây, cung cấp cái nhìn tổng quan về đặc điểm kỹ thuật của RFID:

Bảng 2.8: Thông số kỹ thuật RFID

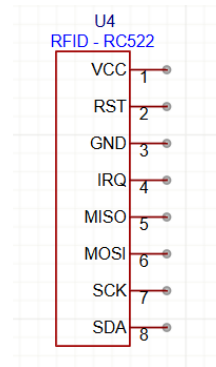
STT	Thông số	Chi tiết
1	Tần số hoạt động	13.56 MHz
2	Khoảng cách đọc	Tối đa 5 cm.
3	Nguồn	3.3V
4	Dòng tiêu thụ	13 - 26 mA.
5	Hỗ trợ thẻ	Thẻ MIFARE 1K, 4K và thẻ không tiếp xúc

6	Giao tiếp	SPI
---	-----------	-----

RFID RC522 được thiết kế với 8 chân, trong đó một số chân đảm nhiệm các vai trò cụ thể để vận hành và kết nối cảm biến. Sơ đồ chân của cảm biến được minh họa như hình sau:



Hình 2.9: RFID và vị trí từng chân



Hình 2.10: Hình ảnh ký hiệu RFID

Hình 2.9, giúp người dùng dễ dàng nhận biết được vị trí và chức năng của từng chân để kết nối với vi điều khiển. Còn hình 2.10, minh họa về ký hiệu của RFID trong phần mềm mô phỏng. Chi tiết về tên và chức năng của các chân được trình bày trong bảng sau:

Bảng 2.9: Tên và chức năng từng chân RFID

STT	Tên	Chức năng
1	VCC	3.3V
2	GND	Nối với mass (đất)
3	RST	Chân reset
4	IRQ	Chân ngắt
5	MISO	Chân dữ liệu vào
6	MOSI	Chân dữ liệu ra
7	SCK	Chân xung Clock
8	SS	Chân chọn chip

Dựa trên Bảng 2.9, có thể thấy mỗi chân của RFID đảm nhiệm một vai trò cụ thể trong việc vận hành và kết nối cảm biến. Chân VCC cung cấp nguồn 3.3V cho module, đảm bảo hoạt động ổn định. Chân GND đóng vai trò tham chiếu điện áp, được nối với điểm chung của hệ thống. Chân SCK, MISO, MOSI và SS là các chân giao tiếp SPI, lần lượt chịu trách nhiệm truyền xung nhịp, dữ liệu ra, dữ liệu vào và chọn chip. Chân RST được dùng để khởi động lại module khi cần, trong khi chân IRQ hỗ trợ ngắt để thông báo khi đọc thẻ, tuy nhiên chân này thường không được sử dụng nếu không cần thiết.

Để giao tiếp với module RFID RC522 trên ESP32, thư viện **MFRC522** là lựa chọn phổ biến nhất, hỗ trợ đọc và ghi dữ liệu từ thẻ RFID hoặc NFC. Thư viện này hoạt động với giao thức SPI và tương thích với Arduino IDE. Thư viện **MFRC522** là một công cụ mạnh mẽ và phổ biến để giao tiếp với mô-đun RFID RC522 trên các nền tảng như ESP32, Arduino, hoặc các vi điều khiển khác, đặc biệt trong các ứng dụng nhà thông minh như kiểm soát truy cập hoặc quản lý thiết bị. Thư viện này sử dụng giao thức SPI để đọc/ghi dữ liệu từ thẻ RFID/NFC hoạt động ở tần số 13.56 MHz, hỗ trợ các ứng dụng an ninh và tự động hóa.

2.3.5. Cảm biến vân tay AS608

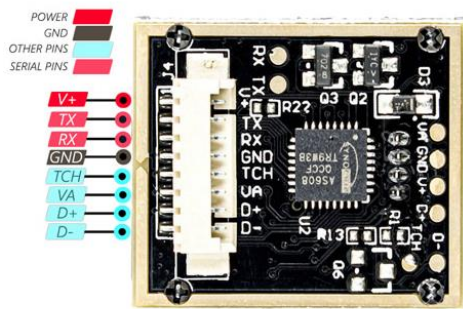
Cảm biến vân tay AS608 là một linh kiện được thiết kế để nhận diện và xác thực vân tay, thường được sử dụng trong các ứng dụng bảo mật như kiểm soát truy cập hoặc xác minh danh tính. Module này sử dụng chip AS608 do Alientek phát triển [11], tích hợp cảm biến quang học (optical sensor) để quét hình ảnh vân tay với độ phân giải cao. AS608 có khả năng lưu trữ và so sánh dấu vân tay thông qua thuật toán xử lý hình ảnh và nhận dạng mẫu tích hợp. Module này giao tiếp với vi điều khiển qua giao thức UART, cho phép truyền dữ liệu nhanh chóng và dễ dàng tích hợp vào các hệ thống nhúng. AS608 cũng đi kèm với bộ nhớ trong, có thể lưu trữ đến 300 dấu vân tay, phù hợp cho các ứng dụng quy mô nhỏ đến trung bình.

Các thông số kỹ thuật chi tiết được trình bày trong Bảng 2.10 sau đây, cung cấp cái nhìn tổng quan về đặc điểm kỹ thuật của AS608:

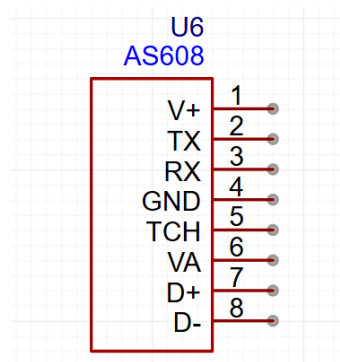
Bảng 2.10: Thông số kỹ thuật AS608

STT	Thông số	Chi tiết
1	Dung lượng lưu trữ	13.56 MHz
2	Tỷ lệ nhận diện sai (FAR)	Dưới 0.001%
3	Tỷ lệ không nhận diện (FRR)	Dưới 1%
4	Nguồn	3.3V
5	Dòng tiêu thụ	80 mA (khi quét), 20 mA (khi nghỉ)
6	Giao tiếp	UART, baud rate 57600 bps
7	Thời gian phản hồi	Dưới 1 giây

AS608 được thiết kế với 6 chân, trong đó một số chân đảm nhiệm các vai trò cụ thể để vận hành và kết nối cảm biến. Sơ đồ chân của cảm biến được minh họa như hình sau:



Hình 2.11: AS608 và vị trí từng chân



Hình 2.12: Hình ảnh ký hiệu AS608

Hình 2.11, giúp người dùng dễ dàng nhận biết được vị trí và chức năng của từng chân để kết nối với vi điều khiển. Còn hình 2.12, minh họa về ký hiệu của AS608 trong phần mềm mô phỏng. Chi tiết về tên và chức năng của các chân được trình bày trong bảng sau:

Bảng 2.11: Tên và chức năng từng chân AS608

STT	Tên	Chức năng
1	VCC	3.3V
2	GND	Nối với mass (đất)
3	TX	Chân giao tiếp UART

4	RX	Chân giao tiếp UART
5	TCH	Chân output của cảm biến chạm Touch
6	VA	Chân cấp nguồn 3.3V cho Touch Sensor
7	D+	Chân tín hiệu USB+
8	D-	Chân tín hiệu USB-

Dựa trên Bảng 2.11, có thể thấy mỗi chân của AS608 đảm nhiệm một vai trò cụ thể trong việc vận hành và kết nối cảm biến. Chân VCC cung cấp nguồn điện 3.3V cho module, đảm bảo hoạt động ổn định. Chân GND đóng vai trò tham chiếu điện áp, được nối với điểm chung của hệ thống. Chân TX và RX là các chân giao tiếp UART, lần lượt chịu trách nhiệm truyền và nhận dữ liệu giữa cảm biến và vi điều khiển, cần được kết nối với các chân UART tương ứng (RX và TX) trên vi điều khiển, đảm bảo tốc độ baud rate phù hợp (mặc định là 57600 bps).

Thư viện phổ biến nhất để giao tiếp với cảm biến vân tay AS608 là **Adafruit_Fingerprint** (Adafruit Fingerprint Sensor Library), được phát triển bởi Adafruit. Thư viện này hỗ trợ các chức năng như đăng ký (enroll), nhận dạng (fingerprint matching), xóa vân tay và quản lý dữ liệu vân tay trên module AS608.

2.3.6. Keypad 4x4

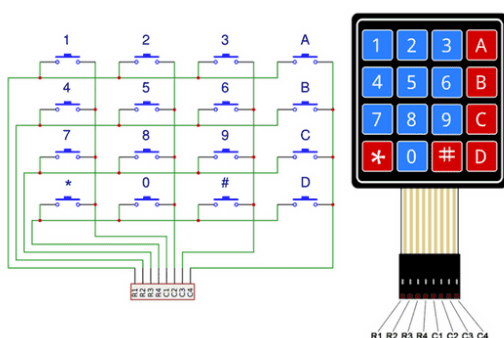
Keypad 4x4 là một module đầu vào dạng bàn phím ma trận, được thiết kế để cung cấp giao diện nhập dữ liệu thủ công cho các hệ thống điện tử nhúng. Module này bao gồm 16 phím được sắp xếp trong một ma trận 4 hàng và 4 cột, thường được đánh số từ 0 đến 9 cùng với các ký hiệu như *, #, A, B, C, D, phù hợp cho các ứng dụng cần nhập mã hoặc lệnh. Keypad 4x4 hoạt động dựa trên nguyên lý đóng/mở công tắc khi phím được nhấn, tạo ra kết nối giữa các hàng và cột tương ứng, từ đó vi điều khiển có thể quét và xác định phím nào được nhấn thông qua giao tiếp kỹ thuật số. Module này thường được làm từ vật liệu bền như nhựa hoặc silicon, đảm bảo độ tin cậy trong các điều kiện sử dụng thông thường.

Các thông số kỹ thuật chi tiết được trình bày trong Bảng 2.12 sau đây, cung cấp cái nhìn tổng quan về đặc điểm kỹ thuật của Keypad 4x4:

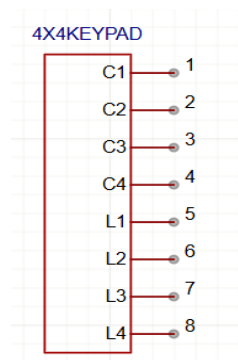
Bảng 2.12: Thông số kỹ thuật Keypad 4x4

STT	Thông số	Chi tiết
1	Số phím	16 phím (0-9, A-D, *, #)
2	Dòng tiêu thụ	Dưới 1 mA
3	Giao tiếp	GPIO
4	Kích thước	70 mm x 77 mm

Keypad 4x4 được thiết kế với 8 chân, trong đó một số chân đảm nhiệm các vai trò cụ thể để vận hành và kết nối. Sơ đồ chân của bàn phím được minh họa như hình sau:



Hình 2.13: Keypad 4x4 và vị trí từng chân



Hình 2.14: Hình ảnh ký hiệu Keypad

Hình 2.13, giúp người dùng dễ dàng nhận biết được vị trí và chức năng của từng chân để kết nối với vi điều khiển. Còn hình 2.14, minh họa về ký hiệu của Keypad trong phần mềm mô phỏng. Chi tiết về tên và chức năng của các chân được trình bày trong bảng sau:

Bảng 2.13: Tên và chức năng từng chân Keypad 4x4

STT	Chân	Ký hiệu	Chức năng chân
1		R1	Hàng 1 được kết nối với các phím 1, 2, 3, A.

2	Chân hàng (Rows: R1 – R4)	R2	Hàng 2 được kết nối với các phím 4, 5, 6, B.
3		R3	Hàng 3 được kết nối với các phím 7, 8, 9, C.
4		R4	Hàng 4 được kết nối với các phím *, 0, #, D.
5	Chân cột (Columns: C1 – C4)	C1	Hàng 5 được kết nối với các phím 1, 4, 7, *.
6		C2	Hàng 6 được kết nối với các phím 2, 5, 8, 0.
7		C3	Hàng 7 được kết nối với các phím 3, 6, 9, #.
8		C4	Hàng 8 được kết nối với các phím A, B, C, D.

Dựa trên Bảng 2.13, có thể thấy các chân của Keypad 4x4 được chia thành hai nhóm chính: R1 đến R4 đại diện cho 4 hàng của ma trận, và C1 đến C4 đại diện cho 4 cột. Khi một phím được nhấn, nó tạo ra kết nối giữa một hàng và một cột tương ứng, cho phép vi điều khiển quét tuần tự các hàng và cột để xác định phím được kích hoạt. Các chân này cần được kết nối với các chân GPIO của vi điều khiển, thường kèm theo điện trở kéo lên (pull-up resistor) hoặc được cấu hình với điện trở nội tại của vi điều khiển để đảm bảo tín hiệu ổn định.

Thư viện phổ biến nhất để giao tiếp với bàn phím 4x4 trên ESP32 là **Keypad**. Thư viện này đơn giản hóa việc đọc các phím được nhấn bằng cách sử dụng kỹ thuật quét ma trận (matrix scanning), giúp xác định vị trí phím dựa trên giao điểm của hàng và cột.

2.3.7. Module Relay 4 kênh

Module Relay 4 kênh là một linh kiện được thiết kế để điều khiển các thiết bị điện thông qua tín hiệu số [12], thường được sử dụng trong các ứng dụng tự động hóa hoặc điều khiển từ xa. Module này bao gồm 4 relay (rơ-le) độc lập, mỗi relay hoạt động như một công tắc điện tử, cho phép bật/tắt các thiết bị điện áp cao

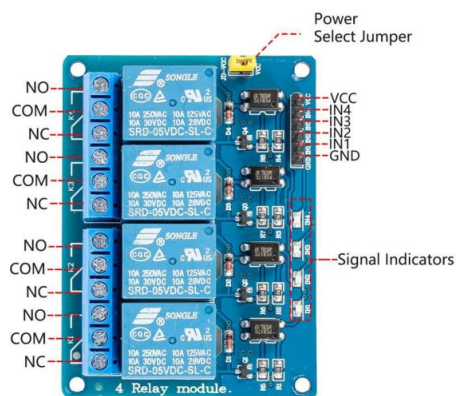
(như động cơ, đèn, hoặc quạt) bằng tín hiệu điện áp thấp từ vi điều khiển. Relay trên module thường là loại relay cơ điện (electromechanical relay), hoạt động dựa trên nguyên lý đóng/mở tiếp điểm nhờ từ trường tạo ra bởi cuộn dây bên trong khi có dòng điện chạy qua. Module Relay 4 kênh thường được tích hợp thêm các opto-coupler để cách ly tín hiệu điều khiển và mạch tải, đảm bảo an toàn và giảm nhiễu. Module này tương thích với các vi điều khiển thông qua tín hiệu đầu vào kỹ thuật số (thường là mức logic 3.3V hoặc 5V).

Các thông số kỹ thuật chi tiết được trình bày trong Bảng 2.14 sau đây, cung cấp cái nhìn tổng quan về đặc điểm kỹ thuật của Module Relay 4 kênh:

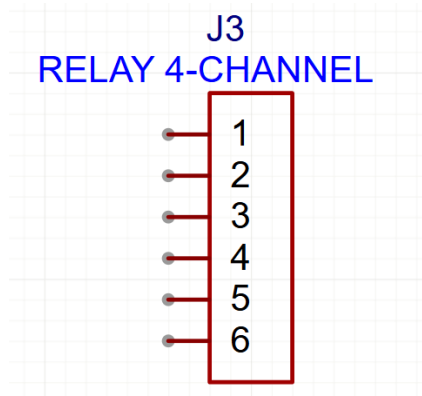
Bảng 2.14: Thông số kỹ thuật Module Relay

STT	Thông số	Chi tiết
1	Số kênh	4 kênh
2	Dòng tiêu thụ	20 mA
3	Giao tiếp	GPIO
4	Nguồn	5V
5	Thời gian phản hồi	Dưới 10 ms

Module Relay 4 kênh được thiết kế với các chân đầu vào điều khiển và các đầu ra tải riêng biệt. Phía đầu vào bao gồm 6 chân để cấp nguồn và điều khiển, trong khi phía đầu ra có 12 chân (3 chân cho mỗi kênh) để kết nối với tải. Sơ đồ chân của module được minh họa như hình sau:



Hình 2.15: Module Relay 4 kênh và vị trí từng chân



Hình 2.16: Hình ảnh ký hiệu Module Relay

Hình 2.15, giúp người dùng dễ dàng nhận biết vị trí và chức năng của từng chân để kết nối với vi điều khiển và thiết bị tải. Còn hình 2.16, minh họa về ký hiệu của Module Relay trong phần mềm mô phỏng. Chi tiết về tên và chức năng của các chân được trình bày trong bảng sau:

Bảng 2.15: Tên và chức năng từng chân Module Relay 4 kênh

STT	Tên	Chức năng
1	VCC	Chân cấp nguồn 5V
2	GND	Chân nối mass
3	IN1	Điều khiển Relay 1 (Tín hiệu logic High/Low)
4	IN2	Điều khiển Relay 2 (Tín hiệu logic High/Low)
5	IN3	Điều khiển Relay 3 (Tín hiệu logic High/Low)
6	IN4	Điều khiển Relay 4 (Tín hiệu logic High/Low)
7	COM1-4	Chân chung mỗi kênh (Kết nối tải)
8	NO1-4	Chân thường mở mỗi kênh
9	NC1-4	Chân thường đóng mỗi kênh

Dựa trên Bảng 2.15, có thể thấy các chân của Module Relay 4 kênh được chia thành hai nhóm chính: nhóm điều khiển và nhóm đầu ra tải. Chân VCC cung cấp nguồn 5V để vận hành module, trong khi chân GND đóng vai trò tham chiếu điện áp, được nối với điểm chung của hệ thống. Các chân IN1 đến IN4 là các chân điều khiển, nhận tín hiệu kỹ thuật số (mức logic 0 hoặc 1) từ vi điều khiển để kích hoạt hoặc tắt từng relay tương ứng; tín hiệu mức thấp (LOW) thường kích hoạt relay, và mức cao (HIGH) sẽ tắt relay. Phía đầu ra, mỗi kênh có 3 chân: COM (chân chung), NO (thường mở - Normally Open), và NC (thường đóng - Normally Closed), cho phép kết nối linh hoạt với tải tùy theo yêu cầu mạch (thường sử dụng cặp COM-NO để bật/tắt thiết bị).

2.3.8. Động cơ Servo

Động cơ Servo là một loại linh kiện cơ điện được thiết kế để điều khiển góc quay chính xác, thường được sử dụng trong các ứng dụng cần điều khiển vị trí như robot, mô hình điều khiển từ xa, hoặc cơ cấu tự động. Điển hình là động cơ SG90, một servo nhỏ gọn do TowerPro sản xuất [13], được ưa chuộng nhờ kích thước nhỏ, trọng lượng nhẹ, và khả năng điều khiển đơn giản. SG90 hoạt động dựa trên tín hiệu PWM (Pulse Width Modulation), trong đó độ rộng xung điều khiển góc quay của trục động cơ, thường trong khoảng từ 0° đến 180° . Động cơ này bao gồm một động cơ DC, bộ truyền động bánh răng, và mạch điều khiển tích hợp, cho phép quay chính xác và giữ vị trí ổn định khi không có tín hiệu thay đổi.

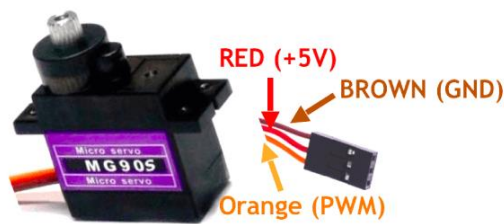
Các thông số kỹ thuật chi tiết được trình bày trong Bảng 2.16 sau đây, cung cấp cái nhìn tổng quan về đặc điểm kỹ thuật của SG90:

Bảng 2.16: Thông số kỹ thuật Servo

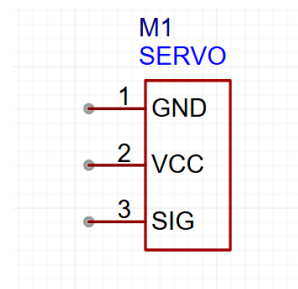
STT	Thông số	Chi tiết
1	Góc quay	0° đến 180°
2	Tốc độ	0.1 giây/ 60°
3	Mô-men xoắn	2.5 kg.cm
4	Nguồn	5V

5	Dòng tiêu thụ	10 mA (trạng thái nghỉ), 150 mA (trạng thái hoạt động)
6	Giao tiếp	PWM

Động cơ Servo SG90 được thiết kế với 3 dây kết nối, đảm nhiệm vai trò cấp nguồn và nhận tín hiệu điều khiển. Sơ đồ dây của động cơ được minh họa như hình sau:



Hình 2.17: Servo MG90S và vị trí từng chân



Hình 2.18: Hình ảnh ký hiệu Servo

Hình 2.17, giúp người dùng dễ dàng nhận biết vị trí và chức năng của từng dây để kết nối với các vi điều khiển. Còn hình 2.18, minh họa về ký hiệu của Servo trong phần mềm mô phỏng. Chi tiết về tên và chức năng của các dây được trình bày trong bảng sau:

Bảng 2.17: Tên và chức năng từng chân Servo

STT	Tên	Chức năng
1	Dây màu đỏ	Dây cấp nguồn 5V.
2	Dây màu nâu	Dây nối mass
3	Dây màu cam	Tín hiệu điều khiển PWM

Dựa trên Bảng 2.17, có thể thấy mỗi dây của SG90 đảm nhiệm một vai trò cụ thể trong việc vận hành động cơ. Dây nâu là dây nối đất (GND), được kết nối với điểm chung của hệ thống. Dây đỏ cung cấp nguồn điện cho động cơ, hoạt động ổn định trong khoảng từ 4.8V đến 6V, thường được kết nối với nguồn 5V từ vi điều khiển hoặc nguồn ngoài. Dây cam là dây tín hiệu, nhận tín hiệu PWM từ vi

điều khiển để điều khiển góc quay; tín hiệu PWM có chu kỳ 20ms (tần số 50Hz) và độ rộng xung từ 1ms đến 2ms tương ứng với góc quay từ 0° đến 180°.

2.3.9. Màn hình LCD 20X4 giao tiếp I2C

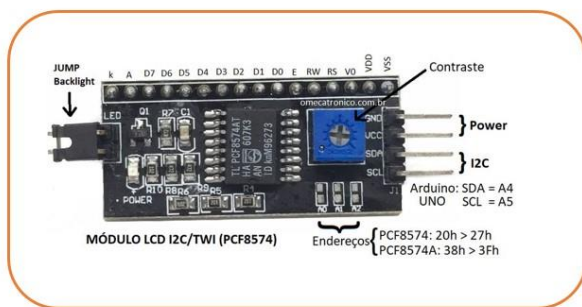
Màn hình LCD 20x4 giao tiếp I2C là một linh kiện hiển thị được thiết kế để trình bày thông tin dưới dạng văn bản, thường được sử dụng trong các ứng dụng nhúng để hiển thị dữ liệu hoặc trạng thái hệ thống. Màn hình này có kích thước 20 ký tự x 4 dòng, cho phép hiển thị tối đa 80 ký tự cùng lúc, phù hợp cho các ứng dụng cần hiển thị nhiều thông tin. Module giao tiếp I2C (thường sử dụng chip PCF8574) được tích hợp để giảm số lượng chân kết nối, cho phép giao tiếp với vi điều khiển qua giao thức I2C với chỉ 2 dây tín hiệu SDA và SCL [14]. Màn hình LCD 20x4 hoạt động dựa trên công nghệ tinh thể lỏng, với đèn nền LED tích hợp để hỗ trợ hiển thị trong điều kiện thiếu sáng, và một điện trở biến (potentiometer) để điều chỉnh độ tương phản.

Để hiểu rõ hơn về khả năng và hiệu suất của linh kiện này, các thông số kỹ thuật chính được trình bày trong Bảng 2.18 sau đây, cung cấp cái nhìn tổng quan về đặc điểm kỹ thuật của màn hình LCD 20x4 giao tiếp I2C:

Bảng 2.18: Thông số kỹ thuật LCD20x4

STT	Thông số	Chi tiết
1	Kích thước hiển thị	20 ký tự và 4 dòng
2	Nguồn cấp	5V
3	Dòng tiêu thụ	Khoảng 40 mA (với đèn nền)
4	Giao tiếp	I2C, địa chỉ mặc định 0x27

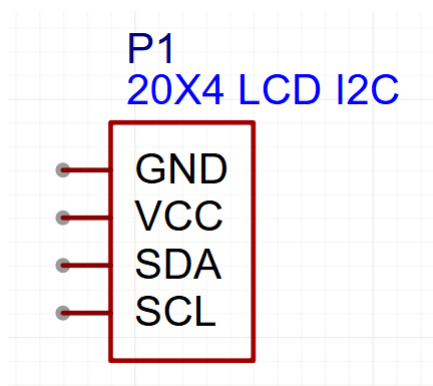
Màn hình LCD 20x4 giao tiếp I2C được thiết kế với 4 chân trên module I2C, đảm nhiệm vai trò cấp nguồn và giao tiếp. Sơ đồ chân của module được minh họa như hình sau:



Hình 2.19: Module I2C PC8574



Hình 2.20: LCD 20x4



Hình 2. 21: Hình ảnh ký hiệu LCD

Hình 2.19 và hình 2.20, giúp người dùng dễ dàng nhận biết vị trí và chức năng của từng chân để kết nối với các vi điều khiển. Còn hình 2.21, minh họa về ký hiệu của LCD trong phần mềm mô phỏng. Chi tiết về tên và chức năng của các chân được trình bày trong bảng sau:

Bảng 2.19: Tên và chức năng từng chân LCD20x4 I2C

STT	Tên	Chức năng
1	VCC	Chân cấp nguồn 5V
2	GND	Chân nối mass
3	SDA	Chân dữ liệu I2C
4	SCL	Chân tín hiệu xung Clock

Dựa trên Bảng 2.19, có thể thấy mỗi chân của module LCD 20x4 giao tiếp I2C đảm nhiệm một vai trò cụ thể trong việc vận hành và kết nối. Chân VCC cung cấp nguồn điện 5V để vận hành màn hình và module I2C. Chân GND đóng vai trò

tham chiếu điện áp, được nối với điểm chung của hệ thống. Chân SDA và SCL là các chân giao tiếp I2C, lần lượt chịu trách nhiệm truyền dữ liệu và xung nhịp, cần kết nối với các chân I2C tương ứng trên vi điều khiển (thường yêu cầu điện trở kéo lên từ 2.2kΩ đến 4.7kΩ với nguồn VCC để đảm bảo tín hiệu ổn định). Địa chỉ I2C của module (mặc định là 0x27) có thể được điều chỉnh thông qua các chân địa chỉ trên module I2C nếu cần.

Để giao tiếp với LCD 2004 (phiên bản I2C) trên ESP32, thư viện phổ biến nhất là **LiquidCrystal_I2C**. Thư viện này hỗ trợ điều khiển LCD qua giao thức I2C, giảm số lượng chân kết nối từ 12 (chế độ song song) xuống còn 4 (VCC, GND, SDA, SCL).

2.9.10. Màn hình OLED giao tiếp I2C

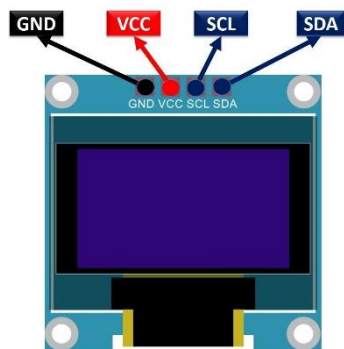
Màn hình OLED (Organic Light-Emitting Diode) là thiết bị hiển thị sử dụng công nghệ phát quang hữu cơ, cung cấp khả năng hiển thị văn bản và đồ họa với độ tương phản cao và hiệu suất năng lượng vượt trội. Công nghệ này dựa trên các diode hữu cơ phát sáng khi có dòng điện chạy qua, không yêu cầu đèn nền như màn hình LCD, do đó giảm tiêu thụ điện năng và tăng độ bền. Màn hình OLED giao tiếp I2C được sử dụng rộng rãi trong các ứng dụng yêu cầu hiển thị thông tin trực quan với kích thước nhỏ gọn [15].

Các thông số kỹ thuật chi tiết được trình bày trong Bảng 2.20 sau đây, cung cấp cái nhìn tổng quan về đặc điểm kỹ thuật của OLED:

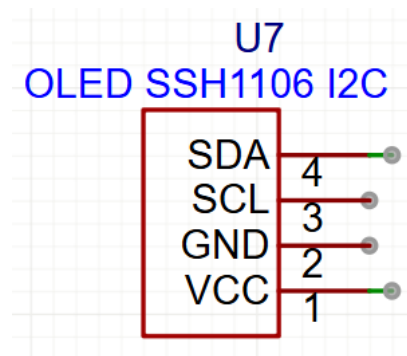
Bảng 2.20: Thông số kỹ thuật OLED

STT	Thông số	Chi tiết
1	Kích thước hiển thị	128x64 pixel
2	Nguồn cấp	3.3V – 5V
3	Độ tương phản	Cao và có thể điều chỉnh được
4	Giao tiếp	I2C, địa chỉ mặc định 0x3C

Màn hình OLED giao tiếp I2C được thiết kế với 4 chân trên module I2C, đảm nhiệm vai trò cấp nguồn và giao tiếp. Sơ đồ chân của module được minh họa như hình sau:



Hình 2.22: OLED và vị trí từng chân



Hình 2.23: Hình ảnh ký hiệu OLED

Hình 2.22, giúp người dùng dễ dàng nhận biết vị trí và chức năng của từng dây để kết nối với các vi điều khiển. Còn hình 2.23, minh họa về ký hiệu của OLED trong phần mềm mô phỏng. Chi tiết về tên và chức năng của các dây được trình bày trong bảng sau:

Bảng 2.21: Tên và chức năng từng chân OLED I2C

STT	Tên	Chức năng
1	VCC	Chân cấp nguồn 3.3V – 5V
2	GND	Chân nối mass
3	SDA	Chân dữ liệu I2C
4	SCL	Chân tín hiệu xung Clock

Dựa trên Bảng 2.21, có thể thấy mỗi chân của module OLED giao tiếp I2C đảm nhiệm một vai trò cụ thể trong việc vận hành và kết nối. Chân VCC cung cấp nguồn điện 3.3V – 5V để vận hành màn hình và module I2C. Chân GND đóng vai trò tham chiếu điện áp, được nối với điểm chung của hệ thống. Chân SDA và SCL là các chân giao tiếp I2C, lần lượt chịu trách nhiệm truyền dữ liệu và xung nhịp, cần kết nối với các chân I2C tương ứng trên vi điều khiển (thường yêu cầu điện trở kéo lên từ 2.2kΩ đến 4.7kΩ với nguồn VCC để đảm bảo tín hiệu ổn định). Địa chỉ I2C của module (mặc định là 0x3C) có thể được điều chỉnh thông qua các chân địa chỉ trên module I2C nếu cần.

Để giao tiếp với màn hình OLED 0.96 inch (SSD1306) trên ESP32, các thư viện phổ biến bao gồm:

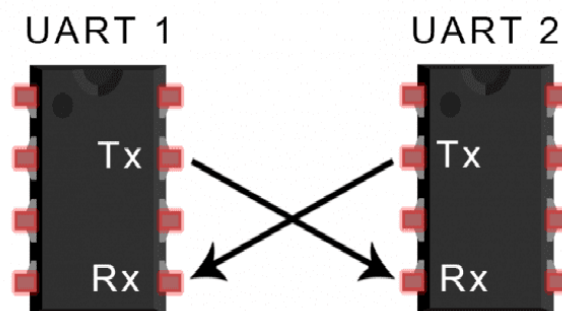
Adafruit_SSD1306 và **Adafruit_GFX**:

- **Adafruit_SSD1306**: Thư viện chuyên dụng để điều khiển màn hình OLED SSD1306 qua I2C hoặc SPI, hỗ trợ hiển thị văn bản, hình ảnh, và đồ họa cơ bản [1, 10].
- **Adafruit_GFX**: Thư viện phụ trợ cung cấp các hàm vẽ đồ họa như đường thẳng, hình tròn, và văn bản tùy chỉnh.

2.4. CÁC CHUẨN TRUYỀN THÔNG SỬ DỤNG

2.4.1. Giao tiếp UART

Giao tiếp UART (Universal Asynchronous Receiver-Transmitter) là một chuẩn truyền thông nối tiếp không đồng bộ, được sử dụng rộng rãi để trao đổi dữ liệu giữa vi điều khiển và các thiết bị ngoại vi [16]. Giao thức này không yêu cầu tín hiệu đồng bộ hóa như clock bên ngoài, thay vào đó sử dụng tốc độ baud rate được cấu hình trước để đảm bảo đồng bộ giữa thiết bị gửi và nhận. UART truyền dữ liệu theo từng byte, với mỗi byte bao gồm một bit bắt đầu (start bit), các bit dữ liệu (thường 8 bit) và một hoặc hai bit dừng (stop bit), đôi khi kèm theo bit kiểm tra chẵn/lẻ (parity bit) để phát hiện lỗi. Giao thức này nổi bật với tính đơn giản và hiệu quả trong các ứng dụng giao tiếp tầm ngắn, như kết nối với cảm biến, module GPS, hoặc thiết bị ngoại vi như máy tính qua cổng COM.

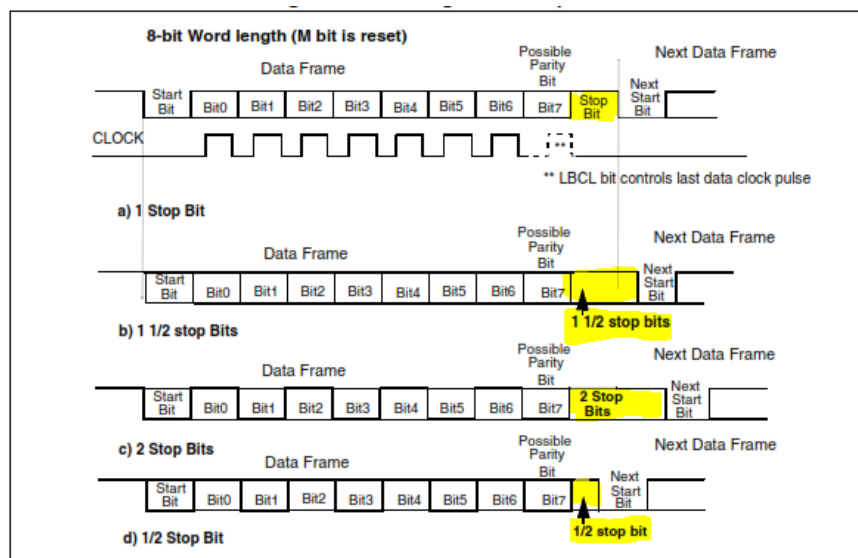


Hình 2.24: Giao tiếp UART giữa hai thiết bị

Tầng vật lý của giao tiếp UART xác định các đặc điểm phần cứng cần thiết để truyền dữ liệu. Giao thức này sử dụng hai dây chính: TX (transmit) để gửi dữ liệu và RX (receive) để nhận dữ liệu, cùng với các dây nguồn và đất để hoàn tất kết nối. Tín hiệu trên các dây được biểu diễn dưới dạng mức điện áp logic, thường là 0V cho mức thấp (logic 0) và 3.3V hoặc 5V cho mức cao (logic 1), tùy thuộc

vào vi điều khiển hoặc thiết bị sử dụng. UART không hỗ trợ truyền dữ liệu đa điểm (multi-drop), mà chỉ hoạt động theo mô hình điểm-điểm (point-to-point), nghĩa là mỗi kết nối UART chỉ liên kết hai thiết bị. Khoảng cách truyền dữ liệu thường bị giới hạn dưới 15m, tùy thuộc vào tốc độ baud rate và chất lượng dây dẫn, do không có cơ chế đồng bộ hóa phức tạp hoặc xử lý nhiễu.

UART hoạt động theo mô hình point-to-point, chỉ kết nối giữa hai thiết bị, không hỗ trợ truyền thông đa điểm (multi-drop). Dữ liệu được đóng gói theo từng khung truyền (data frame) bao gồm: 1 Start bit, 7-9 bit dữ liệu, 1 Parity bit (tùy chọn) và 1-2 Stop bit. Thiết bị phát truyền dữ liệu theo thứ tự từ bit thấp nhất (LSB) đến bit cao nhất (MSB) với tốc độ xác định bằng baud rate. Thiết bị nhận phát hiện Start bit để đồng bộ việc lấy mẫu dữ liệu, kiểm tra Parity bit (nếu có), và xác thực Stop bit để hoàn tất quá trình nhận và tái tạo dữ liệu.



Hình 2.25: Tín hiệu truyền của UART

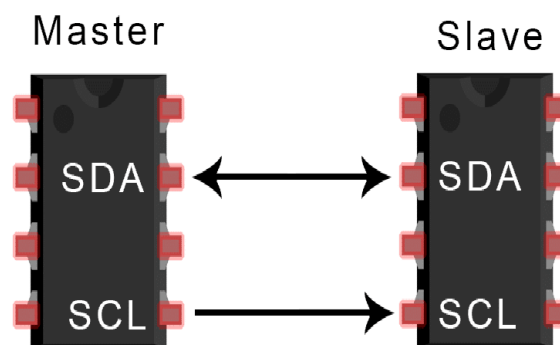
Giao tiếp UART có nhiều ưu điểm như cấu hình đơn giản, chi phí thấp, và hỗ trợ truyền dữ liệu full-duplex, làm cho giao thức này trở thành lựa chọn phổ biến trong các ứng dụng cơ bản. Tuy nhiên, UART cũng có nhược điểm như khoảng cách truyền bị giới hạn, không phù hợp cho truyền dữ liệu đa điểm, và dễ bị nhiễu ở tốc độ cao hoặc trên dây cáp dài nếu không có biện pháp bảo vệ tín hiệu.

UART được ứng dụng phổ biến trong các hệ thống vi điều khiển, dùng để giao tiếp với cảm biến UART, module GPS, module Bluetooth, RFID hoặc các

thiết bị truyền thông nối tiếp khác. Ngoài ra, UART còn được sử dụng trong việc nạp chương trình, debug hệ thống nhưng qua cổng COM hoặc USB to UART, đảm bảo giải pháp truyền thông cơ bản nhưng hiệu quả trong các thiết kế điện tử và IoT.

2.4.2. Giao tiếp I2C

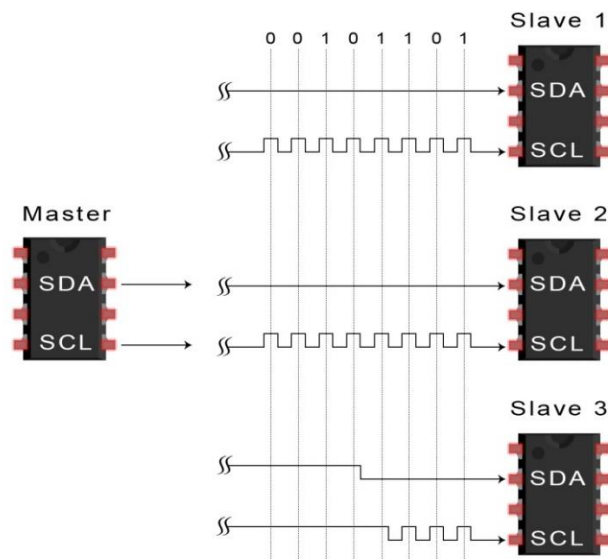
Giao tiếp I2C (Inter-Integrated Circuit) là một chuẩn truyền thông nối tiếp đồng bộ, để kết nối các thiết bị điện tử với nhau, thường trong khoảng cách ngắn [17]. Giao thức này được thiết kế để truyền dữ liệu giữa một thiết bị chủ (master) và nhiều thiết bị phụ (slave) thông qua hai dây tín hiệu chính, giúp giảm số lượng chân kết nối trên vi điều khiển. I2C sử dụng cơ chế địa chỉ hóa để xác định thiết bị cần giao tiếp, với mỗi thiết bị phụ có một địa chỉ duy nhất. Giao thức này hỗ trợ tốc độ truyền dữ liệu từ 100 kbps (chế độ chuẩn) đến 400 kbps (chế độ nhanh) và thậm chí cao hơn trong các phiên bản mới.



Hình 2.26: Giao tiếp I2C giữa hai thiết bị

Tầng vật lý của giao tiếp I2C bao gồm hai dây chính: SDA (Serial Data) để truyền dữ liệu và SCL (Serial Clock) để cung cấp tín hiệu đồng bộ. Cả hai dây đều được kết nối với nguồn điện áp 3.3V hoặc 5V thông qua các điện trở kéo lên có giá trị từ 2.2k Ω đến 4.7k Ω , đảm bảo tín hiệu ở mức cao khi không có thiết bị nào kéo xuống mức thấp. I2C hoạt động theo mô hình đa điểm (multi-drop), cho phép một thiết bị chủ kết nối với nhiều thiết bị phụ trên cùng một bus, miễn là mỗi thiết bị có địa chỉ duy nhất. Tín hiệu trên bus được biểu diễn dưới dạng mức logic.

Khoảng cách truyền dữ liệu thường bị giới hạn dưới 1m do điện dung của dây dẫn và nhiễu tín hiệu, đặc biệt ở tốc độ cao.



Hình 2.27: Tín hiệu truyền I2C

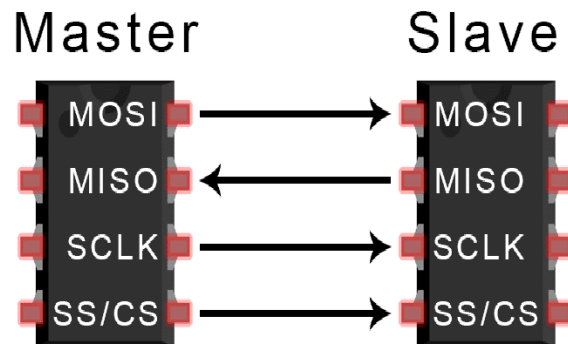
Giao tiếp I2C có ưu điểm nổi bật như chỉ sử dụng hai dây tín hiệu, hỗ trợ kết nối đa thiết bị (multi-slave) trên cùng một bus, và dễ dàng mở rộng hệ thống nhờ cơ chế địa chỉ hóa. Tuy nhiên, nhược điểm của giao thức này bao gồm tốc độ truyền dữ liệu thấp hơn so với SPI, dễ bị ảnh hưởng bởi nhiễu ở khoảng cách xa, và cần quản lý địa chỉ cẩn thận để tránh xung đột giữa các thiết bị phụ.

I2C được sử dụng rộng rãi trong các ứng dụng lý thuyết như kết nối vi điều khiển với cảm biến môi trường như cảm biến nhiệt độ, độ ẩm, màn hình LCD có giao tiếp I2C, hoặc bộ nhớ EEPROM để lưu trữ dữ liệu. Giao thức này cũng phổ biến trong các hệ thống nhúng cần giao tiếp với nhiều thiết bị ngoại vi mà không yêu cầu tốc độ truyền dữ liệu cao, chẳng hạn như trong các thiết bị đo lường hoặc điều khiển cơ bản.

2.4.3. Giao tiếp SPI

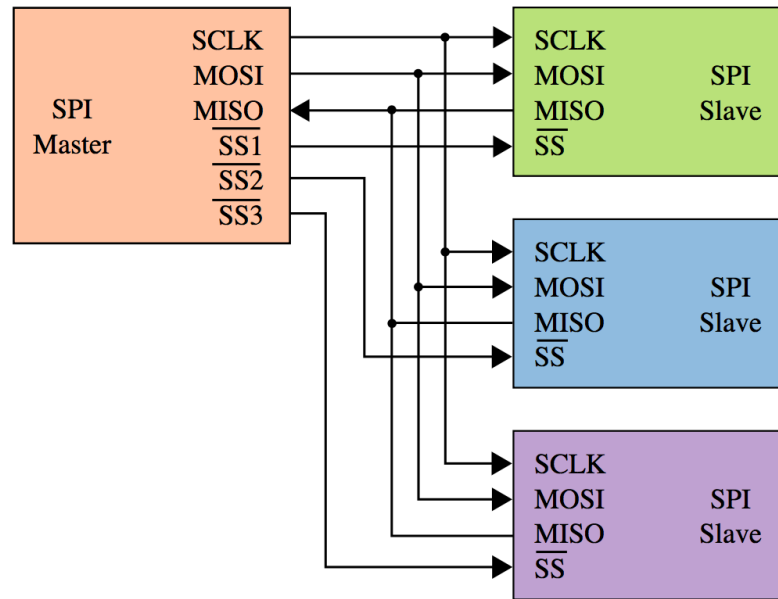
Giao tiếp SPI (Serial Peripheral Interface) là một chuẩn truyền thông nối tiếp đồng bộ, truyền dữ liệu nhanh giữa vi điều khiển và các thiết bị ngoại vi [18]. Giao thức này sử dụng một thiết bị chủ (master) để điều khiển một hoặc nhiều thiết bị phụ (slave) thông qua tín hiệu clock đồng bộ, cho phép truyền dữ liệu với tốc

độ cao hơn so với nhiều giao thức khác. SPI hoạt động dựa trên bốn dây chính, bao gồm tín hiệu dữ liệu, xung nhịp, và chọn thiết bị, giúp tối ưu hóa hiệu suất trong các ứng dụng yêu cầu băng thông lớn. Giao thức này nổi bật với khả năng truyền dữ liệu toàn song công (full-duplex), cho phép gửi và nhận dữ liệu đồng thời.



Hình 2.28: Giao tiếp SPI giữa hai thiết bị

Tầng vật lý của giao tiếp SPI bao gồm bốn dây chính: SCLK (Serial Clock) để cung cấp tín hiệu đồng bộ, MOSI (Master Out Slave In) để gửi dữ liệu từ thiết bị chủ đến thiết bị phụ, MISO (Master In Slave Out) để nhận dữ liệu từ thiết bị phụ về thiết bị chủ, và SS/CS (Slave Select/Chip Select) để chọn thiết bị phụ cụ thể. Tín hiệu trên các dây được biểu diễn dưới dạng mức logic 0V cho mức thấp và 3.3V hoặc 5V cho mức cao, tùy thuộc vào điện áp hoạt động của hệ thống. SPI hỗ trợ kết nối đa điểm thông qua nhiều chân SS/CS, nhưng mỗi thiết bị phụ cần một chân SS riêng biệt trên thiết bị chủ. Khoảng cách truyền dữ liệu thường giới hạn dưới 1m do nhiễu và điện dung dây dẫn, và hiệu suất có thể giảm nếu không sử dụng cáp chất lượng cao.



Hình 2.29: Tín hiệu truyền SPI

Giao tiếp SPI có ưu điểm như tốc độ truyền dữ liệu cao, hỗ trợ full-duplex, và dễ dàng tích hợp với các thiết bị ngoại vi hiệu suất cao. Tuy nhiên, nhược điểm của giao thức này là yêu cầu nhiều dây kết nối tối thiểu 4 dây, không hỗ trợ địa chỉ hóa như I2C và cần một chân SS riêng cho mỗi thiết bị phụ, làm tăng độ phức tạp khi kết nối nhiều thiết bị.

SPI được sử dụng rộng rãi trong các ứng dụng lý thuyết như kết nối vi điều khiển với bộ nhớ flash, màn hình OLED hoặc TFT, cảm biến áp suất, hoặc các module giao tiếp RF như RFM95. Giao thức này cũng phổ biến trong các hệ thống nhúng yêu cầu truyền dữ liệu nhanh và đáng tin cậy, chẳng hạn như trong các thiết bị đo lường chính xác hoặc giao tiếp với các thiết bị ngoại vi tốc độ cao.

2.4.4. Giao tiếp WiFi

WiFi (viết tắt của từ tiếng anh "Wireless Fidelity") là một công nghệ mạng không dây dựa trên các tiêu chuẩn IEEE 802.11 (Institute of Electrical and Electronic Engineers), thường được sử dụng để kết nối không dây mạng nội bộ và internet qua sóng Radio (sóng vô tuyến). WiFi cho phép các thiết bị như máy tính, thiết bị di động và các thiết bị khác có thể giao tiếp với Internet [19].

Mạng WiFi là 1 mạng nội bộ gồm các thiết bị kết nối với bộ phát sóng WiFi, và có thể trao đổi thông tin với nhau hoặc đi ra ngoài internet.

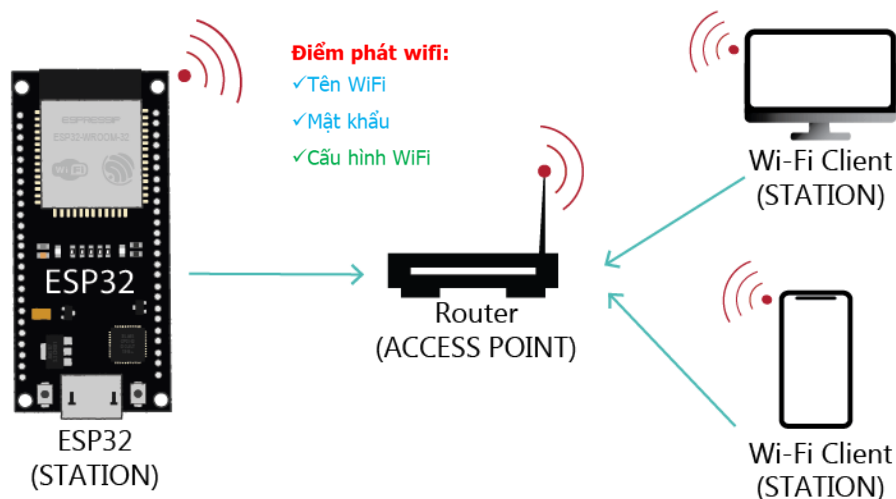
WiFi

802.11AC

Hình 2.30: Giao thức WiFi

Chuẩn WiFi đầu tiên được ra đời năm 1997 với tốc độ chỉ 2 Mbps và đã không ngừng phát triển qua các thế hệ như 802.11b, a, g, n, ac, ad và mới nhất là 802.11ax (còn gọi là WiFi 6), mang lại tốc độ cao hơn, khả năng kết nối tốt hơn và hiệu suất vượt trội. Mỗi chuẩn WiFi qua từng thời kì đều cải thiện về băng tần, tốc độ truyền, khả năng xuyên vật cản và số lượng thiết bị hỗ trợ, giúp WiFi trở thành công nghệ kết nối không dây phổ biến toàn cầu. Ngoài ra, tính năng WiFi hotspot trên thiết bị di động còn cho phép chia sẻ mạng như một bộ phát WiFi di động tiện lợi.

Wi-Fi mang lại tính linh động cao, dễ triển khai và hỗ trợ nhiều thiết bị kết nối cùng lúc mà không cần hạ tầng phức tạp. Ngoài ra, WiFi còn có khả năng mở rộng linh hoạt, với chi phí bảo trì thấp, tốc độ ngày càng được nâng cao và bảo mật tốt nhờ các tiêu chuẩn mã hóa tiên tiến như WPA3.



Hình 2.31: ESP giao tiếp WiFi

WiFi là công nghệ truyền thông không dây dựa trên tiêu chuẩn IEEE 802.11, cho phép các thiết bị kết nối mạng thông qua sóng vô tuyến ở các dải tần 2.4GHz và 5GHz. Cách thức hoạt động của WiFi bắt đầu từ việc thiết bị (station) gửi tín hiệu yêu cầu kết nối đến Access Point (AP), là thiết bị phát sóng WiFi đóng vai trò cầu nối giữa các thiết bị và mạng Internet. AP sẽ xác thực thiết bị dựa trên các phương thức bảo mật như WPA2, WPA3, sau đó cấp phát địa chỉ IP thông qua giao thức DHCP để thiết lập đường truyền dữ liệu. Khi kết nối đã thiết lập, dữ liệu từ thiết bị được đóng gói thành các gói tin (packets) và truyền qua sóng vô tuyến bằng kỹ thuật OFDM (Orthogonal Frequency Division Multiplexing) hoặc DSSS (Direct Sequence Spread Spectrum), tùy vào chuẩn WiFi. Tín hiệu truyền đi được mã hóa để bảo mật và đảm bảo toàn vẹn dữ liệu. WiFi hỗ trợ tốc độ truyền cao, lên đến hàng trăm Mbps hoặc thậm chí hàng Gbps với các chuẩn mới như WiFi 6 (802.11ax), cho phép truyền đồng thời nhiều luồng dữ liệu nhờ công nghệ MIMO (Multiple Input Multiple Output). Nhờ cơ chế này, WiFi trở thành giải pháp kết nối phổ biến cho các thiết bị di động, hệ thống IoT, nhà thông minh và các ứng dụng yêu cầu tính linh hoạt và băng thông lớn.

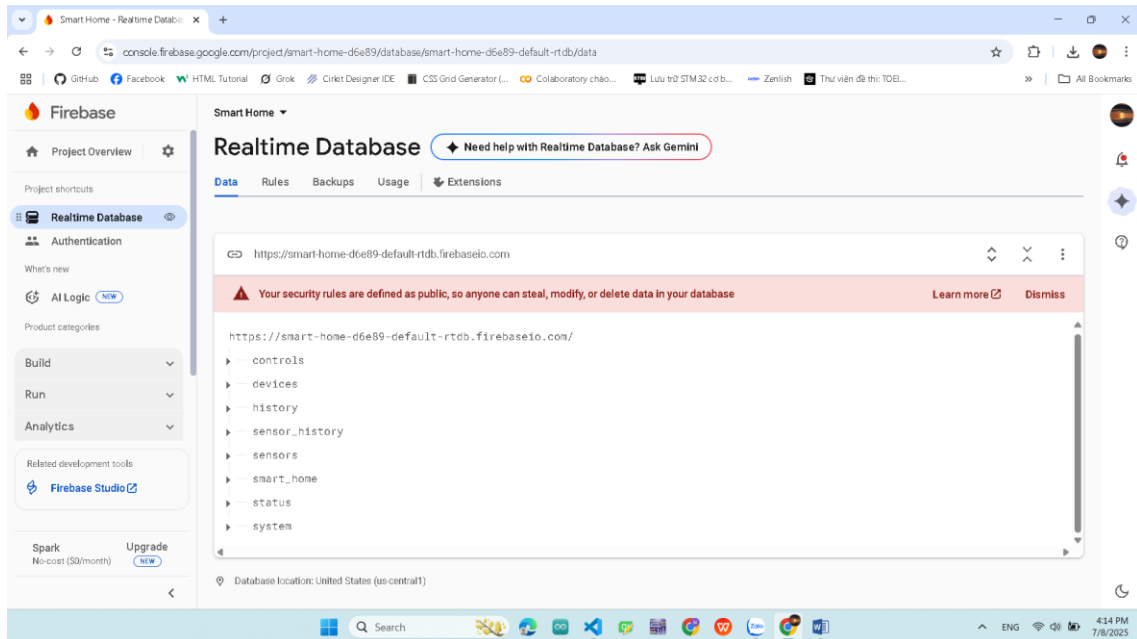
2.5. LƯU TRỮ DỮ LIỆU VÀ GIAO DIỆN NGƯỜI DÙNG

2.5.1. Giới thiệu về Firebase Realtime Database

Firebase là một nền tảng phát triển ứng dụng do Google phát triển, cung cấp nhiều dịch vụ trên đám mây để hỗ trợ việc xây dựng và quản lý các ứng dụng hiện đại. Nền tảng này bao gồm các công cụ như xác thực người dùng, lưu trữ tệp và đặc biệt là các quản lý dữ liệu, trong đó nổi bật là cơ sở dữ liệu thời gian thực [20].

Firebase Realtime Database, một thành phần cốt lõi của nền tảng này, đây là một cơ sở dữ liệu NoSQL được thiết kế để lưu trữ dữ liệu dưới dạng cấu trúc JSON linh hoạt. Cơ sở dữ liệu này cho phép đồng bộ hóa tức thời giữa các thiết bị kết nối, giúp dữ liệu như giá trị cảm biến, trạng thái thiết bị hoặc thông số hệ thống được cập nhật ngay lập tức trên toàn hệ thống. Với mô hình sự kiện, Firebase Realtime Database hỗ trợ theo dõi liên tục các thay đổi dữ liệu, chẳng hạn như khi một cảm biến gửi giá trị mới hoặc trạng thái thiết bị thay đổi, làm cho nó trở thành

công cụ quan trọng trong các hệ thống nhúng cần quản lý dữ liệu động và phản hồi nhanh chóng.



Hình 2.32: Hình ảnh minh họa giao diện Realtime Database

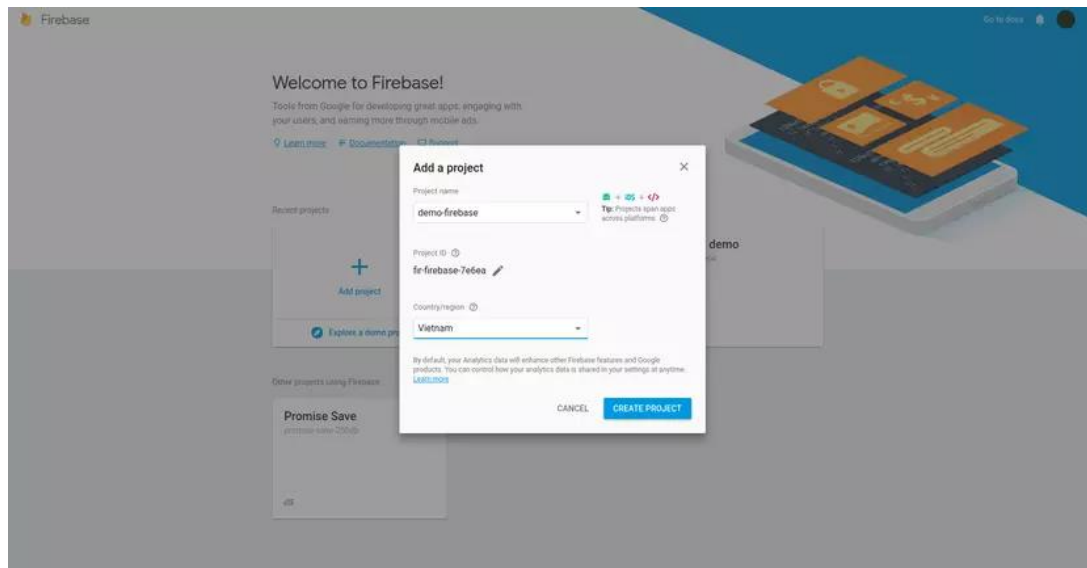
2.5.2. Các bước thiết lập Firebase

Bước 1. Đăng nhập vào trang chủ của Firebase Console

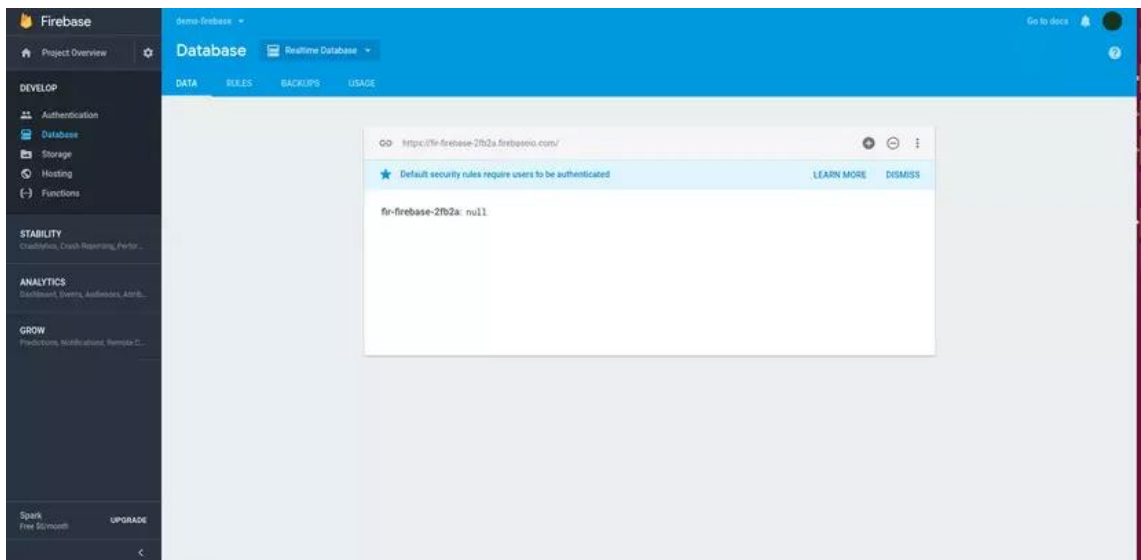
<https://firebase.google.com>

A screenshot of the "Create a project" dialog box in the Firebase console. The dialog has a title bar "Create a project" with a close button (X). It contains two input fields: "Project name" with the text "FirebaseRealtimeDatabase" and "Country/region" with a dropdown menu showing "Vietnam". Below the input fields, there is a paragraph of text: "By default, your Firebase Analytics data will enhance other Firebase features and Google products. You can control how your Firebase Analytics data is shared in your settings at anytime. [Learn more](#)". At the bottom right, there are two buttons: "CANCEL" and "CREATE PROJECT".

Bước 2. Sau khi đã đăng nhập tài khoản Firebase. Chúng ta tạo một project mới trên Firebase. Ở đây mình tạo một project có tên demo-firebase. Chúng ta viết vào dòng Project name và chọn CREAT PROJECT.



Bước 3. Sau khi đã tạo project mới chúng ta sẽ được chuyển vào trang dashboard của project vừa mới tạo. Phía menu bên trái, chúng ta chọn Develop → Database



Bước 4. Tạo SDK cho sever

```

<script src="https://www.gstatic.com/firebasejs/10.0.0/firebase-app.js"></script>
<script src="https://www.gstatic.com/firebasejs/10.0.0/firebase-database.js"></sc
<script>
  const firebaseConfig = {
    apiKey: "...",
    authDomain: "...",
    databaseURL: "https://your-project.firebaseio.com",
    ...
  };
  firebase.initializeApp(firebaseConfig);
  const db = firebase.database();
</script>

```

Bước 5. Copy SDK và dán vào code và thực hiện chạy chương trình

2.5.3. Giới thiệu Web Dashboard giám sát và điều khiển

Web Dashboard là một giao diện người dùng dựa trên trình duyệt web, được tạo ra để hỗ trợ giám sát dữ liệu thời gian thực và điều khiển thiết bị từ xa một cách trực quan và hiệu quả. Giao diện này thường được xây dựng dựa trên ba công nghệ cốt lõi của phát triển web là HTML, CSS, và JavaScript.

- HTML (HyperText Markup Language) là ngôn ngữ đánh dấu dùng để xây dựng cấu trúc cơ bản của trang web, định nghĩa các thành phần như tiêu đề, bảng hoặc nút bấm.
- CSS (Cascading Style Sheets) là công nghệ định dạng, giúp tạo phong cách cho giao diện như điều chỉnh màu sắc, kích thước chữ hoặc bố cục để trang web trở nên trực quan và thân thiện hơn.
- JavaScript là ngôn ngữ lập trình phía trình duyệt, cho phép xử lý các tương tác động như là cập nhật dữ liệu mà không cần tải lại trang hoặc phản hồi khi người dùng nhấn nút.

Nhờ sự kết hợp của HTML, CSS, và JavaScript mà Web Dashboard có thể hiển thị dữ liệu thời gian thực, chẳng hạn như giá trị cảm biến hoặc trạng thái thiết bị, đồng thời cho phép người dùng gửi các lệnh điều khiển, như bật/tắt thiết bị hoặc thay đổi cài đặt, thông qua các thao tác đơn giản trên trình duyệt.

Để kết nối với Firebase, Web Dashboard thường sử dụng API hoặc SDK của Firebase, cho phép giao diện truy cập dữ liệu từ Realtime Database và hiển thị chúng dưới dạng văn bản, biểu đồ hoặc trạng thái thiết bị. Đồng thời, người dùng có thể gửi lệnh điều khiển thông qua giao diện, với các lệnh này được truyền ngược lại qua Firebase đến thiết bị. Với khả năng truy cập từ bất kỳ đâu có kết nối mạng, Web Dashboard trở thành một công cụ quan trọng trong việc quản lý và vận hành các hệ thống nhúng từ xa.

2.6. TỔNG QUAN VỀ CÁC CÔNG CỤ HỖ TRỢ XÂY DỰNG VÀ LẬP TRÌNH

2.6.1. Phần mềm Arduino IDE

Arduino IDE (viết tắt của Arduino Integrated Development Environment) là một môi trường lập trình tích hợp, cho phép người dùng viết mã và nạp chương trình vào các bo mạch Arduino. Đây là một phần mềm mã nguồn mở chính thức của Arduino, được thiết kế thân thiện với người dùng, kể cả những người không có nền tảng kỹ thuật cũng có thể dễ dàng sử dụng [21].

Phần mềm Arduino IDE hỗ trợ đa nền tảng, có thể cài đặt và chạy trên các hệ điều hành phổ biến như Windows, macOS và Linux. Được xây dựng dựa trên nền tảng Java, Arduino IDE cung cấp sẵn nhiều chức năng và lệnh giúp cho việc soạn thảo, kiểm tra lỗi và biên dịch mã trở nên hiệu quả hơn.



Hình 2.33: Phần mềm Arduino IDE

Ứng dụng này cho phép biên dịch mã và lập trình cho nhiều loại bo mạch như Arduino Uno, Arduino Mega, Arduino Micro, ESP32, ESP8266, và nhiều loại khác. Mỗi bo mạch đều có vi điều khiển riêng, chịu trách nhiệm xử lý mã lệnh được nạp từ máy tính. Mã nguồn (hay còn gọi là sketch) được viết trong Arduino

IDE sẽ được biên dịch thành file .hex - định dạng mà vi điều khiển có thể hiểu và thực thi.

Môi trường lập trình Arduino IDE bao gồm hai phần chính: trình soạn thảo mã nguồn và trình biên dịch. Phần soạn thảo cho phép người dùng viết mã theo ngôn ngữ lập trình C hoặc C++, trong khi phần biên dịch chịu trách nhiệm chuyển mã thành file .hex và tải lên bo mạch thông qua cáp USB. Khi quá trình biên dịch hoàn tất, IDE sẽ tạo ra file hex, và file này được truyền đến vi điều khiển tích hợp trên bo mạch, nơi nó được thực thi theo đúng nội dung chương trình đã viết.

Arduino IDE có giao diện đơn giản, dễ sử dụng, phù hợp cho cả người mới và không chuyên lập trình, đồng thời hỗ trợ đa dạng dòng vi điều khiển như Arduino, ESP32, ESP8266, STM32... Với kho thư viện miễn phí, mã nguồn mở, hỗ trợ đa nền tảng và có cộng đồng người dùng đông đảo, Arduino IDE là một công cụ lý tưởng để phát triển và học lập trình nhúng một cách hiệu quả.

2.6.2. Phần mềm Easy EDA

EasyEDA là một công cụ thiết kế mạch điện tử (EDA) hoàn toàn miễn phí, hoạt động trực tiếp trên nền tảng đám mây mà không yêu cầu người dùng phải cài đặt phần mềm trên máy tính cá nhân. Ứng dụng này được phát triển nhằm hỗ trợ kỹ sư điện, giảng viên, sinh viên ngành kỹ thuật, cũng như những người có niềm đam mê với điện tử, giúp họ thực hiện quá trình thiết kế mạch một cách nhanh chóng và tiện lợi hơn [22].



Hình 2.34: Phần mềm Easy EDA

Một số tính năng nổi bật của EasyEDA có thể kể tới như, EasyEDA giúp người dùng vẽ mạch nguyên lý nhanh hơn vì phần mềm này sử dụng các thư viện có sẵn và luôn được cập nhật trên trình duyệt. Phần mềm giúp kiểm tra mạch tín hiệu tương tự, mạch tín hiệu số và hỗn hợp với các mô hình và mạch phụ tích hợp.

Với mạch cần thiết kế nhiều lớp, người dùng vẫn có thể thiết kế nhanh chóng và bố trí linh kiện một cách hợp lý. Giao diện người dùng dễ truy cập và sử dụng, rất tiện lợi khi sử dụng phần mềm online mà không cần bắt buộc phải tải xuống máy tính.

2.6.3. Phần mềm Visual Studio Code

Visual Studio Code (VS Code) là một trình chỉnh sửa mã nguồn miễn phí, hỗ trợ đa hệ điều hành, do Microsoft phát triển. Nhờ vào giao diện trực quan, nhiều tính năng lập trình mạnh mẽ, cùng với khả năng mở rộng linh hoạt thông qua các tiện ích, VS Code đã nhanh chóng trở thành một trong những công cụ được lập trình viên ưa chuộng và sử dụng rộng rãi trên toàn thế giới.

Visual Studio Code (VS Code) cung cấp rất nhiều công cụ và chức năng mạnh mẽ để hỗ trợ lập trình viên trong quá trình phát triển phần mềm. Một trong những chức năng nổi bật nhất là IntelliSense, giúp tự động gợi ý mã lệnh, tên biến, hàm, và thậm chí là tài liệu API, từ đó rút ngắn thời gian viết mã và giảm thiểu lỗi. Ngoài ra, VS Code còn tích hợp sẵn trình quản lý Git, cho phép người dùng thực hiện các thao tác như commit, push, pull, merge ngay trong giao diện mà không cần chuyển sang công cụ khác. Công cụ Debugging của VS Code cũng rất mạnh, hỗ trợ thiết lập breakpoint, theo dõi biến, kiểm soát luồng thực thi cho nhiều ngôn ngữ lập trình như C++, Python, JavaScript một cách trực quan [23].

VS Code còn có Terminal tích hợp, giúp người dùng có thể chạy lệnh trực tiếp trong cùng một cửa sổ mà không cần mở thêm phần mềm ngoài. Ngoài ra, kho Extension Marketplace đồ sộ chính là một trong những điểm mạnh lớn, nơi người dùng có thể tải về các tiện ích mở rộng như formatter code tự động, linting kiểm tra lỗi, hỗ trợ container hóa với Docker, làm việc với machine learning hoặc các framework Web hiện đại như React, Angular, Vue.



Hình 2.35: Phần mềm Visual Studio Code

VS Code là lựa chọn hàng đầu cho lập trình viên bởi vì những lý do như nó miễn phí, mã nguồn mở và có thể hoạt động trên mọi nền tảng (Windows, macOS, Linux).

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

3.1. YÊU CẦU THIẾT KẾ HỆ THỐNG

Hệ thống giám sát nhà thông minh được thiết kế nhằm cung cấp giải pháp tích hợp để quản lý không gian sống, tăng cường an ninh và nâng cao tiện nghi cho các hộ gia đình hiện đại. Yêu cầu thiết kế tập trung vào việc xây dựng một hệ thống dựa trên công nghệ IoT, sử dụng kiến trúc Master-Slave với các node hoạt động độc lập, đồng bộ dữ liệu qua Firebase Realtime Database thông qua kết nối WiFi. Mục tiêu là đảm bảo hiệu suất cao, chi phí triển khai thấp, độ tin cậy ổn định, và khả năng mở rộng linh hoạt.

Yêu cầu thiết kế bao gồm các khía cạnh về hiệu suất và tính năng tổng quát. Hệ thống phải đạt độ trễ phản hồi không vượt quá 2,5 giây cho các tác vụ điều khiển từ xa và đồng bộ dữ liệu, đồng thời duy trì hoạt động ổn định trong điều kiện mạng WiFi thông thường. Thiết kế cần hỗ trợ khả năng xử lý lỗi, đặc biệt khi kết nối mạng bị gián đoạn, thông qua cơ chế lưu trữ tạm thời hoặc đồng bộ lại dữ liệu. Hệ thống phải sử dụng các linh kiện phổ thông, chi phí thấp nhưng đảm bảo chất lượng, nhằm giảm giá thành triển khai. Ngoài ra, thiết kế cần đảm bảo khả năng tích hợp thêm thiết bị hoặc node mới mà không làm giảm hiệu suất tổng thể, đáp ứng nhu cầu mở rộng trong tương lai.

Yêu cầu về giao diện người dùng và bảo mật cũng được đặt ra. Hệ thống cần cung cấp giao diện trực quan, thân thiện, tương thích với nhiều nền tảng (máy tính và thiết bị di động), cho phép người dùng dễ dàng giám sát và điều khiển. Về bảo mật, hệ thống phải đảm bảo dữ liệu truyền qua WiFi và lưu trữ trên nền tảng đám mây được bảo vệ an toàn, tránh truy cập trái phép. Các yêu cầu này tạo nền tảng cho việc thiết kế một hệ thống IoT hiệu quả, đáp ứng xu hướng công nghệ hiện đại và phù hợp với các ứng dụng nhà thông minh.

3.2. ĐẶC TẢ HỆ THỐNG

3.2.1. Chức năng hệ thống tổng thể

Hệ thống giám sát nhà thông minh được thiết kế để thực hiện các chức năng cốt lõi nhằm đáp ứng nhu cầu giám sát, an ninh và điều khiển tự động. Chức năng tổng thể bao gồm thu thập dữ liệu môi trường, xác thực an ninh, điều khiển thiết bị và hiển thị thông tin, tất cả được tích hợp trong một hệ sinh thái IoT hiệu quả. Hệ thống cung cấp khả năng giám sát và điều khiển cả tại chỗ lẫn từ xa thông qua giao diện người dùng trực quan. Các chức năng chính của hệ thống bao gồm:

- Thu thập dữ liệu môi trường, bao gồm các thông số như nhiệt độ, độ ẩm, cường độ ánh sáng và nồng độ khí gas, với tần suất cập nhật tối thiểu 5 giây/lần và độ chính xác cao để hỗ trợ giám sát liên tục.
- Xác thực an ninh đa phương thức, cho phép người dùng truy cập hệ thống thông qua nhập mật khẩu, nhận diện sinh trắc học, hoặc quét thẻ từ, với thời gian xử lý mỗi sự kiện xác thực dưới 2 giây.
- Điều khiển các thiết bị điện như đèn, quạt và cơ cấu đóng/mở cửa, hỗ trợ cả chế độ tự động dựa trên dữ liệu môi trường và điều khiển từ xa qua Web Dashboard, với độ trễ phản hồi không vượt quá 2,5 giây.
- Hiển thị thông tin trạng thái hệ thống, dữ liệu môi trường và kết quả xác thực trên các thiết bị hiển thị tại chỗ, đồng thời cung cấp giao diện Web Dashboard để giám sát và điều khiển từ xa. Dữ liệu từ các chức năng này được đồng bộ qua Firebase, đảm bảo tính nhất quán và thời gian thực.

Hệ thống đảm bảo tích hợp các chức năng này trong một kiến trúc Master-Slave, với các node hoạt động độc lập và đồng bộ qua nền tảng đám mây. Điều này cho phép quản lý hiệu quả các tác vụ phức tạp, từ giám sát môi trường đến điều khiển thiết bị, đồng thời hỗ trợ khả năng mở rộng để tích hợp thêm chức năng hoặc thiết bị trong tương lai.

3.2.2. Chức năng ESP32 – Master

ESP32 – Master đóng vai trò điều phối trung tâm trong hệ thống, chịu trách nhiệm xử lý các tác vụ an ninh và giao tiếp người dùng. Node này được thiết kế để

thực hiện các chức năng yêu cầu độ ưu tiên cao, đảm bảo an toàn và hiệu quả trong quản lý hệ thống. ESP32 – Master hoạt động độc lập, gửi và nhận dữ liệu qua Firebase Realtime Database bằng kết nối WiFi. Các chức năng chính của ESP32 – Master bao gồm:

- Xử lý xác thực an ninh đa phương thức, bao gồm nhập mật khẩu qua bàn phím, nhận diện sinh trắc học, và quét thẻ từ, với thời gian phản hồi mỗi sự kiện dưới 2 giây.
- Điều khiển cơ cấu đóng/mở cửa dựa trên kết quả xác thực hoặc lệnh từ Web Dashboard, đảm bảo vận hành chính xác và an toàn.
- Hiển thị thông tin trạng thái hệ thống, kết quả xác thực và menu cài đặt trên thiết bị hiển thị tại chỗ, cung cấp giao diện tương tác trực quan cho người dùng.
- Đồng bộ dữ liệu xác thực và trạng thái hệ thống lên Firebase, cho phép truy xuất từ xa và tích hợp với các node khác. ESP32 – Master cũng hỗ trợ cài đặt hệ thống, như thêm/xóa thông tin xác thực hoặc thay đổi mật khẩu, thông qua giao diện tại chỗ.

ESP32-Master đảm bảo an ninh và tương tác người dùng hiệu quả, đóng vai trò cốt lõi trong việc duy trì tính toàn vẹn và bảo mật của hệ thống. Với khả năng xử lý nhanh và đồng bộ dữ liệu qua đám mây, node này hỗ trợ vận hành ổn định và linh hoạt trong các kịch bản nhà thông minh.

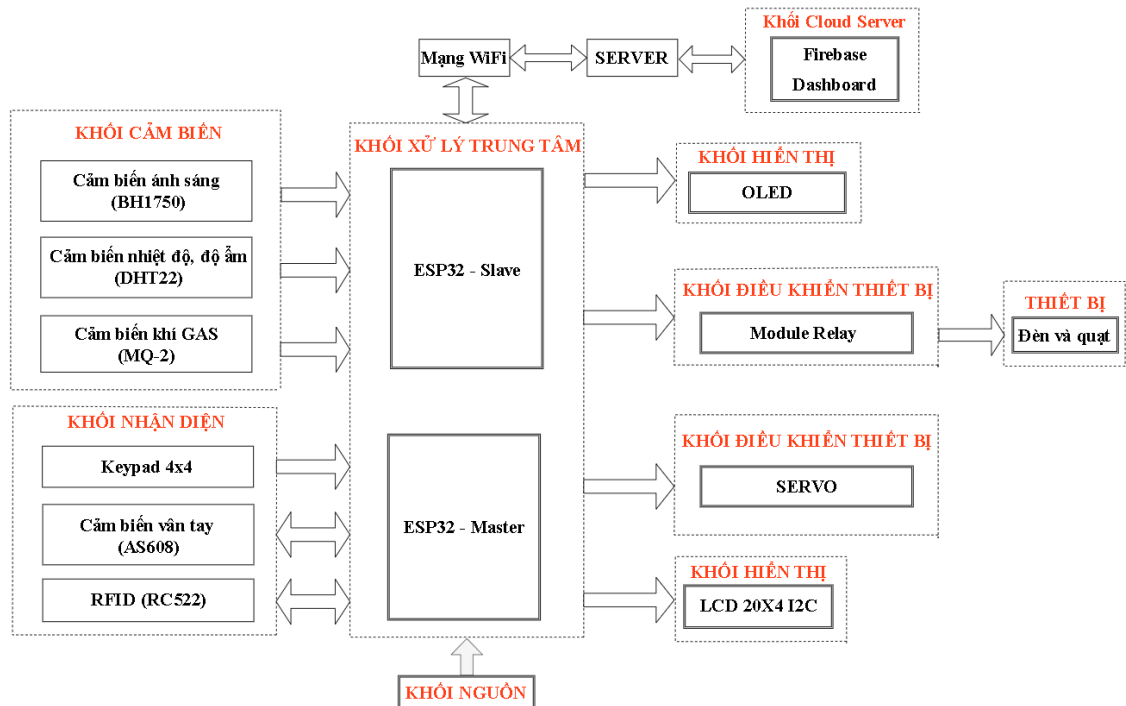
3.2.3. Chức năng ESP32 – Slave

ESP32 – Slave chịu trách nhiệm giám sát môi trường và điều khiển các thiết bị điện, thực hiện các tác vụ hỗ trợ trong hệ thống. Node này hoạt động độc lập, gửi dữ liệu môi trường và trạng thái thiết bị lên Firebase Realtime Database qua WiFi, đồng bộ với ESP32 – Master và Web Dashboard. ESP32 – Slave được thiết kế để giảm tải cho node Master, tối ưu hóa hiệu suất tổng thể. Các chức năng chính của ESP32 – Slave bao gồm:

- Thu thập dữ liệu từ các cảm biến môi trường, bao gồm nhiệt độ, độ ẩm, cường độ ánh sáng và nồng độ khí gas, với tần suất cập nhật tối thiểu 5 giây/lần và độ chính xác cao.
- Điều khiển các thiết bị điện như đèn, quạt dựa trên lệnh từ Web Dashboard hoặc chế độ tự động theo dữ liệu cảm biến, với độ trễ phản hồi dưới 2,5 giây.
- Hiển thị dữ liệu môi trường và trạng thái thiết bị trên thiết bị hiển thị tại chỗ, cung cấp thông tin trực quan cho người dùng.
- Đồng bộ dữ liệu cảm biến và trạng thái thiết bị lên Firebase, cho phép giám sát từ xa và tích hợp với các node khác. ESP32 – Slave cũng hỗ trợ kích hoạt cảnh báo khi phát hiện các thông số môi trường bất thường, như nồng độ khí gas vượt ngưỡng.

ESP32 – Slave đóng vai trò quan trọng trong việc đảm bảo giám sát môi trường liên tục và điều khiển thiết bị hiệu quả. Với thiết kế độc lập và khả năng đồng bộ qua đám mây, node này hỗ trợ vận hành hệ thống ổn định và đáp ứng các yêu cầu tự động hóa trong nhà thông minh.

3.2.4. Sơ đồ khối hệ thống



Hình 3.1: Sơ đồ khối hệ thống

Hình 3.1 là sơ đồ khối hệ thống nhà thông minh được thiết kế để thể hiện mối quan hệ và kết nối giữa các khối chức năng, đảm bảo hoạt động đồng bộ và hiệu quả. Hệ thống bao gồm các khối chính sau:

- **Khối xử lý trung tâm:** Gồm ESP32 – Master và ESP32 – Slave, đóng vai trò trung tâm xử lý của hệ thống. Slave sẽ kết nối với các cảm biến, khối hiển thị và khối điều khiển thiết bị. Trong khi đó, Master sẽ xử lý khối nhận diện, điều khiển thiết bị và cả hai đều giao tiếp với cloud server.
- **Khối cảm biến:** Bao gồm cảm biến ánh sáng BH1750, cảm biến nhiệt độ và độ ẩm DHT22 và cảm biến khí gas MQ2. Các cảm biến này kết nối với ESP32 – Slave qua giao tiếp I2C (BH1750), GPIO (DHT22) và ADC (MQ2), thu thập dữ liệu môi trường, hiển thị lên OLED, bật/tắt thiết bị dựa theo dữ liệu cảm biến và gửi dữ liệu Firebase.
- **Khối nhận diện:** Gồm Keypad4x4 để nhập mật khẩu, cảm biến vân tay AS608 để quét vân tay và module RFID để đọc thẻ UID. Các thành phần này kết nối với ESP32 – Master qua giao tiếp UART (AS608), SPI (RFID)

và GPIO (Keypad), xử lý dữ liệu xác thực và so sánh với thông tin từ Firebase.

- Khối hiển thị: Sử dụng màn hình LCD20x4 I2C, kết nối với ESP32 – Master và màn hình OLED kết nối với ESP32 – Slave qua giao tiếp I2C, hiển thị dữ liệu cảm biến, trạng thái xác thực, thông báo hệ thống và menu cài đặt khi người dùng tương tác qua Keypad.
- Khối điều khiển thiết bị: Bao gồm servo để mở cửa và module relay 4 kênh để bật/tắt thiết bị như đèn. Các thành phần này kết nối với Master và Slave qua GPIO và PWM (servo), nhận lệnh để thực thi các tác vụ điều khiển dựa trên dữ liệu từ ESP32 hoặc Web Dashboard.
- Khối cloud server: Gồm Firebase Realtime Database và Web Dashboard. ESP32 kết nối với Firebase qua Wi-Fi, gửi dữ liệu cảm biến, thông tin xác thực và lịch sử điều khiển lên đám mây, đồng thời nhận lệnh từ Web Dashboard.

3.3. THIẾT KẾ PHẦN CỨNG

3.3.1. Thiết kế khối Master

Khối Master là trung tâm điều phối của hệ thống, chịu trách nhiệm xử lý các tác vụ an ninh và giao tiếp người dùng. Thiết kế phần cứng bao gồm vi điều khiển ESP32 kết nối với các khối chức năng: xác thực (bàn phím, cảm biến sinh trắc học, thẻ từ), điều khiển cửa và hiển thị LCD. Khối này được cấu hình để sử dụng WiFi tích hợp nhằm gửi và nhận dữ liệu từ Firebase, đảm bảo đồng bộ thời gian thực với node Slave và Web Dashboard.

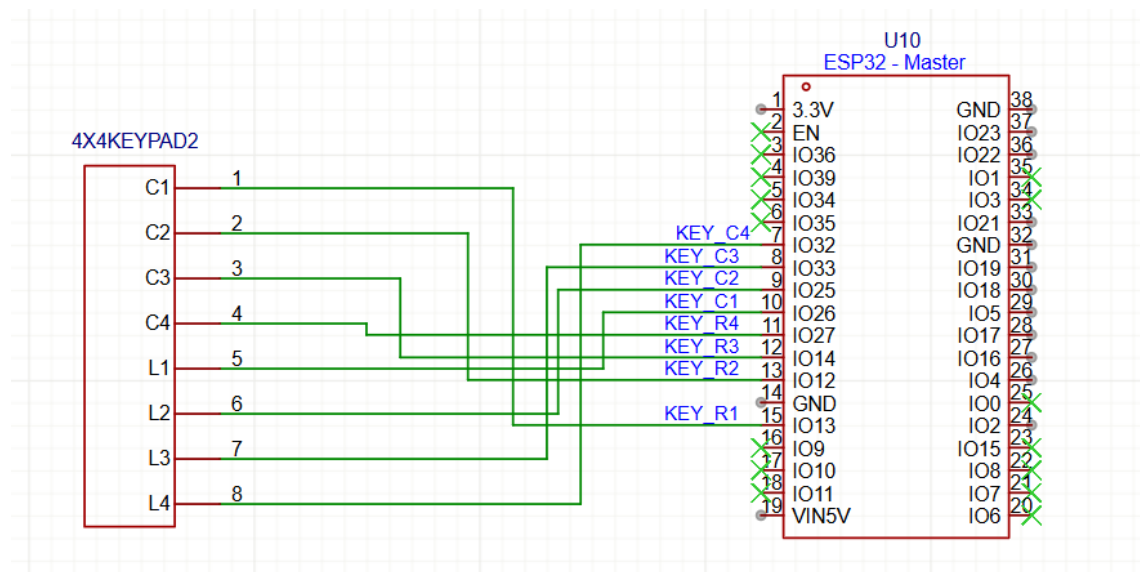
Vi điều khiển ESP32 được chọn do khả năng xử lý mạnh mẽ, hỗ trợ đa dạng giao thức I2C, SPI, UART, PWM và tích hợp WiFi, phù hợp với các ứng dụng IoT. Thiết kế phần cứng của khối Master bao gồm các khối ngoại vi được kết nối qua các chân GPIO, với nguồn điện 3.3V hoặc 5V tùy thuộc vào yêu cầu của từng module. Các khối chức năng được bố trí để tối ưu hóa hiệu suất, giảm thiểu nhiễu tín hiệu và đảm bảo tính ổn định trong vận hành. Thiết kế này cung cấp nền tảng

cứng cáp để thực hiện các tác vụ phức tạp, đáp ứng yêu cầu an ninh và tương tác người dùng của hệ thống.

Thiết kế khối Master đảm bảo khả năng tích hợp linh hoạt và dễ dàng mở rộng. Với số lượng chân GPIO lớn và hỗ trợ nhiều giao thức, khối Master này cho phép kết nối thêm các module trong tương lai mà không cần thay đổi cấu trúc mạch. Việc đồng bộ dữ liệu qua Firebase giúp giảm sự phụ thuộc vào giao tiếp trực tiếp với node Slave, đơn giản hóa thiết kế phần cứng và tăng độ tin cậy.

3.3.1.1. Module bàn phím

Trong hệ thống nhà thông minh, bàn phím ma trận 4x4 kết hợp với ESP32 thường được sử dụng để nhập mật khẩu điều khiển cửa tự động. Người dùng nhấn mã trên bàn phím, ESP32 sẽ quét và xác nhận mã đúng để kích hoạt relay mở khóa cửa. Khi kết nối với các nền tảng IoT như Firebase hệ thống có thể ghi lại lịch sử truy cập, gửi cảnh báo khi nhập sai mã, hoặc điều khiển cửa từ xa qua điện thoại. Ngoài ra, bàn phím còn có thể kết hợp với màn hình LCD để hiển thị các thông báo như “Access Granted” hay “Wrong Password”, giúp tăng tính bảo mật và thân thiện với người dùng. Với thiết kế đơn giản, độ bền cao và khả năng tương thích hoàn hảo với ESP32, bàn phím 4x4 là giải pháp nhập liệu hiệu quả cho các hệ thống khóa cửa thông minh, an ninh gia đình và điều khiển truy cập tự động.



Hình 3.2: Module bàn phím kết nối với Master

Hình 3.2 trên mô tả sơ đồ nguyên lý kết nối giữa bàn phím ma trận 4x4 và vi điều khiển ESP32 – Master, trong đó 8 chân của bàn phím gồm 4 cột (C1–C4) và 4 hàng (L1–L4) được nối trực tiếp với các chân GPIO của ESP32 để thực hiện quá trình quét phím. Cụ thể, các chân cột C1 đến C4 lần lượt kết nối với GPIO26, GPIO25, GPIO33 và GPIO32; trong khi các chân hàng L1 đến L4 được nối với GPIO27, GPIO14, GPIO12 và GPIO13. ESP32 sẽ sử dụng kỹ thuật quét hàng–cột để phát hiện phím được nhấn bằng cách cấu hình các chân tương ứng ở chế độ đầu ra và đầu vào thích hợp. Sơ đồ này hỗ trợ việc lập trình điều khiển bằng thư viện Keypad một cách thuận tiện, giúp bàn phím hoạt động hiệu quả trong các ứng dụng như hệ thống khóa cửa thông minh hoặc điều khiển truy cập. Dựa trên sơ đồ này, về kết nối chân được trình bày trong Bảng 3.1 dưới đây:

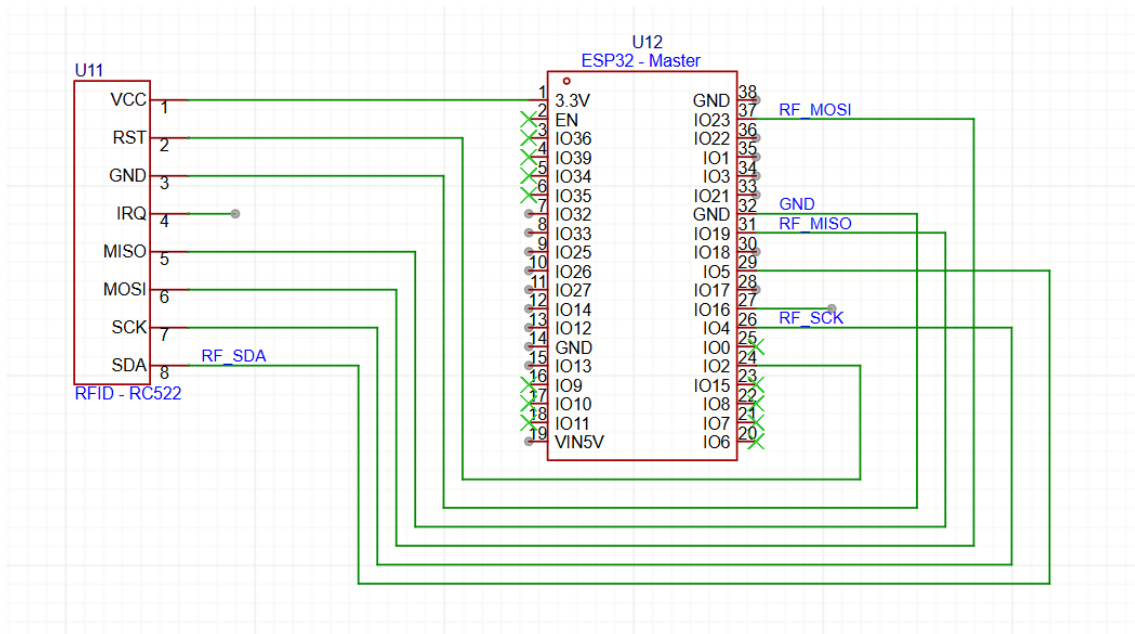
Bảng 3.1: Bảng kết nối chân giữa Keypad và ESP32 – Master

STT	Cảm biến	Chân	ESP32 – Master
1	Keypad 4x4	R1	IO13
2		R2	IO12
3		R3	IO14
4		R4	IO27
5		C1	IO26
6		C2	IO25
7		C3	IO33
8		C4	IO32

3.3.1.2. Module RFID

Trong hệ thống nhà thông minh, RFID kết hợp với ESP32 là giải pháp hiệu quả để nhận dạng người dùng và điều khiển truy cập tự động. Khi quét thẻ, ESP32 nhận mã ID, xác minh tính hợp lệ và kích hoạt relay để mở khóa cửa hoặc điều khiển thiết bị. Nhờ tích hợp WiFi, ESP32 có thể gửi dữ liệu lên nền tảng IoT như Firebase để giám sát trạng thái cửa, lưu lịch sử truy cập và cảnh báo truy cập trái phép. Hệ thống còn hỗ trợ phân quyền theo ID thẻ, nâng cao tính bảo mật. Ngoài

ra, RFID có thể tích hợp với cảm biến, màn hình hoặc trợ lý ảo như Google Assistant để mở rộng chức năng điều khiển. Với độ tin cậy cao, tiêu thụ điện năng thấp và khả năng mở rộng linh hoạt, RFID-ESP32 là lựa chọn phổ biến trong các hệ thống mở cửa không chạm và tự động hóa nhà ở hiện đại.



Hình 3.3: Module RFID kết nối với Master

Hình 3.3 trên mô tả sơ đồ nguyên lý kết nối giữa module đọc thẻ RFID RC522 và vi điều khiển ESP32 – Master. Module RC522 có 8 chân chức năng, trong đó các chân RST, MOSI, MISO, SCK và SDA (SS) được kết nối trực tiếp với các chân GPIO của ESP32 để thực hiện giao tiếp SPI. Cụ thể, chân RST của module nối với GPIO4, MOSI nối với GPIO23, MISO nối với GPIO19, SCK nối với GPIO18 và SDA (chân chọn thiết bị) nối với GPIO5. Nguồn cung cấp cho module là chân VCC được kết nối với nguồn 3.3V của ESP32, và hai chân GND được nối với GND của ESP32 để đảm bảo nối đất ổn định. Trong sơ đồ này, giao tiếp SPI được sử dụng để truyền dữ liệu UID của thẻ từ RC522 về ESP32 phục vụ cho mục đích xác thực truy cập, thường được ứng dụng trong hệ thống khóa cửa thông minh hoặc kiểm soát ra vào. Cách bố trí chân rõ ràng giúp dễ dàng triển khai và lập trình trên phần mềm với thư viện MFRC522. Dựa trên sơ đồ này, về kết nối chân được trình bày trong Bảng 3.2 dưới đây:

Bảng 3.2: Bảng kết nối chân giữa RFID với ESP32 – Master

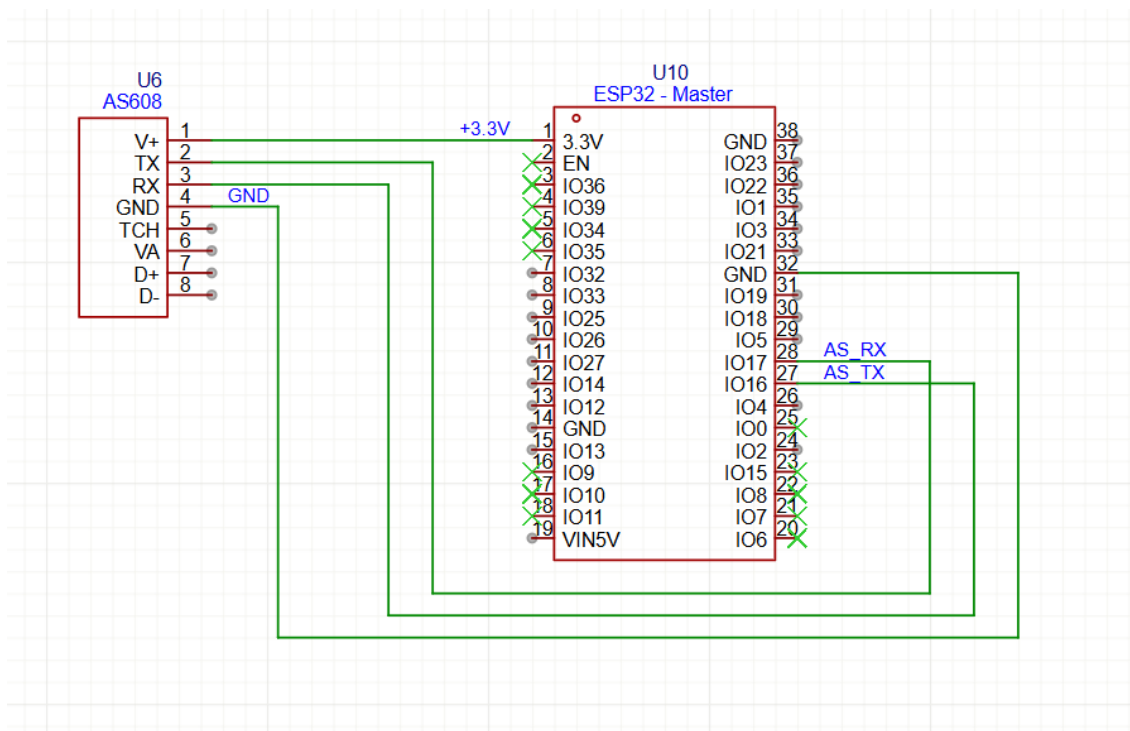
STT	Cảm biến	Chân	ESP32 – Master
1	RFID	3.3V	3.3V
2		GND	GND
3		SS	IO5
4		SCK	IO4
5		MOSI	IO23
6		MISO	IO19
7		RST	IO2

3.3.1.3. Module Cảm biến vân tay AS608

Cảm biến vân tay AS608 kết hợp với ESP32 là giải pháp hiệu quả cho hệ thống khóa cửa thông minh và kiểm soát truy cập tự động. Khi người dùng đặt ngón tay lên cảm biến, AS608 sẽ xử lý và xác thực vân tay trực tiếp, sau đó ESP32 sẽ kích hoạt relay mở cửa nếu vân tay hợp lệ.

Nhờ hỗ trợ giao tiếp UART và hoạt động ổn định ở 3.3V, AS608 dễ dàng tích hợp với ESP32. Hệ thống có thể đồng bộ dữ liệu lên nền tảng IoT như Firebase hoặc MQTT để ghi lại lịch sử truy cập, gửi cảnh báo khi có dấu hiệu truy cập trái phép, hoặc điều khiển cửa từ xa qua WiFi. Ngoài ra, khả năng lưu trữ tới 162 mẫu vân tay giúp quản lý nhiều người dùng một cách linh hoạt.

Với tốc độ nhận dạng nhanh, độ chính xác cao và khả năng hoạt động trong điều kiện ánh sáng yếu, AS608 là lựa chọn lý tưởng cho các ứng dụng như khóa cửa điện tử, hệ thống an ninh gia đình, máy chấm công và các thiết bị bảo mật cá nhân trong hệ thống nhà thông minh hiện đại.



Hình 3.4: Module vân tay kết nối với MASTER

Hình 3.4 trên mô tả sơ đồ nguyên lý kết nối giữa cảm biến vân tay AS608 và vi điều khiển ESP32 – Master. Module AS608 gồm 8 chân, trong đó các chân V+ và GND được nối với nguồn 3.3V và GND của ESP32 để cấp nguồn cho module. Giao tiếp giữa AS608 và ESP32 được thực hiện qua chuẩn UART, với chân TX của AS608 nối với chân GPIO27 (AS_RX) và chân RX nối với GPIO26 (AS_TX) của ESP32. Các chân còn lại như TCH (cảm biến chạm), VA, D+, và D– không được sử dụng trong sơ đồ này. Việc kết nối đúng thứ tự và sử dụng UART giúp ESP32 dễ dàng trao đổi dữ liệu vân tay với module AS608, phục vụ cho các chức năng như xác thực người dùng, mở khóa cửa, hoặc ghi nhận lịch sử truy cập trong hệ thống nhà thông minh. Dựa trên sơ đồ này, về kết nối chân được trình bày trong Bảng 3.3 dưới đây:

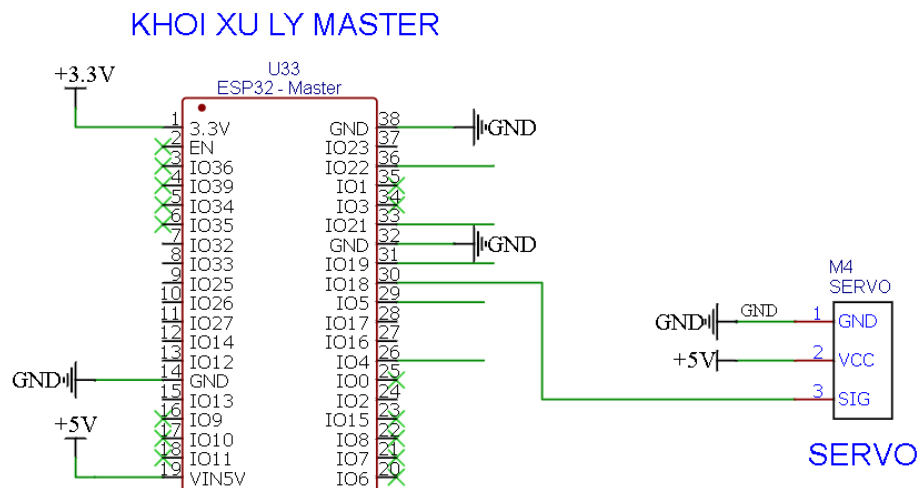
Bảng 3.3: Bảng kết nối chân giữa AS608 với ESP32 MASTER

STT	Cảm biến	Chân	ESP32 – Master
1	AS608	3.3V	3.3V
2		GND	GND
3		TX	IO17

4		RX	IO16
---	--	----	------

3.3.1.4. Khối điều khiển cửa

Khối điều khiển cửa sử dụng động cơ servo để thực hiện thao tác đóng/mở cửa dựa trên kết quả xác thực người dùng hoặc tín hiệu điều khiển từ xa. Nhờ khả năng điều khiển góc chính xác thông qua tín hiệu PWM, hệ thống đảm bảo hoạt động ổn định, an toàn và phản hồi nhanh. Thiết kế này dễ dàng tích hợp với ESP32-Master, phù hợp cho các ứng dụng kiểm soát truy cập trong hệ thống nhà thông minh. Việc cung cấp nguồn ổn định và điều khiển chính xác không chỉ tăng độ tin cậy mà còn kéo dài tuổi thọ của động cơ trong quá trình vận hành lâu dài.



Hình 3.5: Khối điều khiển cửa kết nối với Master

Hình 3.5, mô tả sơ đồ nguyên lý của khối điều khiển cửa kết nối với ESP32 – Master. Động cơ servo được kết nối với ESP32-Master qua một chân PWM, sử dụng tín hiệu điều chế độ rộng xung (PWM) với tần số 50 Hz và chu kỳ 20 ms để điều khiển góc quay từ 0° đến 180°. Nguồn điện 5V được cấp cho servo thông qua mạch nguồn riêng để đảm bảo công suất ổn định, với tụ lọc nhiễu 100 μ F để giảm dao động tín hiệu. ESP32-Master xử lý lệnh điều khiển từ kết quả xác thực (như xác nhận vân tay thành công) hoặc từ Web Dashboard qua Firebase, gửi tín hiệu PWM tương ứng để điều khiển servo. Thiết kế mạch đảm bảo dây tín hiệu ngắn và

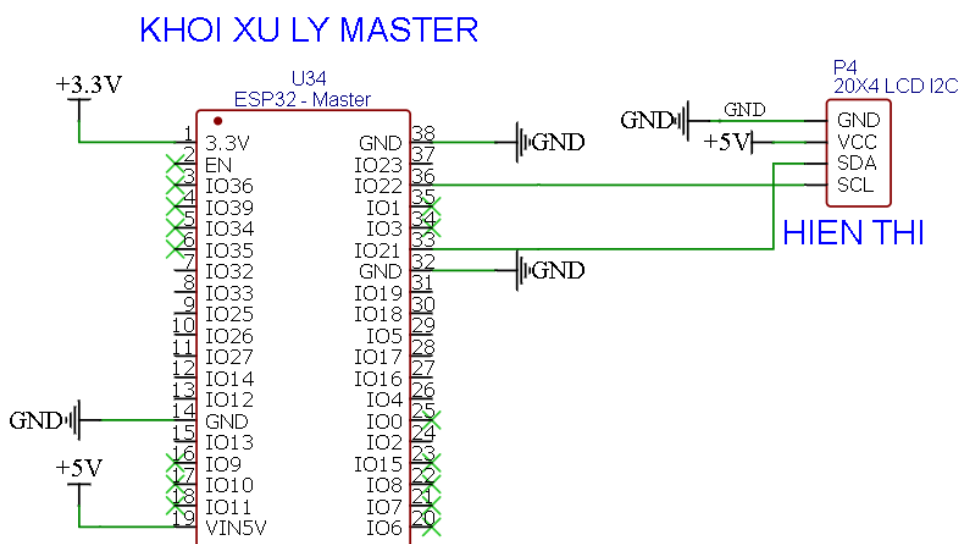
cách ly với các module khác để tránh nhiễu. Dựa trên sơ đồ này, về kết nối chân được trình bày trong Bảng 3.4 dưới đây:

Bảng 3.4: Bảng kết nối chân giữa Servo và ESP32 – Master

STT	Cảm biến	Chân	ESP32 – Master
1	Servo	5V	5V
2		GND	GND
3		PWM	IO18

3.3.1.5. Khởi hiển thị LCD

Khởi hiển thị LCD đóng vai trò là giao diện người dùng trực quan trong hệ thống, sử dụng màn hình LCD 20x4 giao tiếp I2C để hiển thị các thông tin quan trọng như trạng thái hoạt động, kết quả xác thực người dùng và các tùy chọn trong menu cài đặt. Nhờ khả năng hiển thị tối đa 20 ký tự trên 4 dòng, màn hình giúp người dùng dễ dàng theo dõi và thao tác với hệ thống tại chỗ. Trong các ứng dụng nhà thông minh, LCD còn hỗ trợ hiển thị cảnh báo, xác nhận truy cập hoặc hướng dẫn thiết lập, góp phần nâng cao tính thân thiện và hiệu quả trong quá trình vận hành.



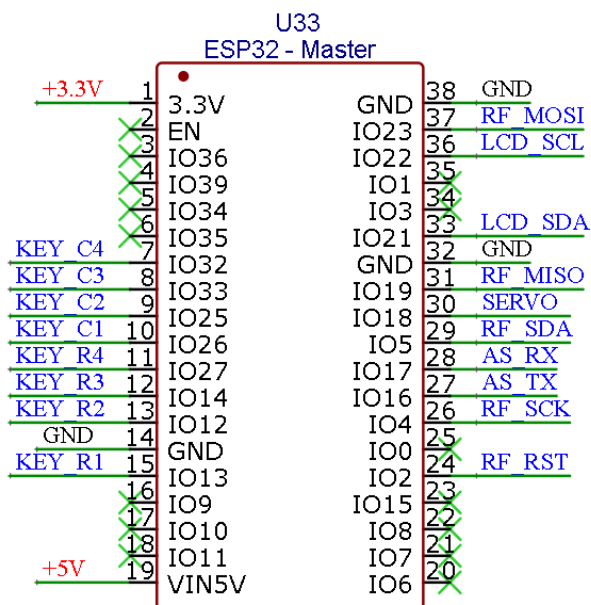
Hình 3.6: Khởi hiển thị LCD kết nối với Master

Hình 3.6, mô tả sơ đồ nguyên lý của khối hiển thị kết nối với ESP32 – Master. Màn hình LCD 20x4, sử dụng chip PCF8574, giao tiếp với ESP32-Master qua giao thức I2C với địa chỉ mặc định 0x27, sử dụng hai chân SDA và SCL. Điện trở kéo lên 2.2k Ω –4.7k Ω được thêm vào để đảm bảo tín hiệu I2C ổn định. Nguồn điện 5V được cấp cho LCD, với mạch điều chỉnh độ tương phản thông qua biến trở 10 k Ω . Màn hình hiển thị tối đa 20 ký tự trên 4 dòng, cho phép trình bày thông tin như trạng thái xác thực, thông báo hệ thống, hoặc các tùy chọn cài đặt. Dựa trên sơ đồ này, về kết nối chân được trình bày trong Bảng 3.5 dưới đây:

Bảng 3.5: Bảng kết nối chân giữa LCD và ESP32 – Master

STT	LCD 20x4 I2C	ESP32 – Master
1	5V	5V
2	GND	GND
3	SCL	IO22
4	SDA	IO21

3.3.1.6. Sơ đồ nguyên lý ESP32 – Master

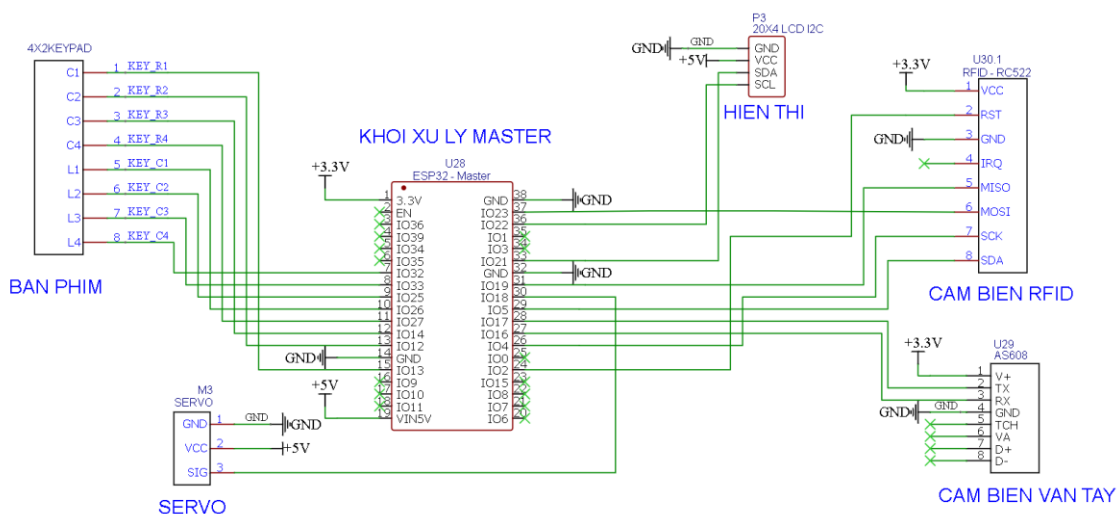


KHOI XU LY MASTER

Hình 3.7: Khối xử lý trung tâm Master

Hình 3.7, minh họa sơ đồ nguyên lý của ESP32 – Master kết nối với các khối xác thực, điều khiển cửa và hiển thị LCD. Sơ đồ được thiết kế để đảm bảo tín hiệu truyền thông ổn định, sử dụng các giao thức I2C, SPI, UART và PWM, với nguồn điện được cấp phù hợp cho từng module.

ESP32 – Master sử dụng các chân GPIO để kết nối với: Module xác thực qua giao thức SPI (cho thẻ từ) và UART (cho cảm biến sinh trắc học); bàn phím qua GPIO; động cơ servo qua PWM; và màn hình LCD qua I2C. Nguồn điện 3.3V được cấp cho ESP32 và các module giao tiếp số, trong khi nguồn 5V được sử dụng cho động cơ servo và LCD. Sơ đồ cũng bao gồm các điện trở kéo lên (pull-up) 2.2kΩ–4.7kΩ cho giao tiếp I2C và tụ lọc nhiễu để ổn định nguồn. WiFi tích hợp của ESP32 được cấu hình để kết nối với Firebase, không yêu cầu phần cứng giao tiếp bổ sung giữa Master và Slave. Hình 3.8, minh họa sơ đồ nguyên lý của ESP32-Master, thể hiện các kết nối chi tiết giữa vi điều khiển và các module ngoại vi.



Hình 3.8: Sơ đồ nguyên lý ESP32 – Master với các khối

Sơ đồ nguyên lý được thiết kế để giảm thiểu độ phức tạp, đảm bảo tính ổn định và dễ dàng bảo trì. Việc sử dụng các giao thức chuẩn và tối ưu hóa bố trí mạch giúp ESP32-Master vận hành hiệu quả, đáp ứng các yêu cầu về an ninh và điều khiển trong hệ thống nhà thông minh. Dựa trên sơ đồ này, về kết nối chân được trình bày trong Bảng 3.1 dưới đây:

Bảng 3.6: Bảng kết nối chân giữ các module với ESP32 Master

STT	Cảm biến	Chân	ESP32 – Master
1	AS608	3.3V	3.3V
2		GND	GND
3		TX	IO17
4		RX	IO16
5	RFID	3.3V	3.3V
6		GND	GND
7		SS	IO5
8		SCK	IO4
9		MOSI	IO23
10		MISO	IO19
11		RST	IO2
12	Keypad 4x4	R1	IO13
13		R2	IO12
14		R3	IO14
15		R4	IO27
16		C1	IO26
17		C2	IO25
18		C3	IO33
19		C4	IO32

3.3.2. Thiết kế khối Slave

Khối Slave là node hỗ trợ trong hệ thống giám sát nhà thông minh, chịu trách nhiệm thu thập dữ liệu môi trường, điều khiển thiết bị điện và hiển thị thông tin tại chỗ. Thiết kế phần cứng của khối này sử dụng vi điều khiển ESP32 làm nền tảng, kết nối với các module cảm biến, relay và hiển thị OLED thông qua các giao thức I2C, GPIO và PWM. Node này hoạt động độc lập, gửi và nhận dữ liệu qua Firebase Realtime Database bằng WiFi, đảm bảo đồng bộ với ESP32 – Master và Web Dashboard.

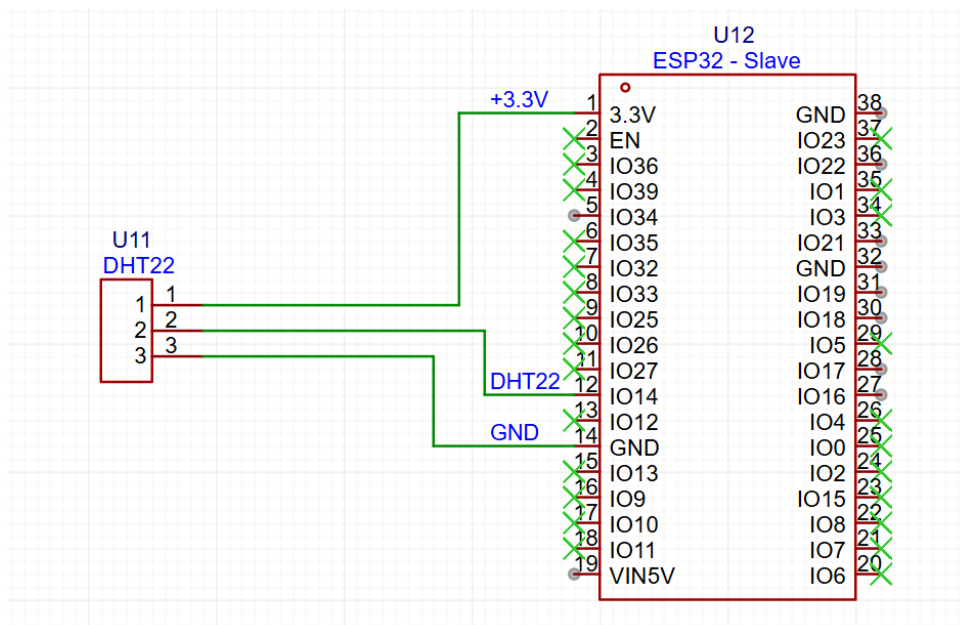
Vi điều khiển ESP32 được chọn nhờ khả năng xử lý hiệu quả, hỗ trợ đa dạng giao thức I2C, GPIO, PWM và tích hợp WiFi, phù hợp cho các ứng dụng IoT. Thiết kế phần cứng bao gồm các khối ngoại vi được kết nối qua các chân GPIO, với nguồn điện 3.3V hoặc 5V tùy thuộc vào yêu cầu của từng module. Các khối chức năng được bố trí để tối ưu hóa hiệu suất, giảm nhiễu tín hiệu và đảm bảo độ ổn định trong quá trình thu thập dữ liệu và điều khiển. Thiết kế này giúp khối Slave thực hiện các tác vụ giám sát và điều khiển một cách hiệu quả, hỗ trợ tích hợp linh hoạt trong hệ thống.

Thiết kế khối Slave đảm bảo khả năng mở rộng và vận hành ổn định. Với số lượng chân GPIO lớn và giao tiếp WiFi tích hợp, node này cho phép thêm các module cảm biến hoặc thiết bị mới mà không cần sửa đổi mạch. Việc đồng bộ dữ liệu qua Firebase giảm sự phụ thuộc vào giao tiếp trực tiếp với khối Master, đơn giản hóa thiết kế phần cứng và tăng độ tin cậy của hệ thống.

3.3.2.1. Cảm biến nhiệt độ DHT22

Cảm biến DHT22 khi kết hợp với ESP32 là giải pháp hiệu quả để giám sát và điều khiển nhiệt độ, độ ẩm trong nhà thông minh. DHT22 liên tục thu thập dữ liệu môi trường và truyền về ESP32 qua giao tiếp một dây. Với khả năng kết nối WiFi, ESP32 sẽ gửi dữ liệu lên nền tảng IoT Firebase để người dùng theo dõi từ xa qua điện thoại.

Không chỉ hiển thị thông số, hệ thống còn có thể tự động điều khiển các thiết bị như quạt, máy lạnh hoặc máy tạo ẩm khi phát hiện nhiệt độ hoặc độ ẩm vượt ngưỡng cài đặt, giúp duy trì môi trường sống lý tưởng. Nhờ tính chính xác cao, tiêu thụ năng lượng thấp và khả năng mở rộng linh hoạt, DHT22 – ESP32 là giải pháp hoàn hảo cho các ứng dụng theo dõi khí hậu phòng, điều khiển thông minh và tối ưu hóa năng lượng trong hệ sinh thái nhà thông minh.



Hình 3.9: Module nhiệt độ kết nối với Slave

Hình 3.9, mô tả sơ đồ nguyên lý của khối cảm biến kết nối với ESP32 – Slave. Module cảm biến nhiệt độ và độ ẩm DHT22 sử dụng giao tiếp GPIO một dây (single-wire), kết nối với một chân GPIO của ESP32, với điện trở kéo lên 4.7kΩ để ổn định tín hiệu. Module cảm biến ánh sáng BH1750 giao tiếp qua I2C, sử dụng hai chân SDA và SCL với địa chỉ mặc định 0x23, hỗ trợ đo cường độ ánh sáng với độ phân giải cao. Module cảm biến khí gas MQ-2 sử dụng giao tiếp analog qua chân ADC của ESP32, với nguồn điện 5V để đảm bảo độ nhạy trong phát hiện nồng độ khí gas. Các module được bố trí với dây tín hiệu ngắn và cách ly hợp lý để giảm nhiễu, đảm bảo dữ liệu môi trường được thu thập chính xác. Dựa trên sơ đồ này, về kết nối chân được trình bày trong Bảng 3.7 dưới đây:

Bảng 3.7: Bảng kết nối chân giữa cảm biến và ESP32 – Slave

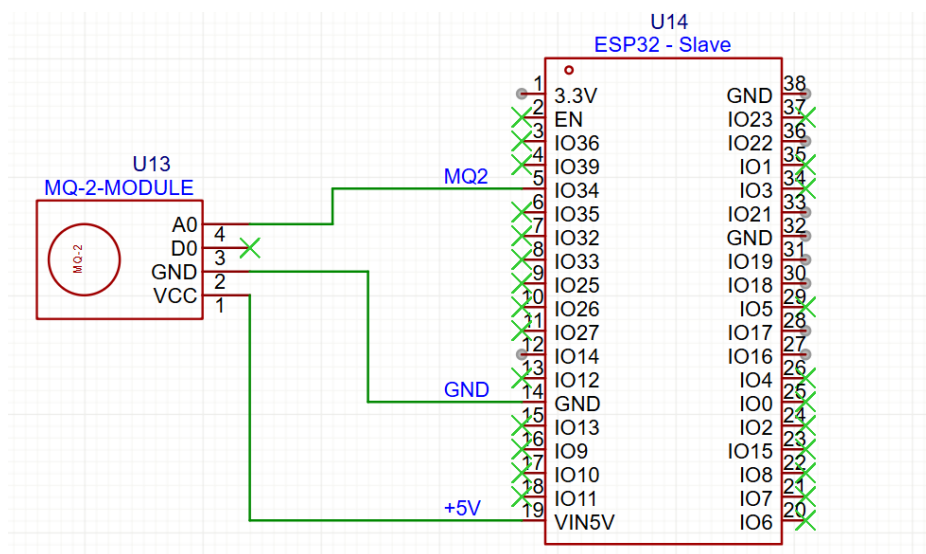
STT	Cảm biến	Chân	ESP32 – Slave
1	DHT22	3.3V	3.3V
2		GND	GND
3		DATA	IO14

3.3.2.2. Cảm biến khí gas MQ2

Cảm biến MQ2 kết hợp với ESP32 là giải pháp hiệu quả để phát hiện rò rỉ gas, khói và các khí dễ cháy trong nhà thông minh. MQ2 phát hiện sự thay đổi

nồng độ khí và xuất tín hiệu analog, được ESP32 đọc và xử lý. Khi nồng độ khí vượt ngưỡng an toàn, hệ thống có thể kích hoạt còi báo động, bật quạt thông gió, khóa van gas, hoặc gửi cảnh báo tức thì qua WiFi đến điện thoại người dùng thông qua nền tảng Firebase.

Ngoài phát hiện rò rỉ gas, hệ thống còn có thể mở rộng để giám sát cháy nổ trong nhà bếp, phòng chứa gas, nhà xưởng, giúp nâng cao an toàn và phòng ngừa sự cố. Nhờ khả năng kết nối mạng và tự động phản ứng nhanh, MQ2 – ESP32 là lựa chọn lý tưởng cho các giải pháp cảnh báo sớm và giám sát khí độc trong hệ thống nhà thông minh hiện đại.



Hình 3.10: Module khí gas kết nối với Slave

Hình 3.10 trên mô tả sơ đồ nguyên lý kết nối giữa cảm biến khí gas MQ-2 và vi điều khiển ESP32 – Slave. Module MQ-2 có 4 chân: VCC, GND, D0 (ngõ ra số), và A0 (ngõ ra tương tự). Trong sơ đồ này, chân VCC của cảm biến được nối với nguồn 5V, chân GND nối với GND của ESP32 để cấp nguồn hoạt động cho module. Chân D0 (ngõ ra số) được kết nối với GPIO33 của ESP32 (được đặt tên là MQ2) để nhận tín hiệu cảnh báo khi nồng độ khí vượt ngưỡng. Chân A0 (ngõ ra analog) không được sử dụng trong sơ đồ này. Với cách kết nối này, ESP32 có thể giám sát trạng thái khí gas trong môi trường và kích hoạt cảnh báo hoặc điều khiển thiết bị khi phát hiện khí độc hại, góp phần đảm bảo an toàn trong hệ thống

nhà thông minh. Dựa trên sơ đồ này, về kết nối chân được trình bày trong Bảng 3.8 dưới đây:

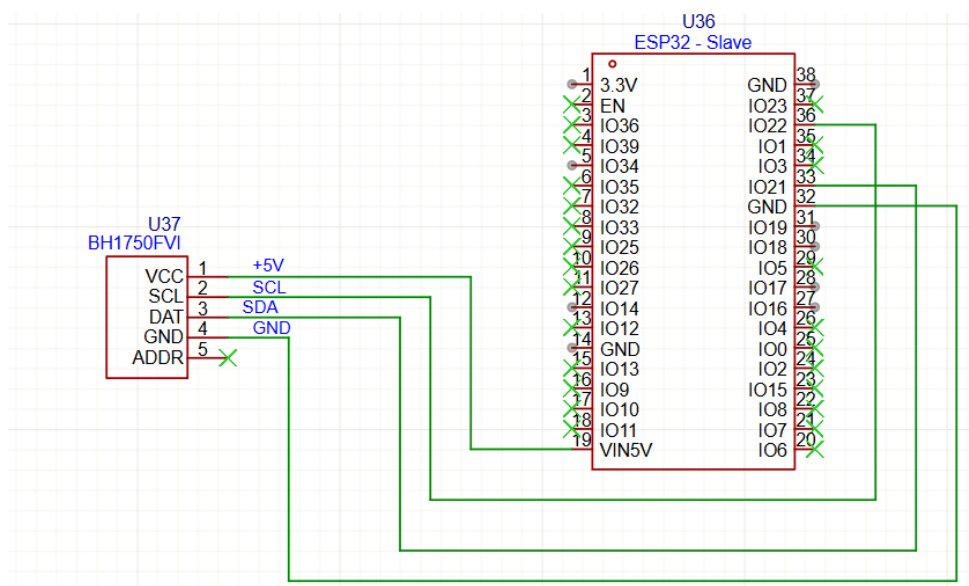
Bảng 3.8: Bảng kết nối chân MQ2 với ESP32 Slave

STT	Module	Chân	ESP32 – Slave
1	MQ-2	5V	5V
2		GND	GND
3		AO	IO34

3.3.2.3. Cảm biến ánh sáng BH1750

Cảm biến BH1750 kết hợp với ESP32 là giải pháp hiệu quả để giám sát và điều khiển ánh sáng tự động trong nhà thông minh. BH1750 đo cường độ ánh sáng môi trường và gửi dữ liệu số qua giao tiếp I2C đến ESP32. Nhờ kết nối WiFi, ESP32 có thể truyền dữ liệu lên nền tảng IoT như Firebase, Blynk hoặc MQTT để người dùng theo dõi ánh sáng từ xa qua smartphone.

Hệ thống có thể thiết lập ngưỡng để tự động bật đèn khi trời tối, đóng/mở rèm cửa theo ánh sáng tự nhiên, hoặc tối ưu hóa chiếu sáng để tiết kiệm điện năng. Với độ chính xác cao, tiêu thụ điện thấp và khả năng hoạt động lâu dài, BH1750 – ESP32 là lựa chọn lý tưởng cho các ứng dụng chiếu sáng thông minh, tự động hóa tiết kiệm năng lượng và tăng cường tiện nghi trong không gian sống hiện đại.



Hình 3.11: Module cảm biến ánh sáng kết nối với Slave

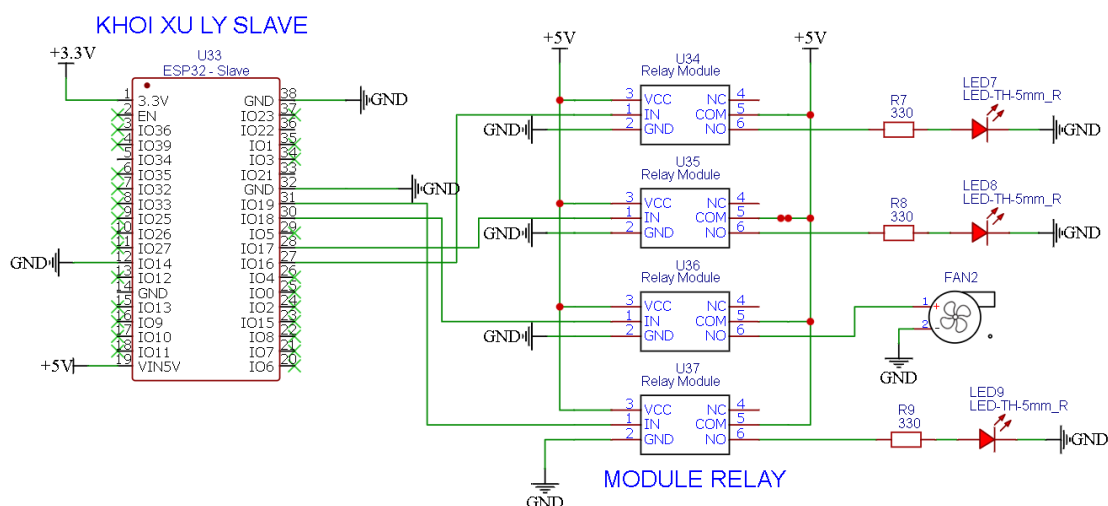
Hình 3.11 trên mô tả sơ đồ nguyên lý kết nối giữa cảm biến ánh sáng BH1750FVI và vi điều khiển ESP32 – Slave. Module BH1750FVI sử dụng giao tiếp I2C với hai chân chính: **SCL** (xung clock) và **SDA** (dữ liệu). Trong sơ đồ này, chân VCC của cảm biến được cấp nguồn 5V, chân GND nối với GND của ESP32. Chân SCL được kết nối với GPIO22 và SDA được kết nối với GPIO21 của ESP32 để truyền nhận dữ liệu I2C. Ngoài ra, chân ADDR được để hở, cho phép sử dụng địa chỉ mặc định 0x23. Kết nối này giúp ESP32 thu thập dữ liệu cường độ ánh sáng từ BH1750FVI, ứng dụng hiệu quả trong các hệ thống tự động điều chỉnh đèn, điều khiển rèm cửa hoặc giám sát môi trường ánh sáng trong nhà thông minh. Dựa trên sơ đồ này, về kết nối chân được trình bày trong Bảng 3.9 dưới đây:

Bảng 3.9: Bảng kết nối chân BH1750 với ESP32 Slave

STT	Module	Chân	ESP32 – Slave
1	BH1750	5V	5V
2		GND	GND
3		SCL	IO22
4		SDA	IO21

3.3.2.4. Khối điều khiển relay

Khối điều khiển relay sử dụng module relay để bật/tắt các thiết bị điện như đèn, quạt dựa trên lệnh từ Web Dashboard hoặc chế độ tự động theo dữ liệu cảm biến. Thiết kế này đảm bảo vận hành an toàn, hiệu quả và tích hợp dễ dàng với ESP32 – Slave.



Hình 3.12: Khối điều khiển relay kết nối với Slave

Hình 3.12, mô tả sơ đồ nguyên lý của khối điều khiển relay kết nối với ESP32 – Slave. Module relay một kênh hoặc đa kênh được kết nối với ESP32-Slave qua các chân GPIO, sử dụng tín hiệu mức logic 3.3V để kích hoạt. Nguồn điện 5V được cấp cho relay thông qua mạch nguồn riêng, với optocoupler cách ly để bảo vệ ESP32 khỏi dòng điện cao từ các thiết bị điện. Tụ lọc nhiễu 100 μ F được thêm vào để ổn định nguồn và diode chống dòng ngược được sử dụng để bảo vệ mạch relay. ESP32-Slave nhận lệnh điều khiển từ Firebase hoặc dựa trên ngưỡng dữ liệu cảm biến (như bật quạt khi nhiệt độ cao), đảm bảo độ trễ phản hồi dưới 2,5 giây. Dựa trên sơ đồ này, về kết nối chân được trình bày trong Bảng 3.10 dưới đây:

Bảng 3.10: Bảng kết nối chân giữa module relay với ESP32 – Slave

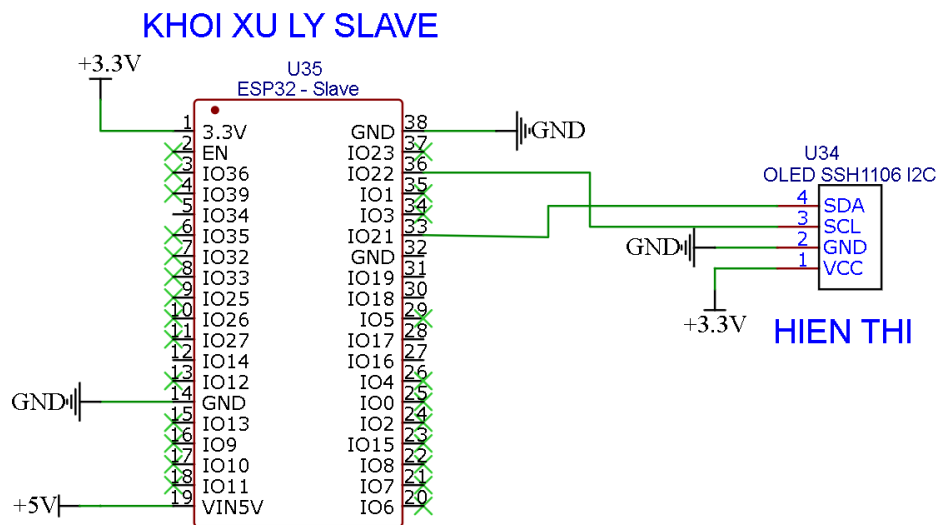
STT	Module	Chân	ESP32 – Slave
1	Module Relay 4 kênh	5V	5V
2		GND	GND
3		IN1	IO16
4		IN2	IO17
5		IN3	IO18
6		IN4	IO19

Về tổng quan, khối điều khiển relay cung cấp khả năng bật/tắt thiết bị điện chính xác, đáp ứng yêu cầu tự động hóa và điều khiển từ xa. Thiết kế cách ly và

nguồn điện ổn định đảm bảo độ bền và an toàn cho hệ thống trong các ứng dụng nhà thông minh.

3.3.2.5. Khối hiển thị OLED

Khối hiển thị OLED sử dụng màn hình OLED SH1106 giao tiếp I2C để hiển thị dữ liệu môi trường và trạng thái thiết bị tại chỗ. Thiết kế này cung cấp giao diện trực quan, dễ sử dụng, và tích hợp hiệu quả với ESP32-Slave.



Hình 3.13: Khối hiển thị OLED kết nối với Slave

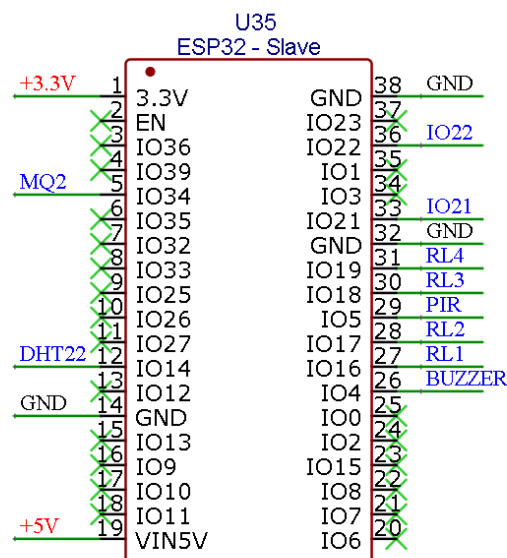
Hình 3.13, mô tả sơ đồ nguyên lý của khối hiển thị kết nối với ESP32 – Slave. Màn hình OLED SH1106, với độ phân giải 128x64 pixel, giao tiếp với ESP32-Slave qua giao thức I2C, sử dụng hai chân SDA và SCL với địa chỉ mặc định 0x3C. Điện trở kéo lên 2.2kΩ–4.7kΩ được thêm vào để đảm bảo tín hiệu I2C ổn định. Nguồn điện 3.3V hoặc 5V được cấp cho OLED, tùy thuộc vào model cụ thể, với tụ lọc nhiễu 100 μF để ổn định nguồn. Màn hình hiển thị các thông số như nhiệt độ, độ ẩm, ánh sáng, trạng thái relay, hoặc thông báo cảnh báo, được lập trình bằng thư viện Adafruit_SH1106. Thiết kế mạch đảm bảo dây tín hiệu ngắn và cách ly với các module khác để tránh nhiễu. Dựa trên sơ đồ này, về kết nối chân được trình bày trong Bảng 3.11 dưới đây:

Bảng 3.11: Bảng kết nối chân giữa OLED và ESP32 - Slave

STT	OLED I2C	ESP32 – Slave
1	5V	5V
2	GND	GND
3	SCL	IO22
4	SDA	IO21

Về tổng quan, khối hiển thị OLED đảm bảo giao diện người dùng tại chỗ thân thiện, đáp ứng yêu cầu hiển thị thông tin trực quan. Việc sử dụng I2C giúp giảm số lượng chân GPIO và tăng khả năng tích hợp, hỗ trợ vận hành hiệu quả trong các ứng dụng IoT.

3.3.2.6. Sơ đồ nguyên lý ESP32 – Slave



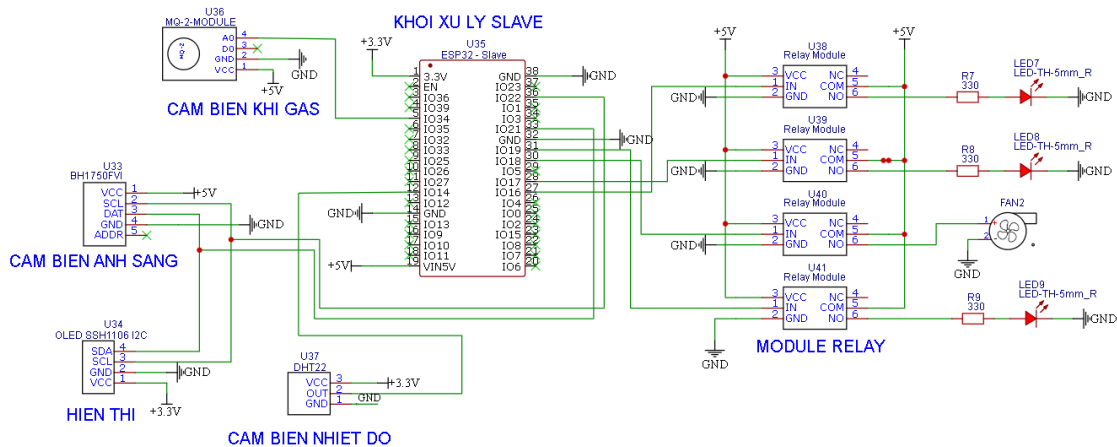
KHOI XU LY SLAVE

Hình 3.14: Khối xử lý trung tâm Slave

Hình 3.14, minh họa sơ đồ nguyên lý của ESP32 – Slave kết nối với các khối cảm biến, điều khiển relay và hiển thị OLED. Sơ đồ được thiết kế để đảm bảo truyền tín hiệu ổn định, sử dụng các giao thức I2C, GPIO, và PWM, với nguồn điện phù hợp cho từng module.

ESP32 – Slave sử dụng các chân GPIO để kết nối với: Các cảm biến môi trường qua giao thức I2C (cho cảm biến ánh sáng) và GPIO (cho cảm biến nhiệt độ/độ ẩm, khí gas); Module relay qua GPIO để điều khiển thiết bị điện và màn

hình OLED qua I2C. Nguồn điện 3.3V được cấp cho ESP32 và các module giao tiếp số, trong khi nguồn 5V được sử dụng cho relay và OLED. Sơ đồ bao gồm các điện trở kéo lên 2.2kΩ–4.7kΩ cho giao tiếp I2C và tụ lọc nhiễu 100 μF để ổn định nguồn. WiFi tích hợp của ESP32 được cấu hình để kết nối với Firebase. Hình 3.15 minh họa sơ đồ nguyên lý của ESP32-Slave, thể hiện các kết nối chi tiết giữa vi điều khiển và các module ngoại vi.

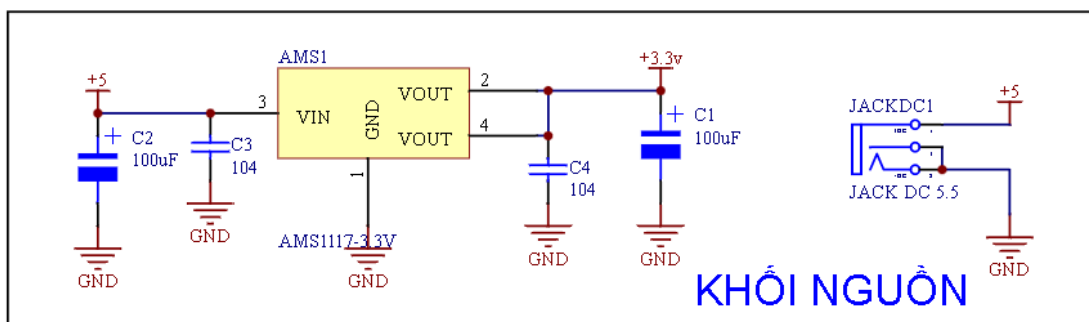


Hình 3.15: Sơ đồ nguyên lý ESP32 – Slave với các khối

Sơ đồ nguyên lý được thiết kế để giảm độ phức tạp, đảm bảo tính ổn định và dễ bảo trì. Việc sử dụng các giao thức chuẩn và tối ưu hóa bố trí mạch giúp ESP32 – Slave vận hành hiệu quả, đáp ứng yêu cầu giám sát môi trường và điều khiển thiết bị trong hệ thống nhà thông minh.

3.3.3. Thiết kế khối nguồn

Khối nguồn của hệ thống giám sát nhà thông minh được thiết kế để cung cấp điện áp ổn định cho vi điều khiển ESP32 (Master và Slave) và các module ngoại vi, đảm bảo vận hành liên tục, an toàn và hiệu quả trong các ứng dụng IoT. Thiết kế tập trung vào việc đáp ứng yêu cầu điện áp 3.3V và 5V, tối ưu hóa công suất tiêu thụ và giảm thiểu nhiễu điện để hỗ trợ kiến trúc Master – Slave, trong đó các node hoạt động độc lập và đồng bộ dữ liệu qua Firebase Realtime Database bằng WiFi. Khối nguồn được cấu hình để đáp ứng nhu cầu năng lượng của toàn bộ hệ thống, đảm bảo độ tin cậy và khả năng mở rộng.



Hình 3.16: Khởi nguồn

Hình 3.16, mô tả khởi nguồn sử dụng nguồn đầu vào từ bộ chuyển đổi AC-DC, cung cấp hai mức điện áp chính: 3.3V cho ESP32 và các module giao tiếp số như cảm biến ánh sáng, thẻ từ, cảm biến sinh trắc học, bàn phím, OLED và 5V cho các module yêu cầu công suất cao như relay, động cơ servo, LCD, cảm biến khí gas. Bộ điều chỉnh điện áp tuyến tính AMS1117-3.3 được sử dụng để ổn định điện áp từ nguồn đầu vào 5V xuống 3.3V. Tụ lọc nhiều 100 μ F và 10 μ F được bố trí ở đầu ra của bộ điều chỉnh để giảm dao động điện áp. Bảng 3.7, trình bày các thông số kỹ thuật tổng quát của khởi nguồn. Và Bảng 3.12, chi tiết hóa điện áp, dòng điện và công suất tiêu thụ của các thành phần chính trong hệ thống.

Bảng 3.12: Thông số kỹ thuật của khởi nguồn

Thông số	Chi tiết
Điện áp đầu vào	5V
Điện áp đầu ra	5V hoặc 3.3V
Dòng điện tối đa	2A
Linh kiện chính	AMS1117, tụ lọc 100 μ F
Hiệu suất	> 85%

Bảng 3.13: Bảng thông số điện áp, dòng và công suất tiêu thụ của hệ thống

Linh kiện	Điện áp hoạt động (V)	Dòng tiêu thụ (mA)	Công suất tiêu thụ (mW)
ESP32	3.3V	80mA	264mW
OLED	3.3V	20mA	66mW

DHT22	3.3V	1.5mA	4.95mW
MQ-2	5V	150mA	750mW
BH1750	5V	0.5mA	2.5mW
Relay 4 kênh	5V	20mA	100mW
Servo	5V	100mA	500mW
LCD 20x4	5V	10mA	50mW
RFID	3.3V	13mA	42.9mW
Keypad	3.3V	1mA	3.3mW
AS608	3.3V	50mA	165mW

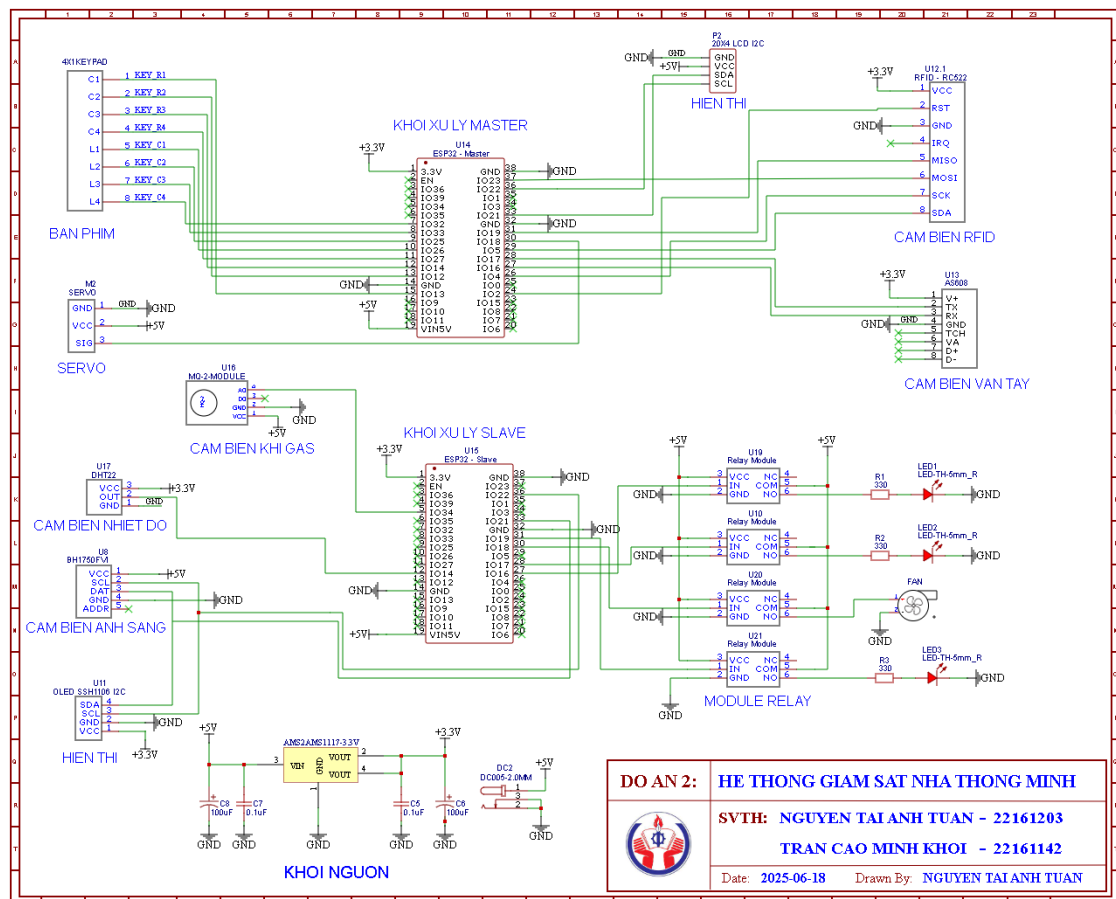
Bảng 3.13, trình bày các thông số về dòng điện, điện áp và công suất tiêu thụ của từng linh kiện trong hệ thống nhà thông minh, thể hiện nhu cầu năng lượng của từng khối chức năng. Giúp cung cấp cái nhìn tổng quan về mức tiêu thụ điện của từng linh kiện, giúp xác định nguồn cấp phù hợp để đảm bảo hệ thống hoạt động ổn định.

Nguồn DC 5V – 3A được chọn để cung cấp nguồn cho toàn hệ thống nhằm đảm bảo dòng điện tối đa 3000mA, đáp ứng nhu cầu tiêu thụ của tất cả các khối chức năng. Tổng dòng tiêu thụ của hệ thống, bao gồm OLED (66mA), ESP32 (80mA), cảm biến (152mA), nhận diện (64mA), relay (80mA cho 4 kênh), servo (100mA) và LCD (10mA), là khoảng 552mA, tương ứng với công suất 2760mW (2.76W), thấp hơn nhiều so với khả năng cung cấp của nguồn. Nguồn 5V – 3A không chỉ đáp ứng được yêu cầu về dòng điện, công suất mà còn có chi phí thấp, dễ thay thế và tương thích với các linh kiện phổ biến, giúp hệ thống hoạt động ổn định và dễ bảo trì.

Về tổng quan, khối nguồn được thiết kế với mạch phân tách để cấp điện riêng cho ESP32 – Master và ESP32 – Slave, đảm bảo mỗi node hoạt động độc lập mà không gây nhiễu lẫn nhau. Các module ngoại vi được cấp nguồn qua các đầu nối riêng biệt, với công tắc hoặc cầu chì bảo vệ (dòng định mức 3A) để tránh quá

tải. Cơ chế giám sát nguồn được tích hợp, sử dụng chân ADC của ESP32 để đo điện áp và cảnh báo khi nguồn không ổn định. Thiết kế này đảm bảo hệ thống vận hành liên tục, an toàn và tiết kiệm năng lượng, đáp ứng yêu cầu về độ tin cậy và khả năng mở rộng trong các ứng dụng nhà thông minh.

3.3.4. Sơ đồ nguyên lý toàn bộ hệ thống

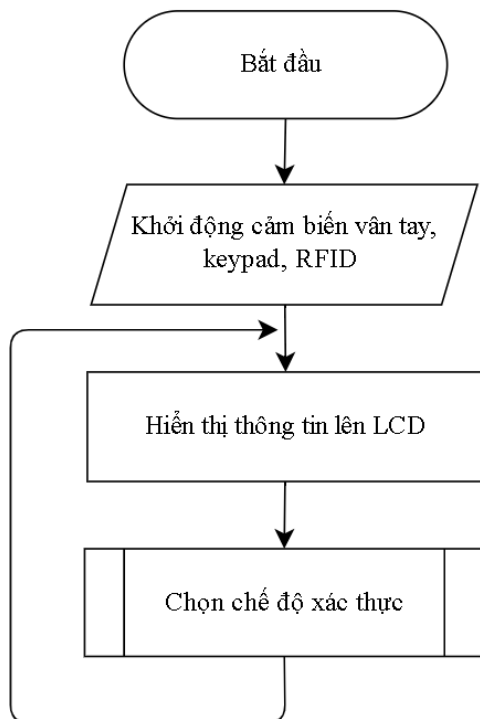


Hình 3.17: Sơ đồ nguyên lý toàn bộ hệ thống

Hình 3.17, mô tả sơ đồ nguyên lý toàn bộ hệ thống nhà thông minh, thể hiện mối quan hệ giữa các khối chức năng chính như vi điều khiển, cảm biến, nhận diện, hiển thị, điều khiển thiết bị.

3.4. THIẾT KẾ PHẦN MỀM

3.4.1. Lưu đồ giải thuật cho ESP32 – Master



Hình 3.18: Lưu đồ giải thuật chính ESP32 – Master

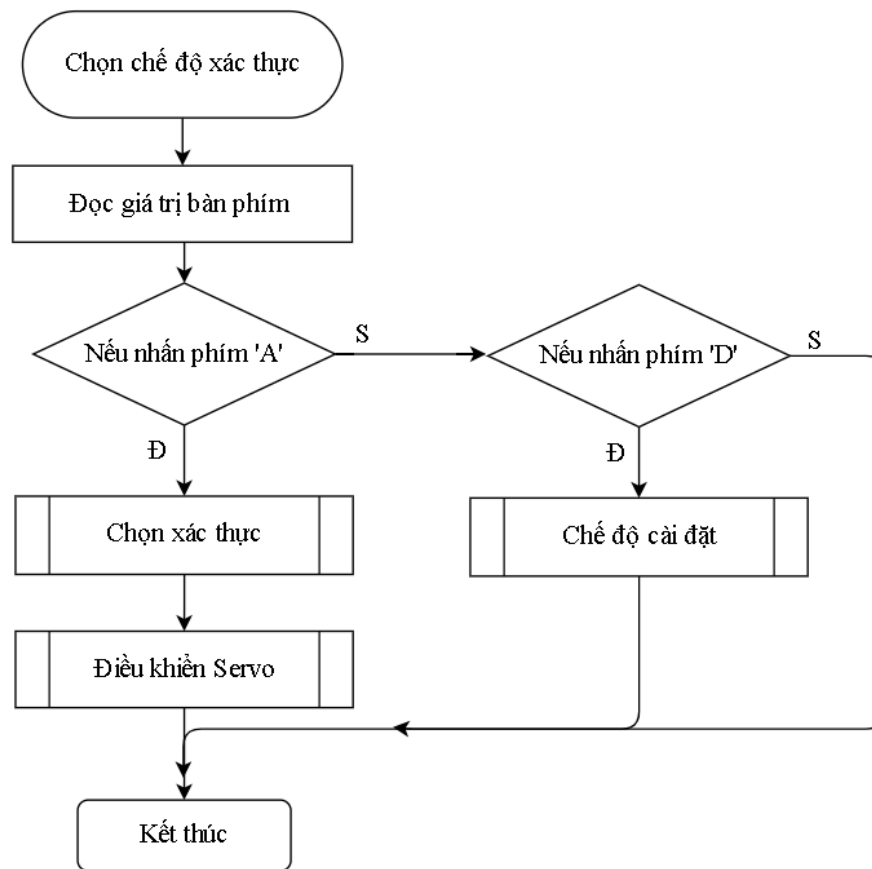
Hình 3.18, mô tả lưu đồ giải thuật chính cho Master. Hệ thống bắt đầu với giai đoạn khởi tạo ban đầu, sau đó chuyển sang giai đoạn khởi động các cảm biến vân tay, bàn phím và module RFID. Đây là bước thiết lập phần cứng cần thiết để hệ thống có thể nhận diện và xử lý các tín hiệu đầu vào từ người dùng.

Sau khi hoàn tất khởi động, hệ thống sẽ hiển thị thông tin lên màn hình LCD để cung cấp giao diện tương tác cho người dùng. Màn hình LCD sẽ hiển thị các thông báo hướng dẫn, trạng thái hệ thống hoặc kết quả xác thực.

Tiếp theo, hệ thống chuyển sang chế độ chờ để lựa chọn phương thức xác thực. Tại bước này, người dùng có thể chọn sử dụng cảm biến vân tay, nhập mã PIN qua bàn phím, hoặc sử dụng thẻ RFID để thực hiện việc xác thực danh tính.

Sau khi người dùng chọn và thực hiện xác thực, hệ thống sẽ quay trở lại giai đoạn hiển thị thông tin trên LCD để thông báo kết quả, sau đó tiếp tục vào chu trình chờ lựa chọn phương thức xác thực tiếp theo. Quá trình này được lặp lại liên tục, tạo thành một vòng lặp hoạt động ổn định của hệ thống.

3.4.1.1. Lưu đồ giải thuật con chế độ xác thực



Hình 3.19: Lưu đồ giải thuật con chế độ xác thực

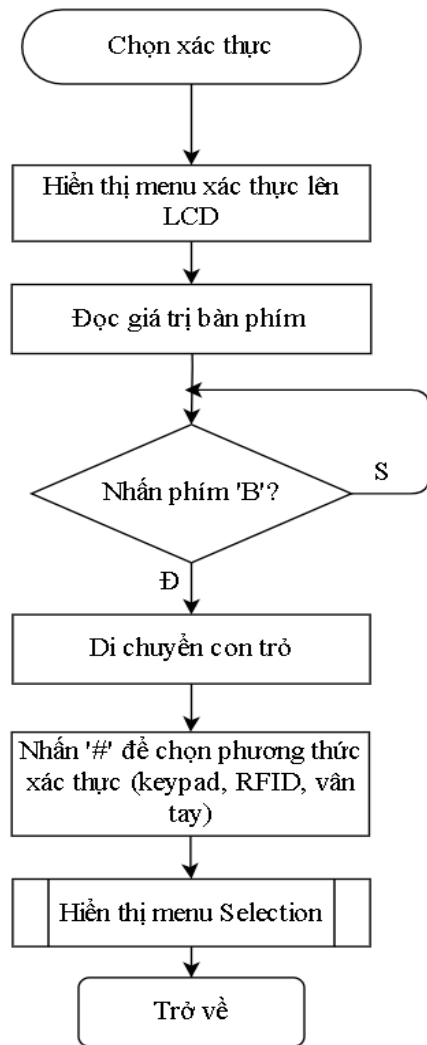
Hình 3.19, mô tả lưu đồ giải thuật con chế độ xác thực. Hệ thống bắt đầu bằng việc đọc giá trị từ bàn phím để xác định phương thức xác thực mà người dùng mong muốn. Quá trình này sẽ kiểm tra hai trường hợp chính thông qua cấu trúc điều kiện phân nhánh.

Trường hợp đầu tiên, khi người dùng nhấn phím 'A', hệ thống sẽ chuyển sang chế độ xác thực bằng mật khẩu, vân tay hoặc thẻ RFID. Sau khi hoàn tất quá trình xác thực, hệ thống sẽ gửi tín hiệu điều khiển servo motor để thực hiện việc mở khóa cửa nếu xác thực thành công, sau đó kết thúc quy trình.

Trường hợp thứ hai, khi người dùng nhấn phím 'D', hệ thống sẽ chuyển sang chế độ cài đặt để thêm mới hoặc chỉnh sửa thông tin người dùng như đăng ký vân tay mới, thẻ RFID mới hoặc thay đổi mã PIN. Sau khi hoàn tất việc cài đặt, hệ thống sẽ quay trở lại trạng thái chờ để tiếp tục đọc lựa chọn từ bàn phím.

Nếu không có phím nào được nhấn hoặc phím được nhấn không phải là 'A' hoặc 'D', hệ thống sẽ tiếp tục ở trạng thái chờ và lặp lại quá trình đọc bàn phím cho đến khi có lựa chọn hợp lệ từ người dùng.

3.4.1.2. Lưu đồ giải thuật con chọn xác thực



Hình 3.20: Lưu đồ giải thuật con chọn xác thực

Hình 3.20, mô tả lưu đồ con cho chọn xác thực. Hệ thống bắt đầu bằng việc hiển thị menu xác thực lên màn hình LCD để hướng dẫn người dùng các tùy chọn có sẵn. Sau đó, hệ thống đọc giá trị từ bàn phím để xác định lựa chọn của người dùng.

Quy trình kiểm tra điều kiện đầu tiên là việc xác định người dùng có nhấn phím 'B' hay không. Nếu người dùng nhấn phím 'B', hệ thống sẽ quay trở lại menu chính và kết thúc quy trình con này. Đây là chức năng cho phép người dùng thoát khỏi menu xác thực và quay lại giao diện chính của hệ thống.

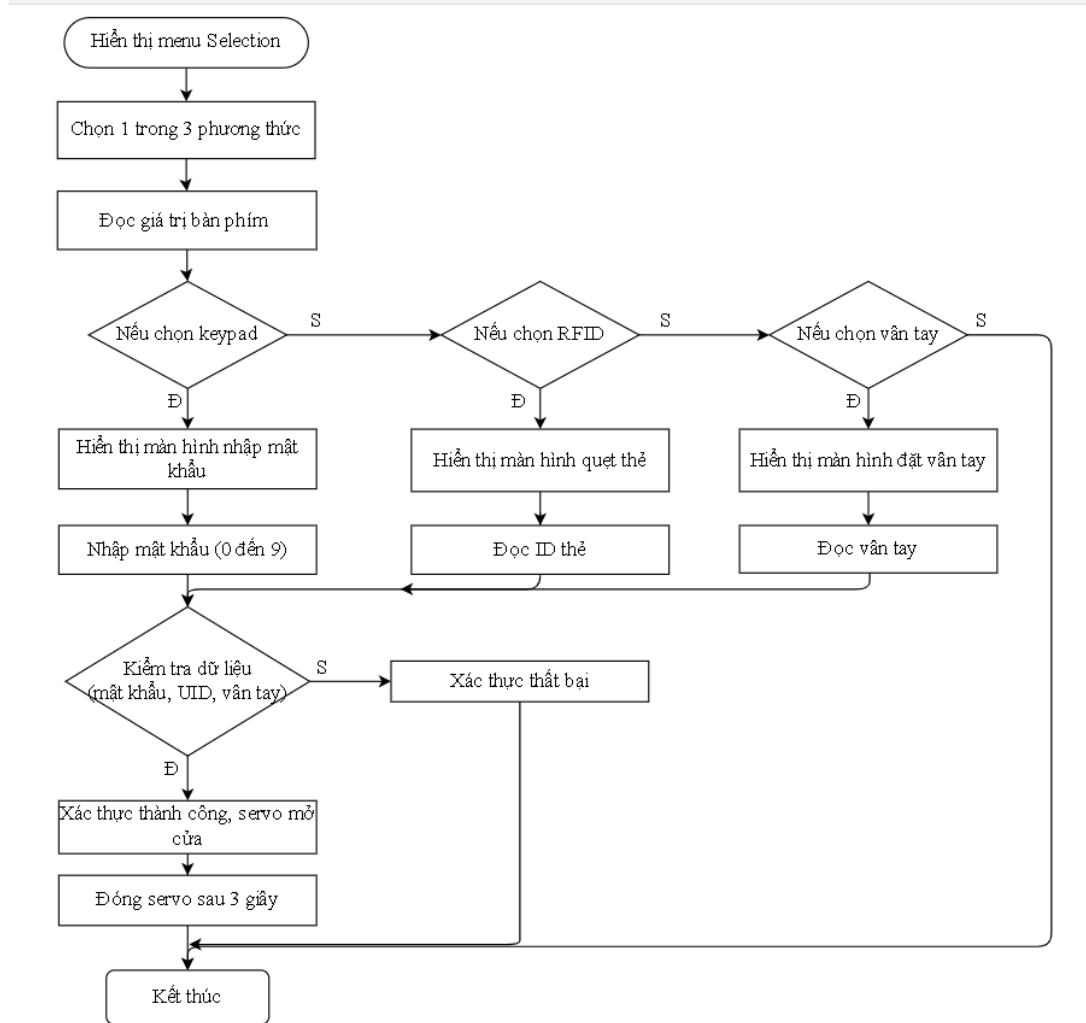
Trong trường hợp người dùng không nhấn phím 'B', hệ thống sẽ chuyển sang bước "Di chuyển con trỏ" để chuẩn bị cho việc lựa chọn phương thức xác thực cụ thể. Tiếp theo, hệ thống sẽ đọc ký tự '#' để xác nhận lựa chọn phương thức xác thực mong

muốn, bao gồm các tùy chọn như xác thực bằng bàn phím (mã PIN), thẻ RFID, hoặc cảm biến vân tay.

Sau khi xác định được phương thức xác thực, hệ thống sẽ hiển thị menu Selection tương ứng trên màn hình LCD để hướng dẫn người dùng thực hiện bước

tiếp theo. Cuối cùng, quy trình con sẽ trở về chương trình chính để tiếp tục xử lý quá trình xác thực theo phương thức đã được chọn.

3.4.1.3. Lưu đồ giải thuật con hiển thị menu Selection



Hình 3.21: Lưu đồ giải thuật con hiển thị menu Selection

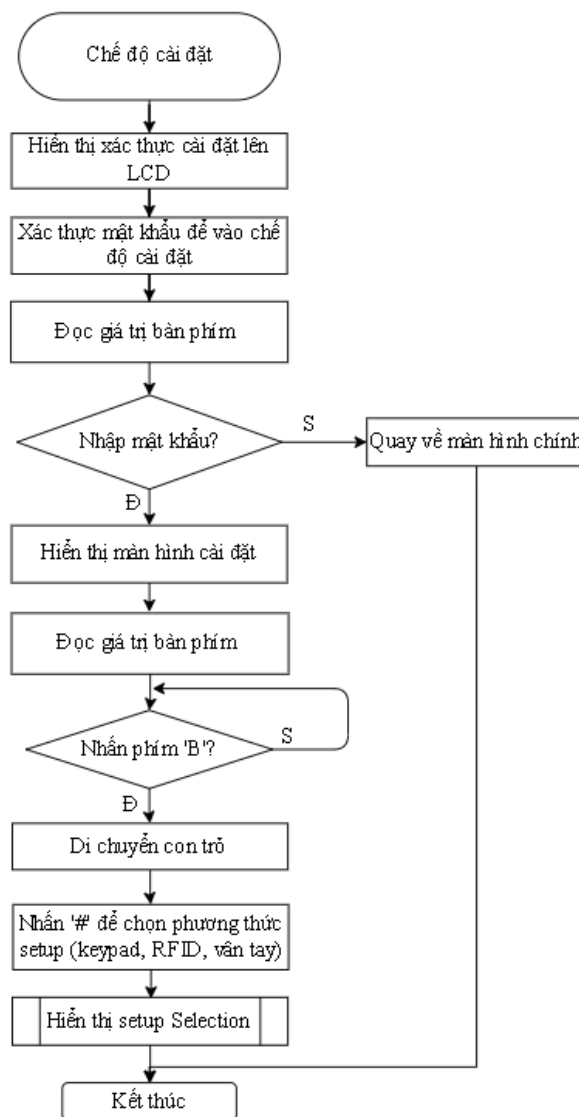
Hình 3.21, mô tả hệ thống bắt đầu bằng việc hiển thị menu Selection và cho phép người dùng chọn 1 trong 3 phương thức xác thực có sẵn. Sau đó, hệ thống đọc giá trị từ bàn phím để xác định lựa chọn của người dùng.

Quy trình phân nhánh theo ba trường hợp chính. Trường hợp đầu tiên khi người dùng chọn keypad, hệ thống sẽ hiển thị màn hình nhập mật khẩu và yêu cầu nhập mã khẩu có độ dài 9 ký tự. Trường hợp thứ hai khi chọn RFID, hệ thống hiển thị màn hình quét thẻ và đọc ID thẻ từ người dùng. Trường hợp thứ ba khi chọn vân tay, hệ thống hiển thị màn hình đặt vân tay và thực hiện quá trình đọc vân tay.

Sau khi thu thập dữ liệu xác thực từ một trong ba phương thức trên, hệ thống sẽ thực hiện kiểm tra dữ liệu bao gồm việc xác minh mật khẩu, UID thẻ RFID, hoặc dữ liệu vân tay với cơ sở dữ liệu đã được lưu trữ trước đó.

Nếu quá trình kiểm tra cho kết quả thành công, hệ thống sẽ thực hiện xác thực thành công và điều khiển servo motor để mở cửa. Servo sẽ hoạt động trong 3 giây để cho phép người dùng vào, sau đó tự động đóng lại. Trong trường hợp xác thực thất bại, hệ thống sẽ hiển thị thông báo lỗi và quay trở lại menu Selection để người dùng thử lại. Cuối cùng, quy trình kết thúc và trở về chương trình chính.

3.4.1.4. Lưu đồ giải thuật con chế độ cài đặt



Hình 3.22: Lưu đồ giải thuật con chế độ cài đặt

Hình 3.22, mô tả hệ thống bắt đầu bằng việc hiển thị các tùy chọn cài đặt lên màn hình LCD để hướng dẫn người dùng. Tiếp theo, hệ thống yêu cầu xác thực mật khẩu để vào chế độ cài đặt nhằm đảm bảo chỉ có người dùng được ủy quyền mới có thể truy cập vào các chức năng cài đặt hệ thống.

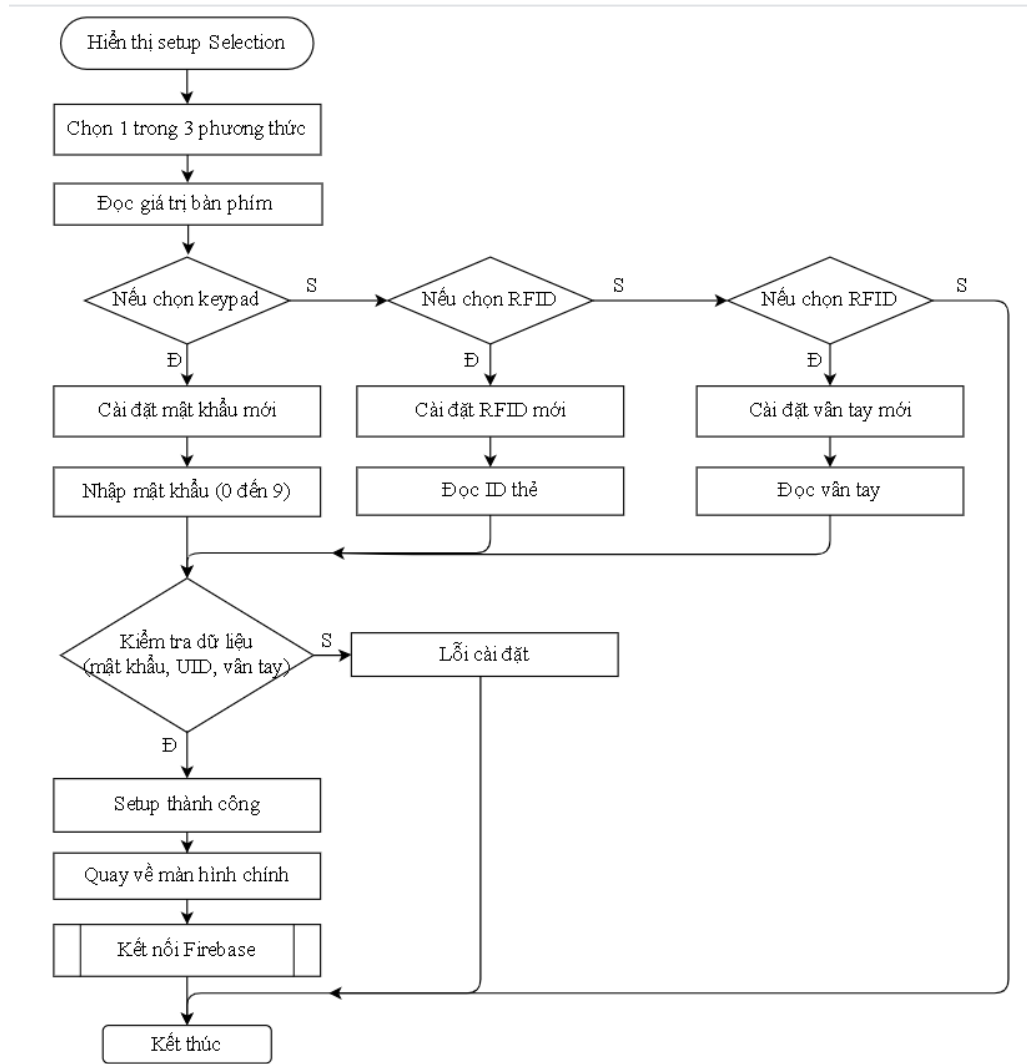
Sau khi hiển thị hướng dẫn, hệ thống đọc giá trị từ bàn phím để xác định lựa chọn của người dùng. Nếu người dùng chọn nhập mật khẩu, hệ thống sẽ tiến hành xác thực. Trong trường hợp mật khẩu không chính xác, hệ thống sẽ quay về màn hình chính và kết thúc quy trình cài đặt để ngăn chặn truy cập trái phép.

Khi mật khẩu được xác thực thành công, hệ thống sẽ hiển thị màn hình cài đặt và đọc giá trị từ bàn phím để xác định chức năng cài đặt cụ thể. Tại đây, hệ thống kiểm tra xem người dùng có nhấn phím 'B' để thoát khỏi menu cài đặt hay không. Nếu nhấn phím 'B', hệ thống sẽ quay về màn hình chính.

Trong trường hợp người dùng không chọn thoát, hệ thống sẽ chuyển sang bước "Di chuyển con trỏ" để chuẩn bị cho việc lựa chọn phương thức cài đặt. Tiếp theo, hệ thống đọc ký tự '#' để xác nhận lựa chọn phương thức setup, bao gồm các tùy chọn như cài đặt keypad (thay đổi mật khẩu), RFID (đăng ký thẻ mới), hoặc vân tay (đăng ký vân tay mới).

Cuối cùng, hệ thống hiển thị menu Setup Selection tương ứng với phương thức đã chọn để hướng dẫn người dùng thực hiện các bước cài đặt chi tiết, sau đó kết thúc quy trình và trở về chương trình chính.

3.4.1.5. Lưu đồ giải thuật con hiển thị setup Selection



Hình 3.23: Lưu đồ giải thuật con hiển thị setup Selection

Hình 3.23, mô tả hệ thống bắt đầu bằng việc hiển thị menu setup Selection và cho phép người dùng chọn 1 trong 3 phương thức cài đặt có sẵn. Sau đó, hệ thống đọc giá trị từ bàn phím để xác định lựa chọn của người dùng.

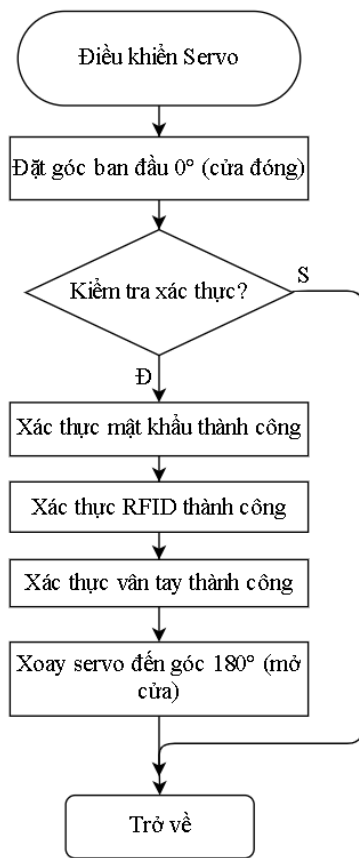
Quy trình phân nhánh theo ba trường hợp chính. Trường hợp đầu tiên khi người dùng chọn keypad, hệ thống sẽ thực hiện cài đặt mật khẩu mới và yêu cầu nhập mật khẩu có độ dài 9 ký tự. Trường hợp thứ hai khi chọn RFID, hệ thống

thực hiện cài đặt RFID mới và đọc ID thẻ từ người dùng để lưu trữ. Trường hợp thứ ba khi chọn vân tay, hệ thống thực hiện cài đặt vân tay mới và đọc dữ liệu vân tay từ cảm biến.

Sau khi thu thập dữ liệu từ một trong ba phương thức trên, hệ thống sẽ thực hiện kiểm tra dữ liệu để xác minh tính hợp lệ của mật khẩu, UID thẻ RFID, hoặc dữ liệu vân tay vừa được nhập. Nếu quá trình kiểm tra cho kết quả thất bại hoặc dữ liệu không hợp lệ, hệ thống sẽ hiển thị thông báo "Lỗi cài đặt" và quay trở lại menu setup Selection để người dùng thử lại.

Trong trường hợp kiểm tra dữ liệu thành công, hệ thống sẽ hiển thị thông báo "Setup thành công" và quay về màn hình chính. Tiếp theo, hệ thống thực hiện kết nối với Firebase để lưu trữ dữ liệu cài đặt mới vào cơ sở dữ liệu đám mây, đảm bảo thông tin được đồng bộ và lưu trữ an toàn. Cuối cùng, quy trình kết thúc và trở về chương trình chính.

3.4.1.6. Lưu đồ giải thuật con điều khiển servo



Hình 3.24: : Lưu đồ giải thuật con điều khiển servo

Hình 3.24, mô tả hệ thống bắt đầu bằng việc đặt servo motor về góc ban đầu 0° tương ứng với vị trí cửa đóng. Sau đó, hệ thống thực hiện kiểm tra xác thực để xác định liệu có cho phép mở cửa hay không thông qua cấu trúc điều kiện phân nhánh.

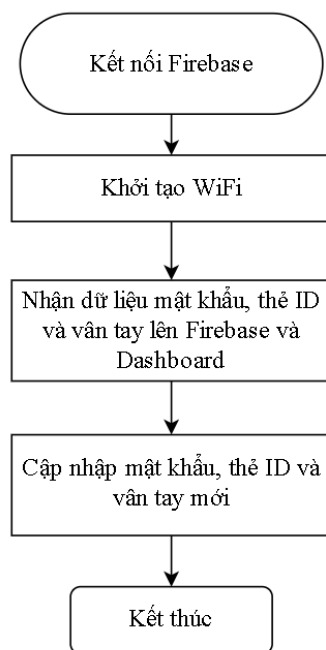
Nếu quá trình kiểm tra xác thực cho kết quả thất bại, hệ thống sẽ quay trở lại trạng thái chờ và tiếp tục ở góc 0° mà không thực hiện việc mở cửa. Điều này đảm bảo rằng cửa chỉ được mở khi có xác thực hợp lệ.

Trong trường hợp xác thực thành công, hệ thống sẽ thực hiện chuỗi các bước xác nhận theo thứ tự: xác thực mặt khẩu thành công, xác thực RFID thành công, và xác thực vân tay thành công. Các bước này

có thể được hiểu là việc ghi nhận kết quả xác thực từ các phương thức khác nhau hoặc là quá trình xác nhận đa lớp.

Sau khi hoàn tất tất cả các bước xác thực, hệ thống sẽ xoay servo motor đến góc 180° để thực hiện việc mở cửa. Góc 180° tương ứng với vị trí cửa mở hoàn toàn, cho phép người dùng đi qua. Cuối cùng, quy trình con kết thúc và trở về chương trình chính để tiếp tục các hoạt động khác của hệ thống.

3.4.1.7. Lưu đồ giải thuật con giao tiếp Firebase với Master



Hình 3.25: Lưu đồ giải thuật con giao tiếp Firebase với Master

Hình 3.25, mô tả hệ thống bắt đầu bằng việc khởi tạo kết nối WiFi để thiết lập kết nối internet cần thiết cho việc truy cập Firebase. Đây là bước quan trọng đầu tiên vì Firebase yêu cầu kết nối mạng ổn định để thực hiện các thao tác đồng bộ dữ liệu.

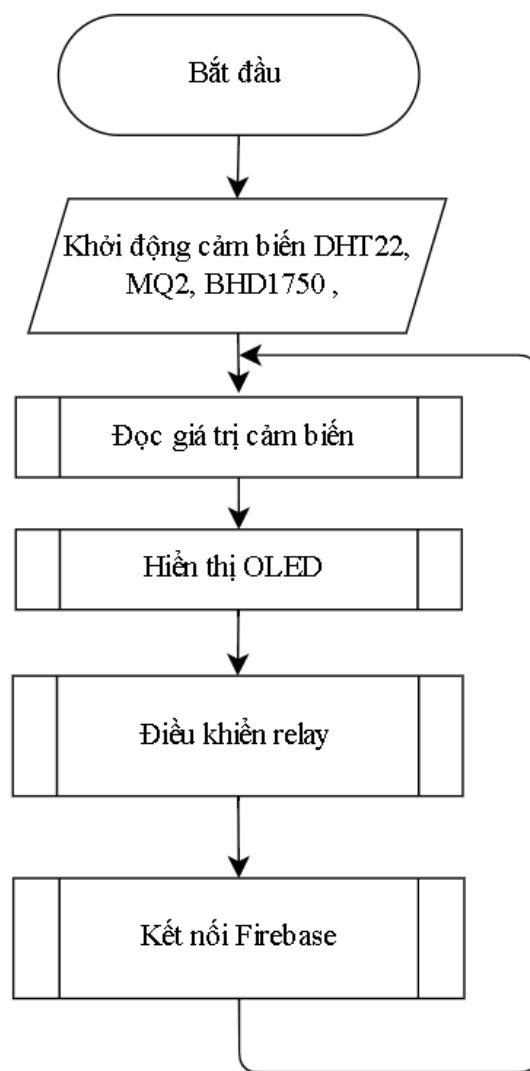
Sau khi kết nối WiFi được thiết lập thành công, hệ thống sẽ thực hiện việc nhận dữ liệu mật khẩu, thẻ ID và vân tay từ Firebase cũng như Dashboard. Quá trình này bao gồm việc tải xuống cơ sở dữ liệu người dùng đã được lưu trữ trước

đó trên Firebase và cập nhật thông tin từ Dashboard quản lý để đảm bảo hệ thống có dữ liệu xác thực mới nhất.

Tiếp theo, hệ thống sẽ thực hiện cập nhật mật khẩu, thẻ ID và vân tay mới vào cơ sở dữ liệu Firebase. Bước này cho phép đồng bộ hóa các thông tin xác thực mới được thêm vào hệ thống từ chế độ cài đặt, đảm bảo rằng dữ liệu được lưu trữ an toàn trên cloud và có thể truy cập từ Dashboard quản lý.

Cuối cùng, sau khi hoàn tất quá trình đồng bộ dữ liệu lên Firebase, quy trình kết thúc và trở về chương trình chính. Việc kết nối với Firebase không chỉ đảm bảo tính bảo mật của dữ liệu mà còn cho phép quản lý từ xa thông qua Dashboard, tạo điều kiện thuận lợi cho việc giám sát và quản lý hệ thống.

3.4.2. Lưu đồ giải thuật cho ESP32 – Slave



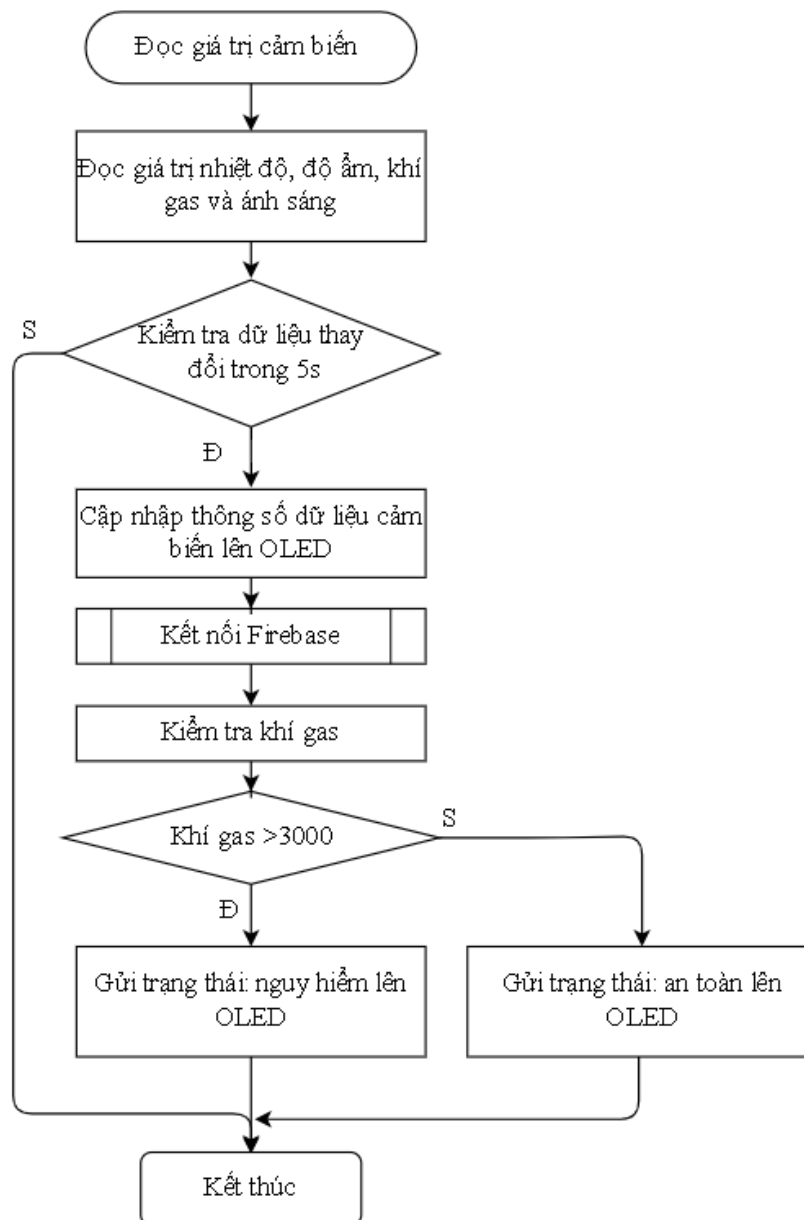
Hình 3.26: Lưu đồ giải thuật ESP – Slave

Hình 3.26, mô tả quá trình bắt đầu khi ESP32 khởi động và thực hiện giai đoạn khởi tạo các cảm biến bao gồm DHT22 (cảm biến nhiệt độ và độ ẩm), MQ2 (cảm biến khí gas), và BH1750 (cảm biến ánh sáng). Sau khi hoàn tất việc khởi tạo, hệ thống chuyển sang giai đoạn thu thập dữ liệu từ các cảm biến đã được cấu hình.

Dữ liệu thu thập được từ các cảm biến sẽ được hiển thị trực tiếp trên màn hình OLED để người dùng có thể theo dõi các thông số môi trường theo thời gian thực. Đồng thời, hệ thống thực hiện việc điều khiển relay dựa trên các ngưỡng đã được thiết lập trước, cho phép tự động hóa các thiết bị điện tương ứng với điều kiện môi trường.

Cuối cùng, toàn bộ dữ liệu cảm biến và trạng thái điều khiển được truyền đến Firebase thông qua kết nối WiFi, tạo ra một hệ thống giám sát từ xa. Sau khi hoàn tất việc gửi dữ liệu, hệ thống quay trở lại giai đoạn đọc cảm biến, tạo thành một vòng lặp liên tục để duy trì hoạt động giám sát và điều khiển không ngừng.

3.4.2.1. Lưu đồ giải thuật con đọc giá trị cảm biến



Hình 3.27: Lưu đồ giải thuật con đọc giá trị cảm biến

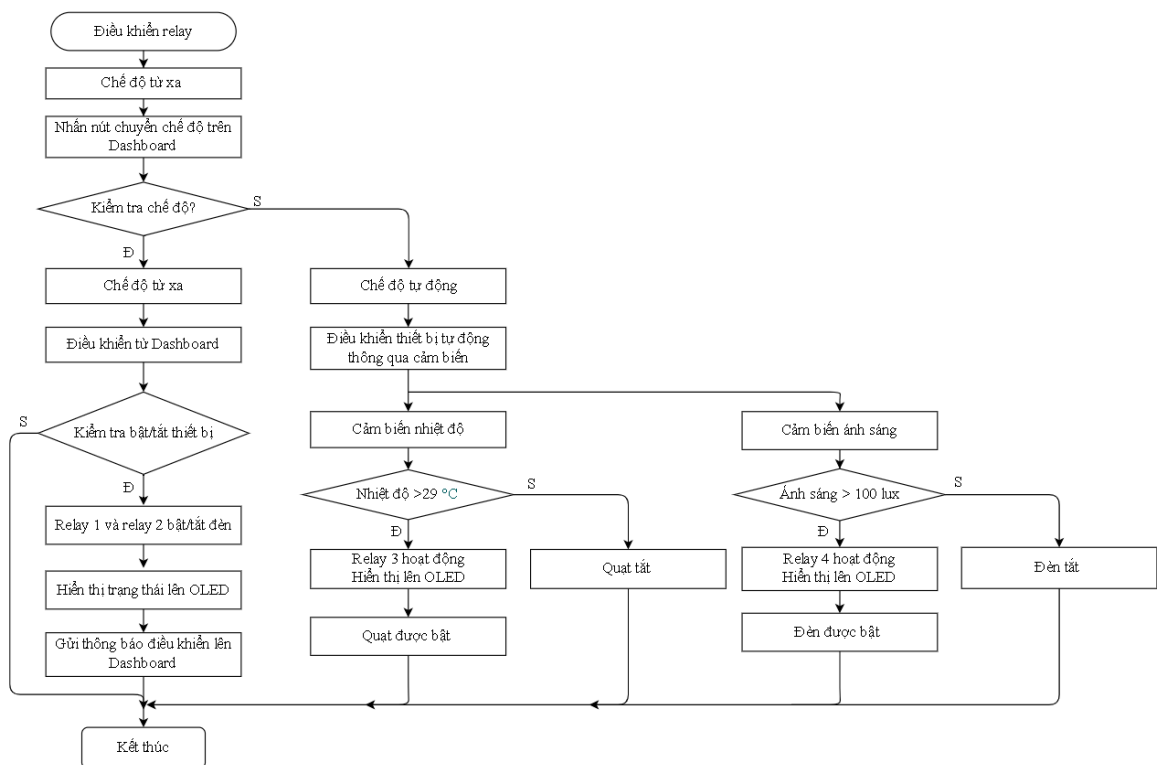
Hình 3.27, mô tả hệ thống bắt đầu bằng việc thu thập dữ liệu từ tất cả các cảm biến trong mạng, bao gồm nhiệt độ, độ ẩm, khí gas và ánh sáng. Sau khi hoàn tất việc đọc dữ liệu, hệ thống thực hiện kiểm tra tình trạng thay đổi của dữ liệu trong khoảng thời gian 5 giây để tránh việc cập nhật quá thường xuyên và tối ưu hóa hiệu suất.

Nếu dữ liệu đã thay đổi sau 5 giây, hệ thống sẽ cập nhật thông tin mới lên màn hình OLED để hiển thị các thông số hiện tại, sau đó thiết lập kết nối với

Firestore để đồng bộ dữ liệu lên cloud. Tiếp theo, hệ thống thực hiện việc kiểm tra nồng độ khí gas để đánh giá mức độ an toàn của môi trường.

Tại bước kiểm tra khí gas, nếu nồng độ vượt quá ngưỡng 3000 ppm, hệ thống sẽ gửi trạng thái cảnh báo "nguy hiểm" lên OLED và kích hoạt các biện pháp bảo vệ tương ứng. Ngược lại, nếu nồng độ khí gas nằm trong giới hạn an toàn, hệ thống hiển thị trạng thái "an toàn" trên OLED. Sau khi hoàn tất tất cả các thao tác kiểm tra và cập nhật, chương trình con kết thúc và trả về điều khiển cho chương trình chính.

3.4.2.2. Lưu đồ giải thuật con điều khiển relay

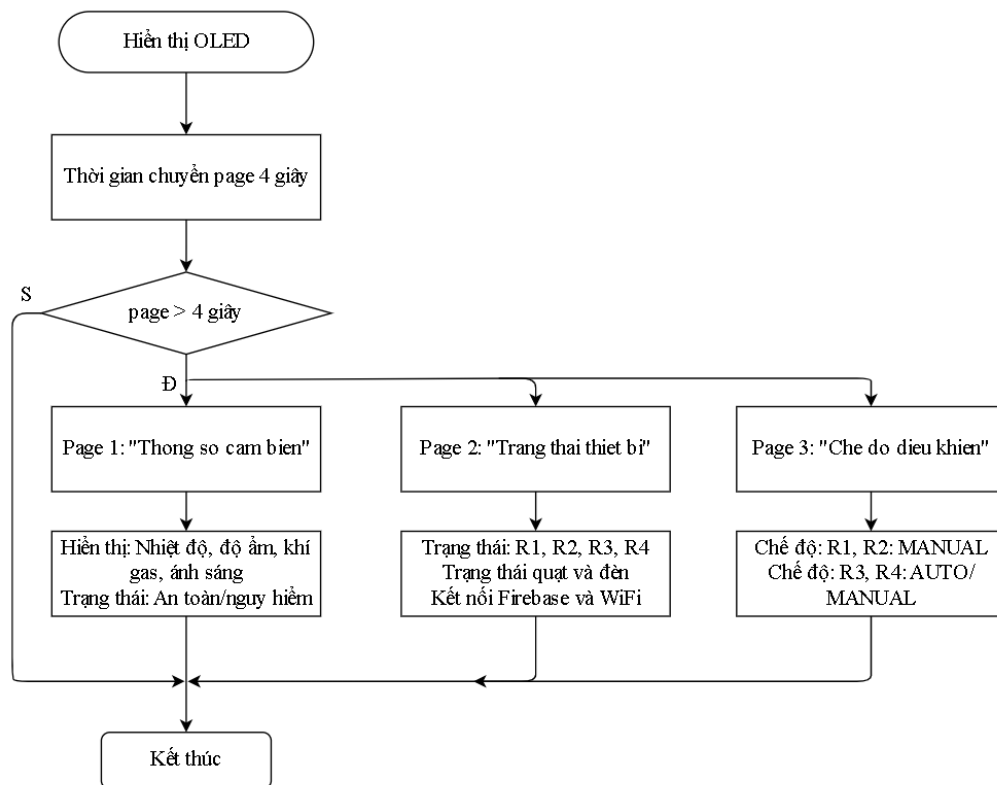


Hình 3.28: Lưu đồ giải thuật con điều khiển relay

Hình 3.28, mô tả chương trình bắt đầu bằng việc khởi tạo chế độ tự xa và nhận tín hiệu chuyển đổi chế độ từ Dashboard. Hệ thống kiểm tra trạng thái chế độ hoạt động hiện tại - nếu đang ở chế độ tự xa, hệ thống sẽ thực hiện điều khiển từ Dashboard thông qua việc kiểm tra trạng thái các thiết bị được bật/tắt. Khi thiết bị được bật, relay 1 và relay 2 sẽ được kích hoạt để bật đèn, đồng thời trạng thái hoạt động được hiển thị trên OLED và gửi thông báo điều khiển lên Dashboard.

Trong chế độ tự động, hệ thống thực hiện điều khiển thông minh dựa trên dữ liệu cảm biến môi trường. Đối với cảm biến nhiệt độ, khi nhiệt độ vượt quá 29°C, relay 3 sẽ được kích hoạt để bật quạt làm mát và hiển thị trạng thái trên OLED, ngược lại quạt sẽ được tắt khi nhiệt độ trở về mức bình thường. Tương tự, với cảm biến ánh sáng, khi cường độ ánh sáng dưới 100 lux, relay 4 sẽ kích hoạt để bật đèn chiếu sáng và cập nhật trạng thái lên OLED, còn khi ánh sáng đủ mạnh thì đèn sẽ tự động tắt. Sau khi hoàn tất tất cả các thao tác kiểm tra và điều khiển, chương trình con kết thúc và trả về điều khiển cho chương trình chính.

3.4.2.3. Lưu đồ giải thuật con hiển thị OLED



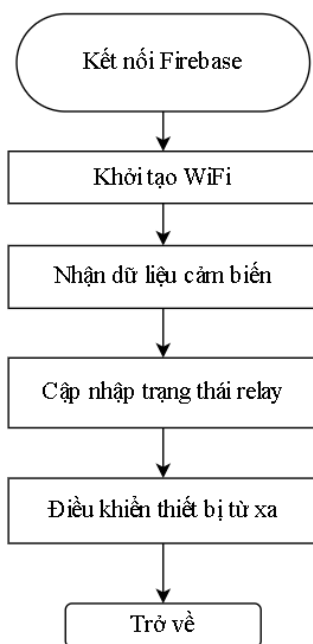
Hình 3.29: Lưu đồ giải thuật con hiển thị OLED

Hình 3.29, mô tả chương trình bắt đầu bằng việc khởi tạo thời gian chuyển trang với chu kỳ 4 giây, tạo ra một hệ thống hiển thị thông tin luân phiên. Hệ thống kiểm tra điều kiện thời gian để xác định khi nào cần chuyển sang trang tiếp theo, đảm bảo người dùng có thể xem được toàn bộ thông tin một cách có tổ chức.

Giao diện được chia thành ba trang thông tin chính với nội dung khác nhau. Trang 1 hiển thị "Thông số cảm biến" bao gồm các thông tin về nhiệt độ, độ ẩm, nồng độ khí gas và ánh sáng, kèm theo trạng thái an toàn hoặc nguy hiểm của môi trường. Trang 2 có tiêu đề "Trạng thái thiết bị" và hiển thị trạng thái hoạt động của các relay R1, R2, R3, R4 cùng với tình trạng kết nối quạt và đèn, bao gồm cả thông tin kết nối Firebase và WiFi. Trang 3 được đặt tên "Chế độ điều khiển" và hiển thị trạng thái của R1, R2 với chế độ MANUAL, cùng với R3, R4 có thể hoạt động ở chế độ AUTO hoặc MANUAL.

Sau khi hoàn tất việc hiển thị thông tin trên từng trang trong chu kỳ 4 giây, hệ thống tự động chuyển đổi giữa các trang để cung cấp thông tin toàn diện cho người dùng, sau đó chương trình con kết thúc và trả về điều khiển cho chương trình chính.

3.4.2.4. Lưu đồ giải thuật giao tiếp kết nối Firebase với Slave



Hình 3.30: Lưu đồ giải thuật giao tiếp Firebase với Slave

Hình 3.30, mô tả chương trình bắt đầu bằng việc khởi tạo kết nối WiFi để thiết lập đường truyền internet cho thiết bị ESP32. Sau khi kết nối WiFi thành công, hệ thống tiến hành thu thập toàn bộ dữ liệu từ các cảm biến trong mạng, bao gồm thông tin về nhiệt độ, độ ẩm, nồng độ khí gas và cường độ ánh sáng để chuẩn bị cho việc truyền dữ liệu.

Tiếp theo, hệ thống thực hiện cập nhật trạng thái hoạt động của các relay lên Firebase, cho phép theo dõi tình trạng điều khiển các thiết bị như quạt, đèn và các thiết bị khác từ xa. Đồng thời, chương trình thực hiện chức năng điều khiển thiết bị từ xa thông qua việc nhận và xử lý các lệnh điều khiển được gửi từ Dashboard hoặc ứng dụng di động.

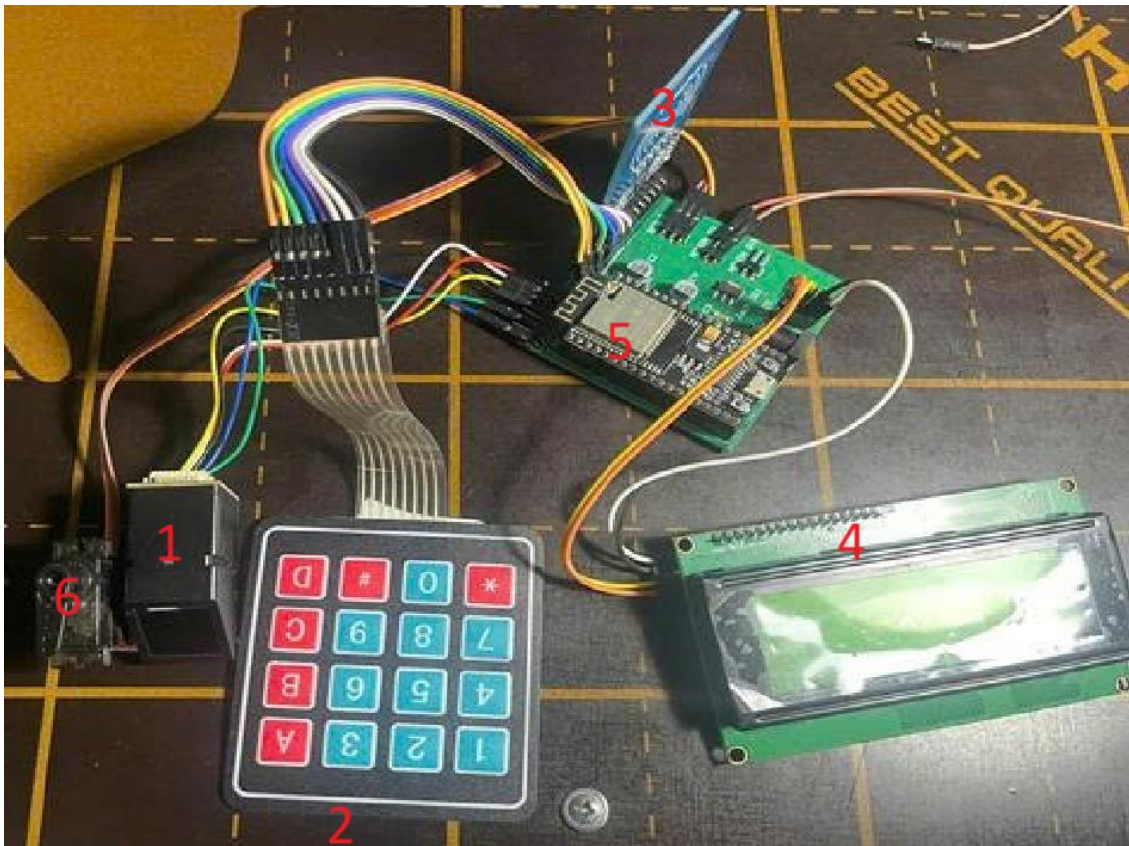
Sau khi hoàn tất việc đồng bộ hóa dữ liệu cảm biến, cập nhật trạng thái relay và xử lý các lệnh điều khiển từ xa, chương trình con kết thúc và trả về điều khiển cho chương trình chính. Quy trình này được lặp lại liên tục để duy trì kết nối và đồng bộ dữ liệu theo thời gian thực với hệ thống Firebase.

CHƯƠNG 4: KẾT QUẢ VÀ ĐÁNH GIÁ

4.1. KẾT QUẢ PHẦN CỨNG

4.1.1. Khối Master

Mô hình phần cứng thực tế của khối master được xây dựng dựa trên bo mạch chính là ESP32, tích hợp các module như keypad 4x4, cảm biến vân tay AS608 và module RFID để đảm nhiệm vai trò xử lý xác thực và giao tiếp với Firebase. Các thành phần linh kiện được lắp ráp trên một bo mạch PCB nhỏ gọn với các chân cắm module để thuận tiện cho việc thay đổi linh kiện bị hư hỏng.



Hình 4.1: Mạch PCB của khối Master

Hình 4.1, mô tả bố trí về phần cứng và có thể thấy rõ các thành phần chính có trong hệ thống bao gồm:

1. Cảm biến vân tay AS608.
2. Keypad 4x4.
3. Module RFID.

4. Màn hình LCD 20x4 I2C.
 5. Vi điều khiển ESP32.
 6. Động cơ servo.
- Khi cấp nguồn cho khối master, LCD sẽ hiển thị thông tin như hình 4.2:



Hình 4.2: LCD hiển thị thông tin khi cấp nguồn

Hình 4.2, hiển thị thông tin khi hệ thống được khởi động thành công với các thông tin cơ bản trên màn hình LCD như:

- Hàng 0: “HE THONG GIAM SAT”
- Hàng 1: “NHA THONG MINH”
- Hàng 2: “NGUYEN TAI ANH TUAN”
- Hàng 3: “DH SPKT TP.HCM”

4.1.1.1. Xác thực bằng bàn phím mật khẩu

Đầu tiên, khi nhấn phím ‘A’ trên keypad để vào chế độ xác thực, LCD sẽ hiển thị menu như ảnh sau:



Hình 4.3: Chế độ xác thực

Hình 4.3, mô tả khi nhấn phím ‘A’ trên bàn phím thì sẽ vào chế độ xác thực. Gồm có một trong ba phương thức xác thực là mật khẩu, thẻ ID và vân tay.

- Thứ nhất, khi chọn vào phương thức mật khẩu, hệ thống sẽ yêu cầu người dùng nhập mật khẩu để xác thực như hình 4.4:



Hình 4.4: Phương thức mật khẩu

- Khi đã nhập mật khẩu thành công thì servo sẽ mở cửa và LCD sẽ hiển thị như hình 4.5:



Hình 4.5: Xác thực mật khẩu thành công

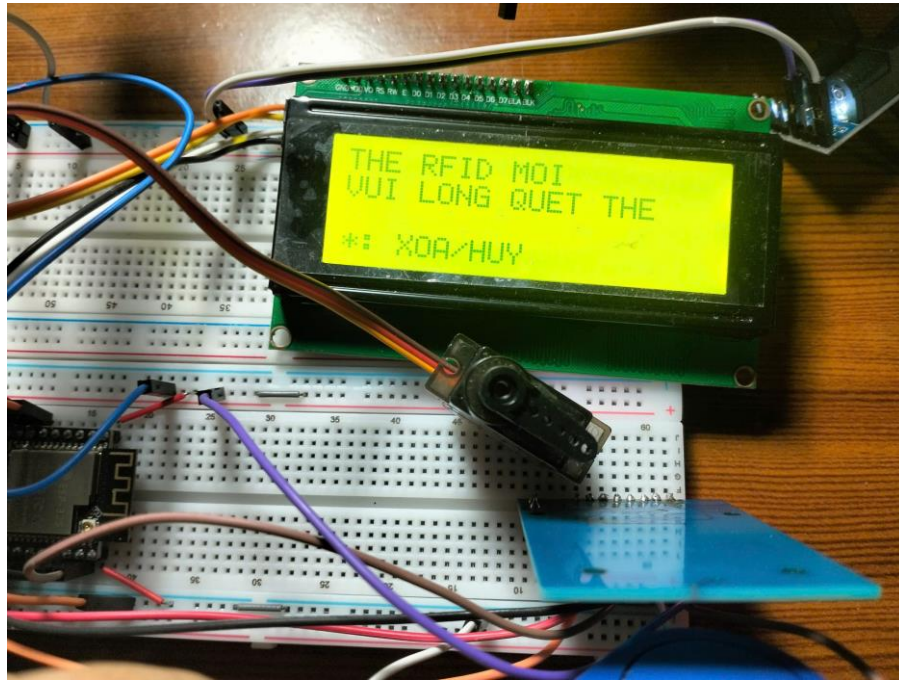
- Khi nhập mật khẩu sai thì servo sẽ không hoạt động và LCD sẽ báo như hình 4.6:



Hình 4.6: Xác thực mật khẩu sai

4.1.1.2. Xác thực bằng thẻ RFID

- Thứ hai, khi chọn vào phương thức thẻ RFID, hệ thống sẽ yêu cầu người dùng quét thẻ để xác thực như hình 4.7:



Hình 4.7: Phương thức thẻ RFID

- Khi đã quét thẻ thành công thì servo sẽ mở cửa và LCD sẽ hiển thị như hình 4.8:



Hình 4.8: Xác thực thẻ RFID thành công

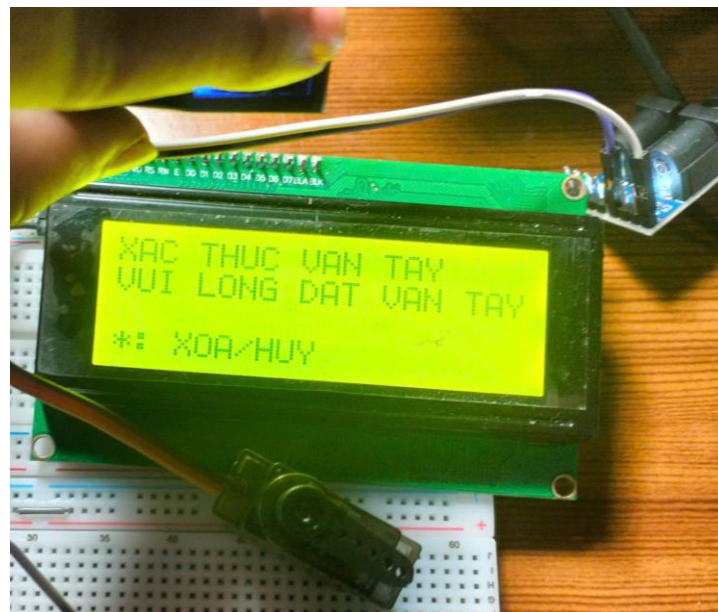
- Khi quét thẻ sai thì servo sẽ không hoạt động và LCD sẽ báo như hình 4.9:



Hình 4.9: Xác thực thẻ RFID sai

4.1.1.3. Xác thực bằng vân tay

- Thứ ba, khi chọn vào phương thức vân tay, hệ thống sẽ yêu cầu người dùng đặt vân tay vào để xác thực như hình 4.10:



Hình 4.10: Phương thức vân tay

- Khi đã đặt vân tay thành công thì servo sẽ mở cửa và LCD sẽ hiển thị như hình 4.11:



Hình 4.11: Xác thực vân tay thành công

- Khi đặt vân tay sai thì servo sẽ không hoạt động và LCD sẽ báo như hình 4.12:



Hình 4.12: Xác thực vân tay sai

4.1.1.4. Chế độ cài đặt

Khi nhấn phím 'D' trên bàn phím để vào chế độ cài đặt:

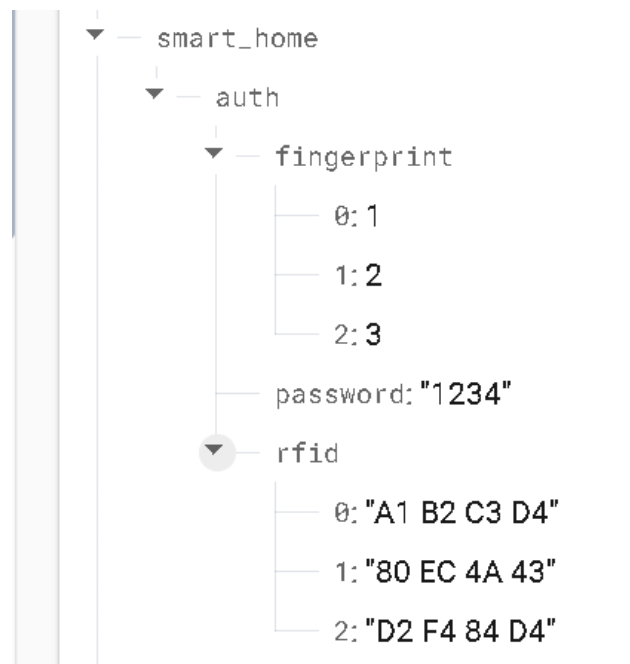


Hình 4.13: Chế độ cài đặt

Hình 4.13, mô tả khi nhấn phím ‘D’ trên bàn phím thì sẽ vào chế độ cài đặt. Gồm có ba phương thức cài đặt mới là mật khẩu, thẻ ID và vân tay.

4.1.1.5. Dữ liệu trên Firebase

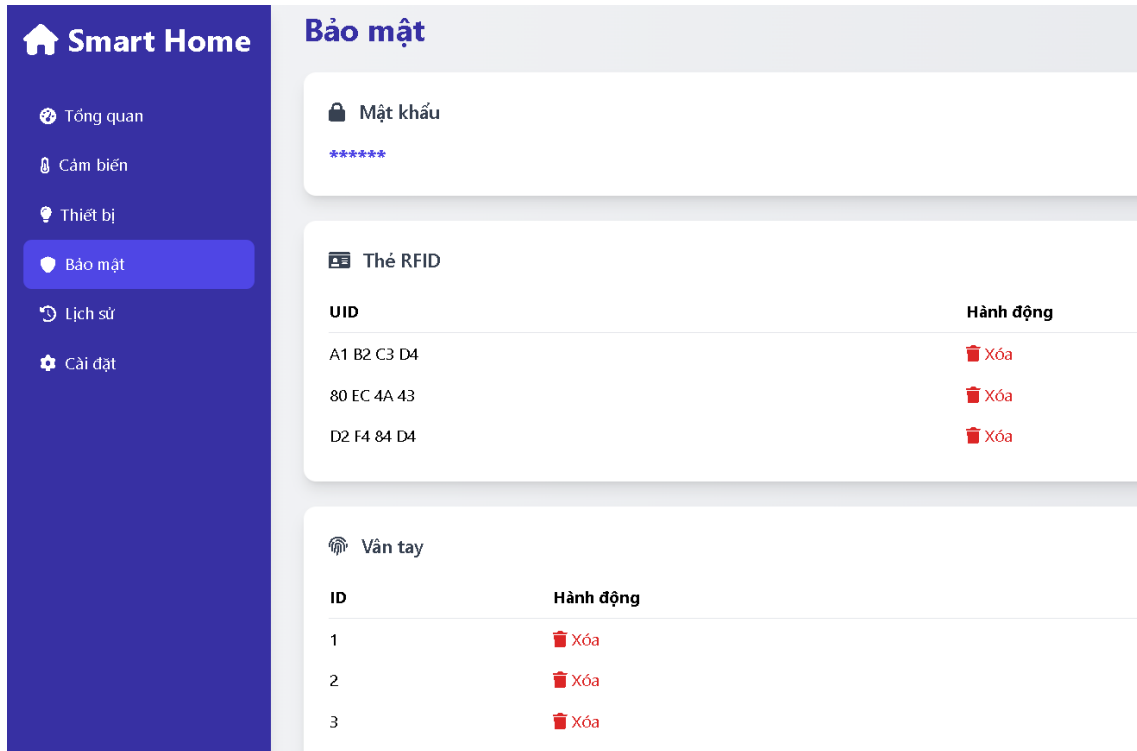
Trên Firebase sẽ lưu trữ dữ liệu của các phương thức xác thực đã được cài đặt đó lại như hình 4.14:



Hình 4.14: Dữ liệu xác thực được lưu trữ trên Firebase

4.1.1.6. Dữ liệu trên Dashboard

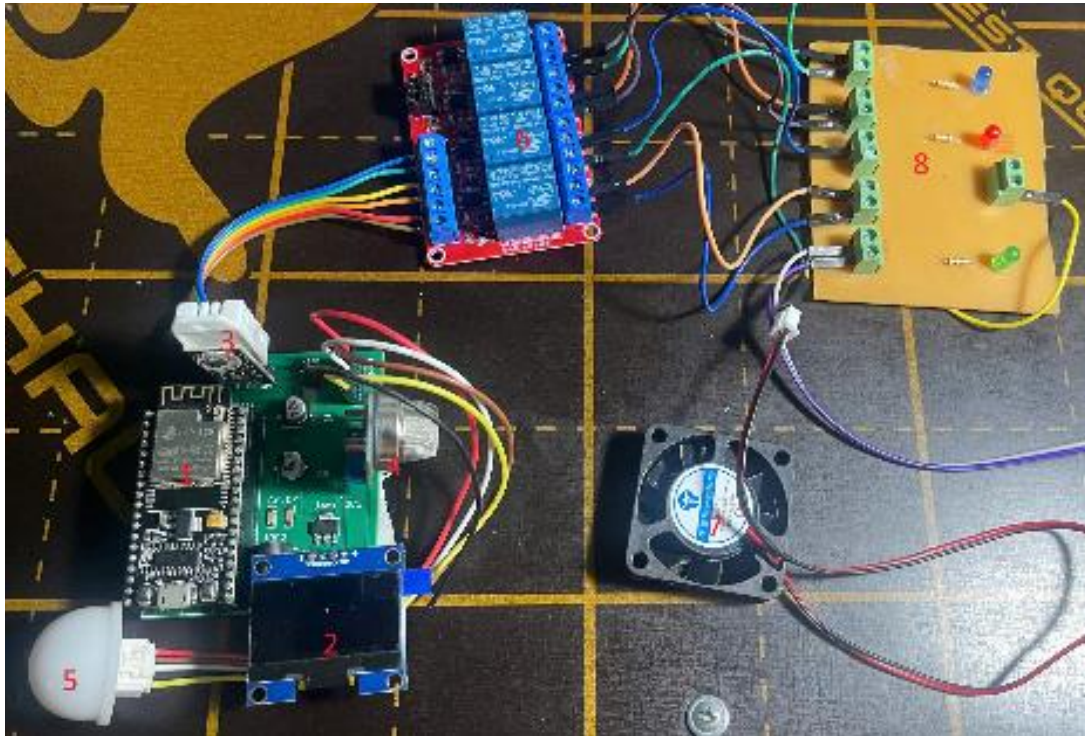
Đồng thời đó thì trên Dashboard cũng sẽ lưu trữ lại toàn bộ dữ liệu xác thực đó khi đồng bộ với Firebase và tại đây cũng sẽ có các nút để xóa các dữ liệu đó nếu như cần được minh họa như hình 4.15:



Hình 4.15: Dữ liệu xác thực lưu trữ trên Dashboard

4.1.2. Khối Slave

Mô hình phần cứng thực tế của khối master được xây dựng dựa trên bo mạch chính là ESP32, tích hợp các module như cảm biến nhiệt độ và độ ẩm (DHT22); Cảm biến khí gas (MQ-2); Cảm biến ánh sáng (BH1750); Module relay và màn hình OLED để đảm nhiệm vai trò giám sát các thông số cảm biến, hiển thị thông tin, điều khiển từ xa và tự động và giao tiếp với Firebase. Các thành phần linh kiện được lắp ráp trên một bo mạch PCB nhỏ gọn với các chân cắm module để thuận tiện cho việc thay đổi linh kiện bị hư hỏng.



Hình 4.1, mô tả bố trí về phần cứng và có thể thấy rõ các thành phần chính có trong hệ thống bao gồm:

1. Vi điều khiển ESP32.
2. Màn hình OLED.
3. Cảm biến nhiệt độ, độ ẩm DHT22.
4. Cảm biến khí Gas MQ-2.
5. Cảm biến ánh sáng BH1750.
6. Module relay 4 kênh.
7. Thiết bị quạt.
8. Thiết bị đèn.

Đầu tiên khi cấp nguồn cho khối slave thì OLED sẽ hiển thị thông tin khởi động như sau:



Hình 4.16: Kết nối WiFi



Hình 4.17: Kết nối Firebase



Hình 4.18: Test relay

4.1.2.1. Hiển thị thông số cảm biến lên OLED

Sau khi khởi động xong thì OLED thì hiển thị luân phiên các page (trang) mỗi 4 giây, các trang sau sẽ chứa các thông tin sau:

- Trang 1: Thông số cảm biến



Hình 4.19: OLED hiển thị thông số cảm biến

Hình 4.19, minh họa OLED hiển thị thông số của các cảm biến như nhiệt độ, độ ẩm, khí gas và ánh sáng. Cũng như là trạng thái của khí gas là an toàn hay nguy hiểm.

4.1.2.2. Hiển thị trạng thái điều khiển thiết bị lên OLED

- Trang 2: Trạng thái thiết bị



Hình 4.20: OLED hiển thị trạng thái thiết bị

Hình 4.20, minh họa OLED hiển thị trạng thái ON/OFF của thiết bị relay1, relay2, relay3 và relay4. Cũng như là trạng thái của quạt và đèn, hiển thị kết nối WiFi thành công và kết nối thành công Firebase.

4.1.2.3. Chế độ điều khiển từ xa thông qua Dashboard

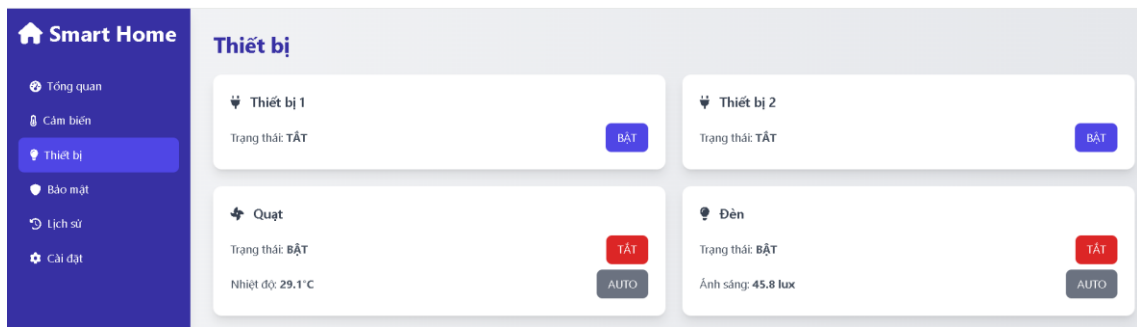
- Trang 3: Chế độ điều khiển



Hình 4.21: OLED hiển thị chế độ điều khiển

Hình 4.21, minh họa OLED hiển thị chế độ điều khiển gồm hai chế độ là điều khiển từ xa (MANUAL) và tự động (AUTO).

- Chế độ điều khiển từ xa thông qua Dashboard:



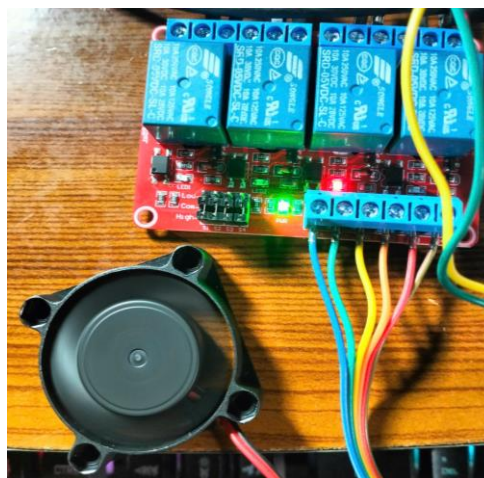
Hình 4.22: Điều khiển từ xa thông qua Dashboard

Hình 4.22, minh họa chế độ điều khiển từ xa qua các thiết bị 1, thiết bị 2, quạt và đèn. Thiết bị 1 và 2 sẽ có một chế độ cố định là điều khiển từ xa, còn quạt và đèn sẽ có 2 chế độ đó là vừa tự động và từ xa.



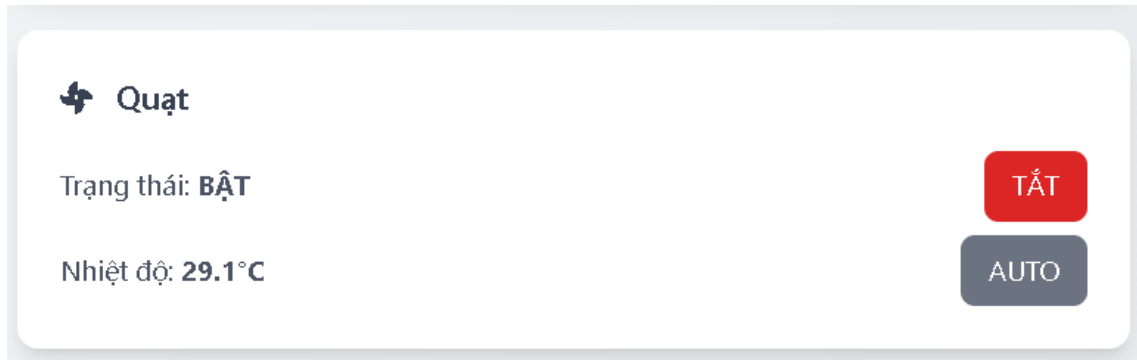
Hình 4.23: LED 1 và LED 2 ở chế độ từ xa

Hình 4.23, minh họa cho chế độ điều khiển từ xa điều khiển relay1 và relay2 để bật/tắt LED.

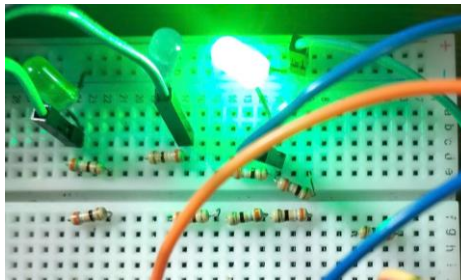


Hình 4.24: Quạt ở hai chế độ từ và từ động

Hình 4.24, minh họa cho hai chế độ để điều khiển quạt đó là điều khiển từ xa và tự động. Khi ở chế độ từ xa thì sẽ điều khiển thông qua các nút nhấn trên Dashboard. Tại Dashboard có các nút chuyển chế độ giữa AUTO và MANUAL. Và khi vào chế độ tự động thì quạt sẽ được bật/tắt dựa vào giá trị của nhiệt độ môi trường như Hình 4.25 minh họa:

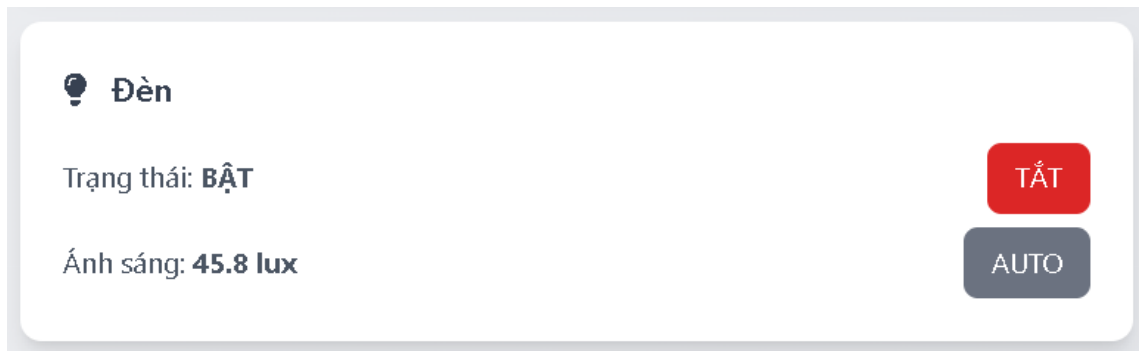


Hình 4.25: Quạt ở hai chế độ



Hình 4.26: LED 4 ở hai chế độ từ xa và tự động

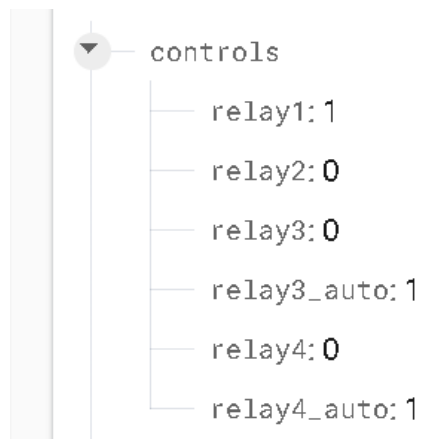
Hình 4.26, minh họa cho hai chế độ để điều khiển LED4 đó là điều khiển từ xa và tự động. Khi ở chế độ từ xa thì sẽ điều khiển thông qua các nút nhấn trên Dashboard. Tại Dashboard có các nút chuyển chế độ giữa AUTO và MANUAL. Và khi vào chế độ tự động thì LED4 sẽ được bật/tắt dựa vào giá trị của cường độ ánh sáng như Hình 4.27 minh họa:



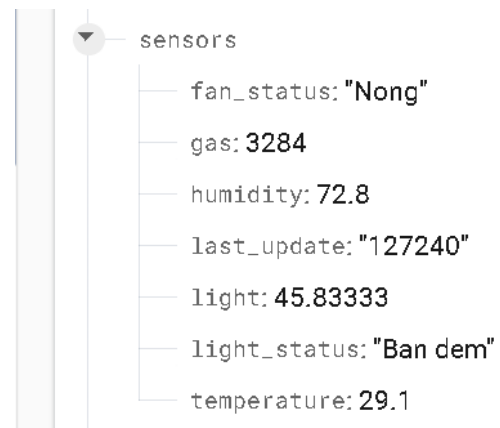
Hình 4.27: Đèn ở hai chế độ

4.1.2.4. Dữ liệu trên Firebase

Trên Firebase sẽ lưu trữ các dữ liệu cảm biến cũng như trạng thái điều khiển thiết bị:



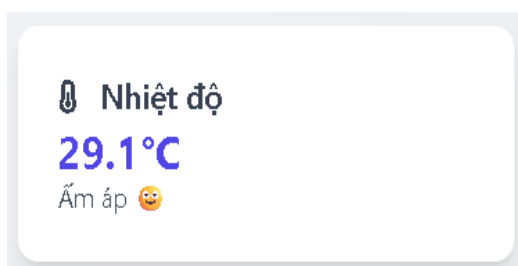
Hình 4.28: Trạng thái điều khiển thiết bị trên Firebase



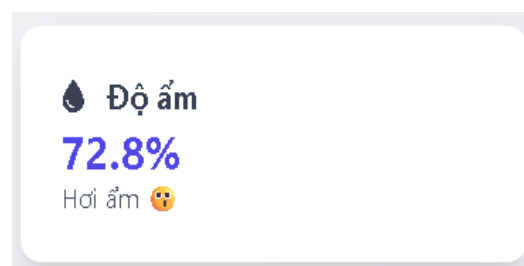
Hình 4.29: Dữ liệu cảm biến trên Firebase

4.1.2.5. Dữ liệu trên Dashboard

Trên Dashboard cũng sẽ hiển thị các dữ liệu cảm biến cũng như là điều khiển thiết bị:



Hình 4.30: Thông số nhiệt độ



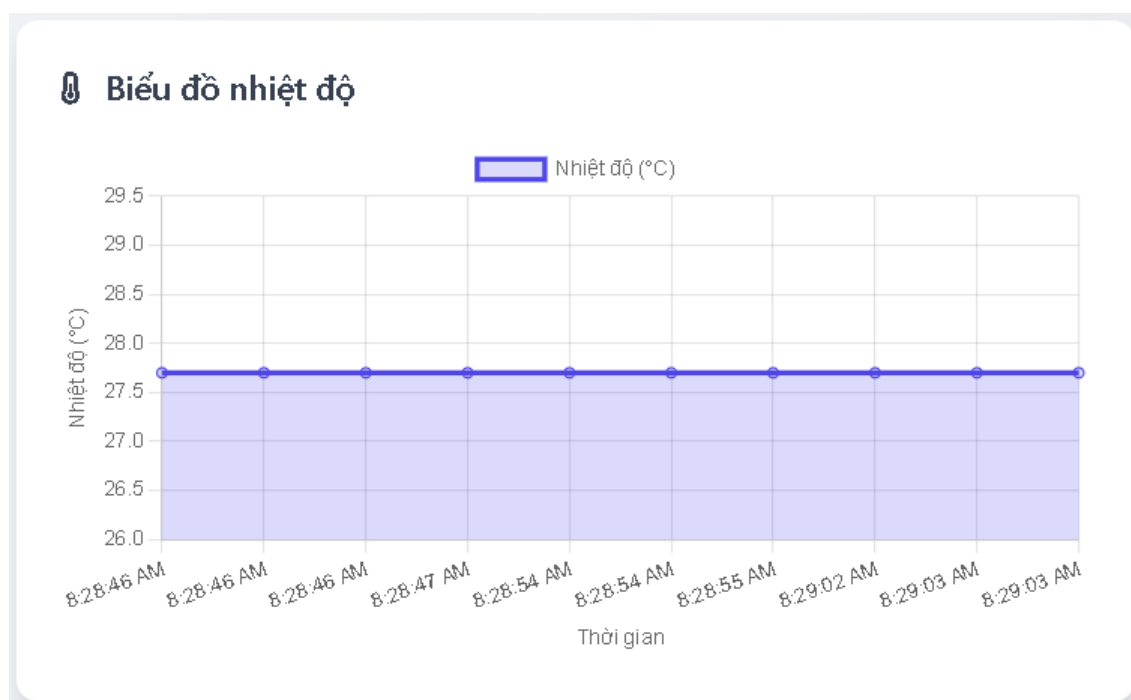
Hình 4.31: Thông số độ ẩm



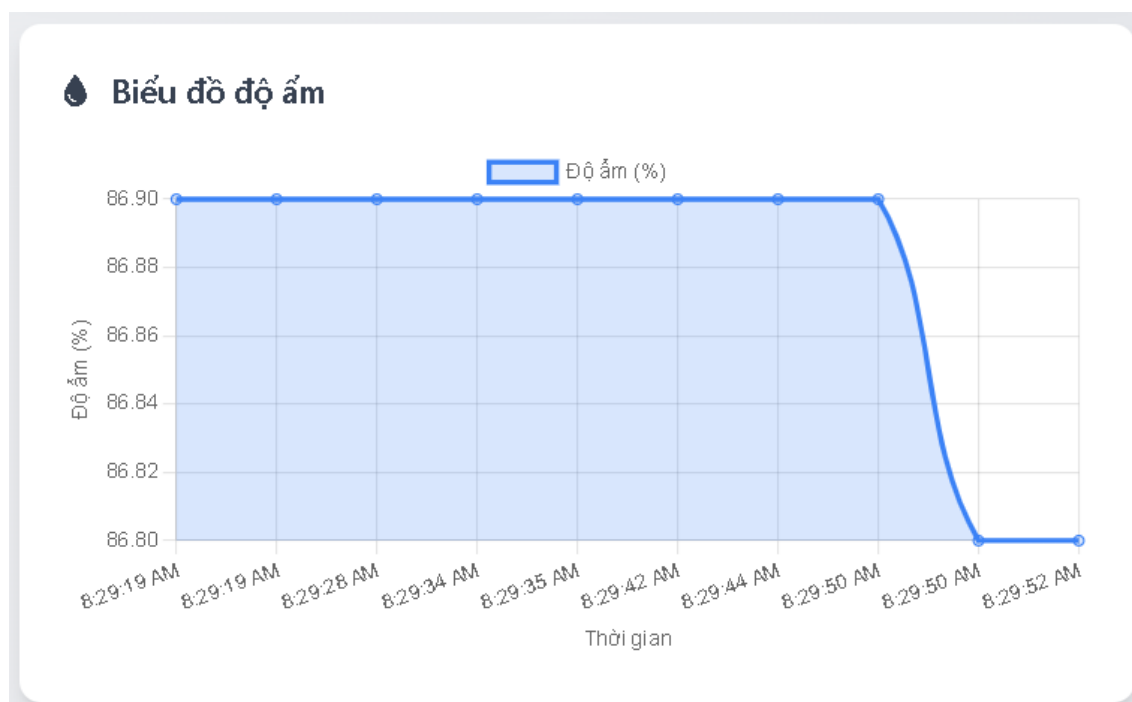
Hình 4.32: Thông số khí gas



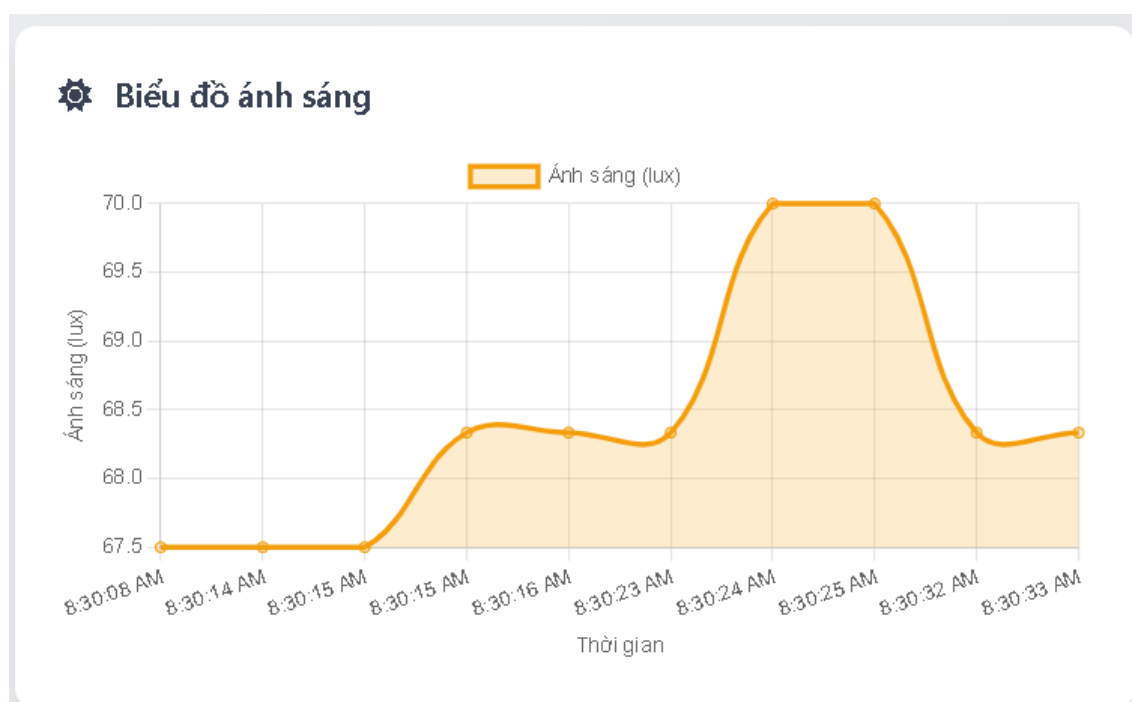
Hình 4.33: Thông số ánh sáng



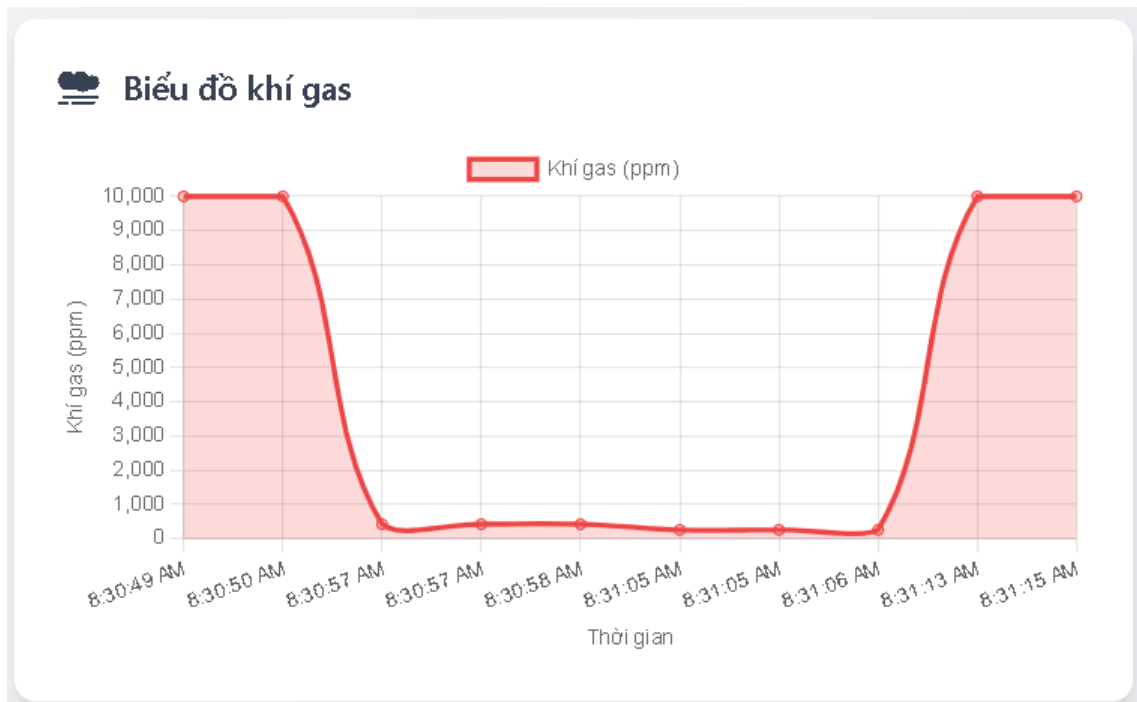
Hình 4.34: Biểu đồ thông số nhiệt độ



Hình 4.35: Biểu đồ thông số độ ẩm



Hình 4.36: Biểu đồ thông số ánh sáng

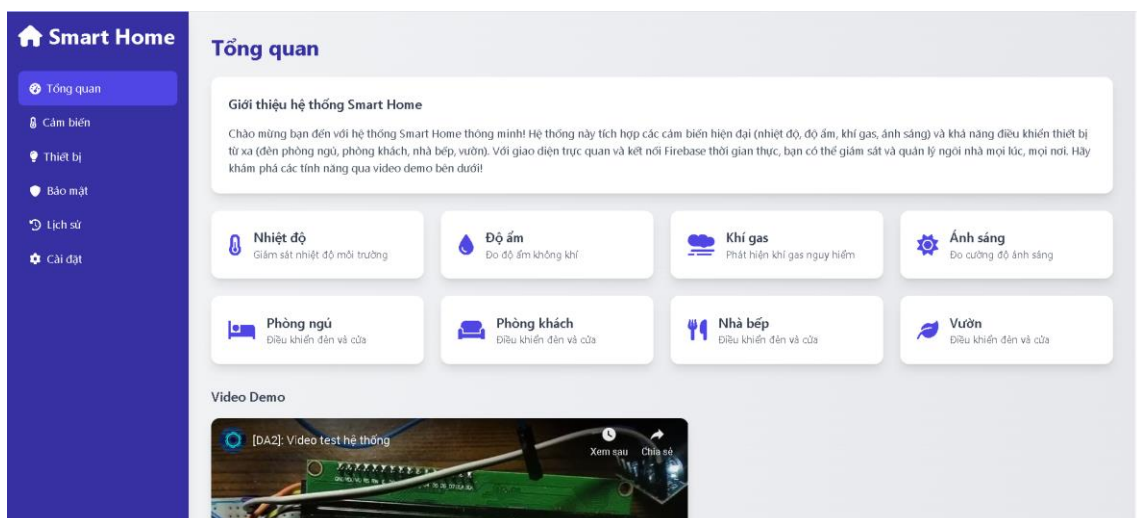


Hình 4.37: Biểu đồ thông số khí gas

4.2. GIAO DIỆN PHẦN MỀM

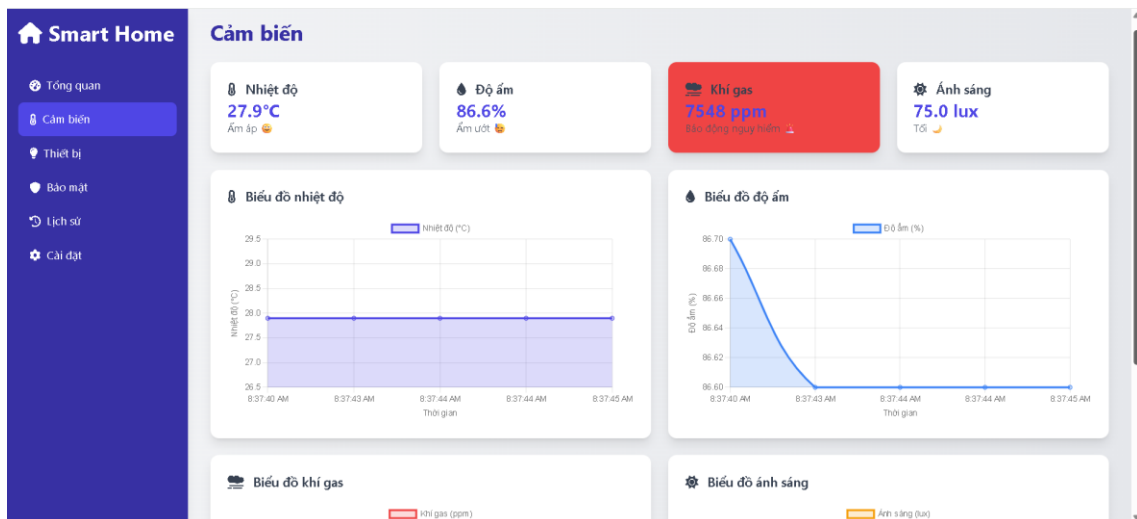
Giao diện dashboard được thiết kế để giám sát theo dõi dữ liệu của các cảm biến, trạng thái điều khiển thiết bị cũng như là điều khiển từ xa. Và cũng là nơi lưu trữ các dữ liệu xác thực. Dashboard sẽ gồm các trang chính như sau:

- Trang tổng quan:



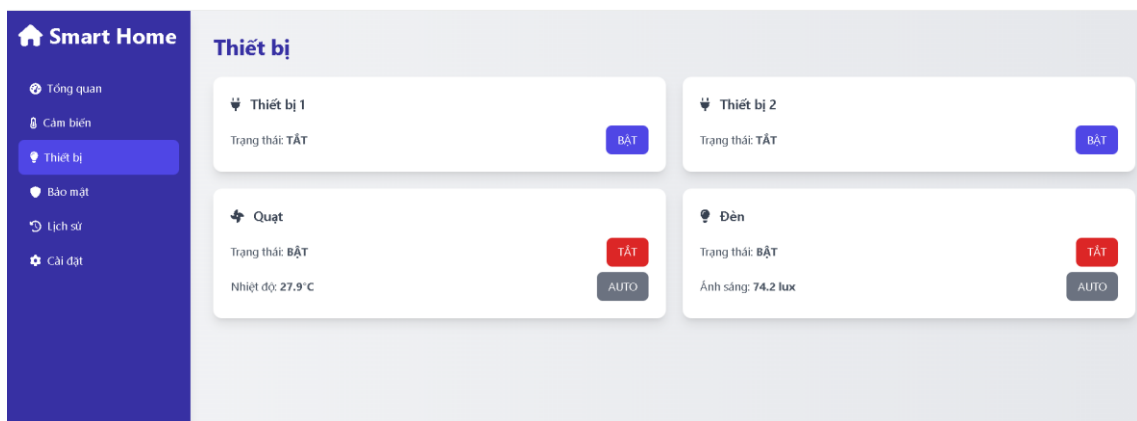
Hình 4.38: Trang tổng quan

- Trang cảm biến



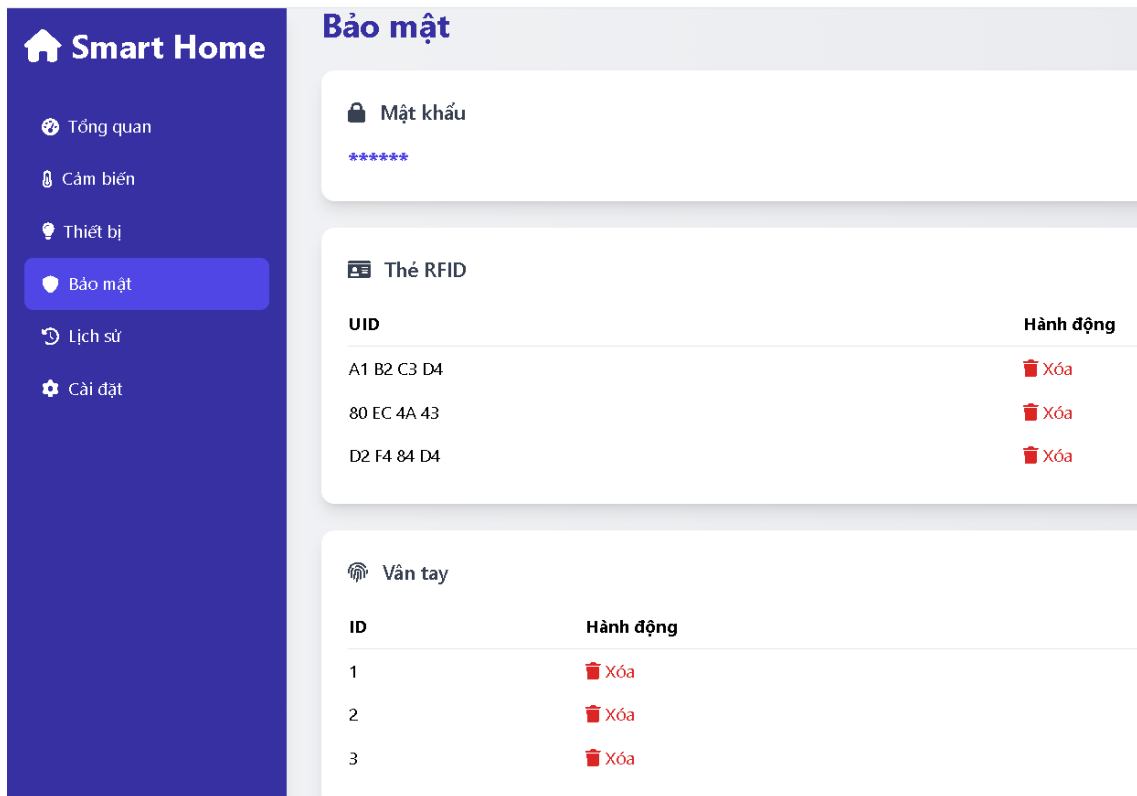
Hình 4.39: Trang cảm biến

- Trang thiết bị



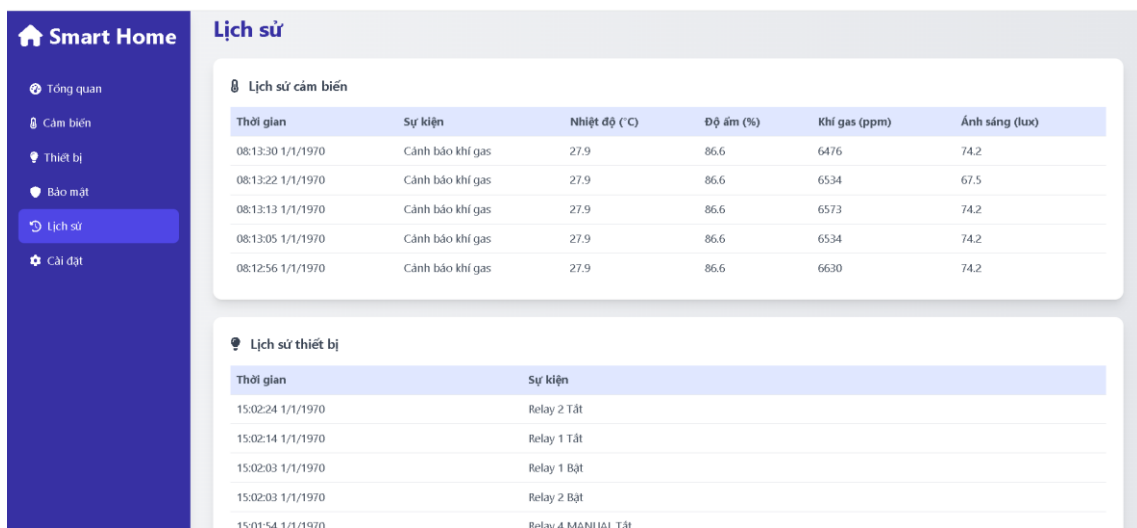
Hình 4.40: Trang thiết bị

- Trang bảo mật



Hình 4.41: Trang bảo mật

- Trang lịch sử



Hình 4.42: Trang lịch sử

4.3. ĐÁNH GIÁ VÀ NHẬN XÉT HỆ THỐNG

Hệ thống giám sát nhà thông minh được phát triển dựa trên sự kết hợp ESP32 – Master và ESP32 – Slave đã được thử nghiệm và đánh giá qua các chức

năng chính: giám sát môi trường, xác thực người dùng, điều khiển thiết bị và cài đặt an ninh. Dựa trên kết quả thực nghiệm, hệ thống hoạt động tốt, đáp ứng được các yêu cầu cơ bản về giám sát và quản lý ngôi nhà thông minh. Tuy nhiên, vẫn còn tồn tại một số hạn chế cần được cải thiện để tăng tính ổn định và hiệu quả trong môi trường thực tế.

4.3.1. Đánh giá hiệu suất

- Hiệu suất phần cứng: Mô hình phần cứng thực tế hoạt động ổn định, với các thành phần như cảm biến (DHT22, MQ-2, BH1750), servo và LCD được tích hợp tốt với ESP32 Slave. Các chế độ xác thực (keypad, RFID, vân tay) và chế độ cài đặt cũng đạt độ chính xác tầm khoảng ~90%, đảm bảo khả năng vận hành ổn định trong điều kiện thử nghiệm.
- Hiệu suất phần mềm: Giao diện dashboard trên Firebase hiển thị dữ liệu cảm biến và lịch sử điều khiển chính xác trong khoảng ~97%, với độ trễ trung bình là 2 giây. Tính năng điều khiển từ xa qua dashboard có thời gian phản hồi trung bình khoảng 3 giây, cho phép người dùng bật/tắt thiết bị hoặc mở cửa từ xa một cách cơ bản.

4.3.2. Ưu điểm và những hạn chế của hệ thống

Ưu điểm:

- Tích hợp đa năng: Hệ thống kết hợp hiệu quả giữa phần cứng và phần mềm, cho phép giám sát, xác thực, điều khiển và cài đặt trong một nền tảng duy nhất, tạo ra một giải pháp toàn diện cho ngôi nhà thông minh.
- Bảo mật cao: Các phương thức xác thực mật khẩu, quét vân tay, quét thẻ và khả năng cài đặt linh hoạt đảm bảo bảo mật cho ngôi nhà, chỉ cho phép những người được ủy quyền truy cập vào hệ thống.
- Tích hợp từ xa: Giao diện dashboard cho phép người dùng theo dõi và điều khiển ngôi nhà mọi lúc, mọi nơi thông qua kết nối Internet, tăng sự tiện nghi và an toàn cho người dùng.

- Chi phí hợp lý: Sử dụng các linh kiện phổ biến và nền tảng Firebase miễn phí giúp giảm chi phí triển khai, phù hợp với các dự án cá nhân hoặc quy mô nhỏ.

Hạn chế:

- Độ nhạy của cảm biến vân tay: Khi đặt ngón tay không đúng vị trí hoặc trên bề mặt cảm biến có bụi thì sẽ dẫn đến xác thực thất bại, làm giảm độ tin cậy trong một số trường hợp.
- Độ trễ kết nối: Với độ trễ trung bình khoảng 3 giây trong việc điều khiển thiết bị từ xa qua dashboard có thể trải nghiệm của người dùng bị ảnh hưởng, đặc biệt là trong các tính hướng khẩn cấp.
- Khả năng mở rộng hạn chế: Hiện tại hệ thống chỉ sử dụng relay để điều khiển đèn, quạt cơ bản và servo để mở cửa, cần bổ sung thêm các thiết bị khác như điều hòa,...

CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. KẾT LUẬN

Hệ thống nhà thông minh đã được thiết kế và triển khai thành công, tích hợp hiệu quả giữa phần cứng và phần mềm để đáp ứng các yêu cầu cơ bản về giám sát môi trường, xác thực người dùng, điều khiển thiết bị, và cài đặt an ninh. Hệ thống sử dụng ESP32 Slave để xử lý dữ liệu từ các cảm biến, điều khiển thiết bị và hiển thị thông tin trên LCD, trong khi ESP32 Master đảm nhiệm nhận diện người dùng và xác thực người dùng điều khiển động cơ. Cả 2 ESP đều kết nối Wi-Fi và giao tiếp với Firebase Realtime Database.

Kết quả thực nghiệm cho thấy hệ thống hoạt động ổn định. Các chức năng giám sát môi trường và điều khiển thiết bị hoạt động hiệu quả, với độ trễ trung bình 2 giây khi cập nhật dữ liệu trên dashboard và 3 giây khi điều khiển từ xa. Chế độ xác thực đạt độ chính xác khoảng 90%, và giao diện dashboard trên Firebase hiển thị dữ liệu với độ chính xác 97%, mang lại trải nghiệm tiện lợi cho người dùng cả tại chỗ và từ xa. Hệ thống cũng đảm bảo bảo mật cao với các phương thức xác thực đa dạng (mật khẩu, vân tay, thẻ RFID) và khả năng cài đặt linh hoạt, cùng với chi phí triển khai hợp lý nhờ sử dụng linh kiện phổ biến và nền tảng Firebase miễn phí.

Tuy nhiên, hệ thống vẫn còn một số hạn chế như độ nhạy của cảm biến vân tay, độ trễ kết nối khi điều khiển từ xa và khả năng mở rộng hạn chế. Dù vậy, hệ thống đã đạt được mục tiêu đề ra, cung cấp một giải pháp toàn diện và thực tế cho ngôi nhà thông minh, phù hợp với nhu cầu giám sát và quản lý của người dùng hiện đại.

5.2. HƯỚNG PHÁT TRIỂN

Dựa trên kết quả thực nghiệm và các hạn chế đã được xác định, hệ thống nhà thông minh có thể được cải thiện và mở rộng trong tương lai để nâng cao hiệu quả và đáp ứng tốt hơn nhu cầu thực tế. Một số hướng phát triển cụ thể bao gồm:

- **Cải thiện độ nhạy của cảm biến vân tay:** Tích hợp thuật toán xử lý tín hiệu nâng cao hoặc sử dụng cảm biến vân tay thế hệ mới để giảm tỷ lệ xác thực thất bại khi ngón tay đặt sai vị trí hoặc bề mặt cảm biến có bụi.
- **Giảm độ trễ kết nối:** Tối ưu hóa giao thức truyền dữ liệu giữa ESP32 và Firebase hoặc bổ sung bộ nhớ đệm cục bộ để giảm độ trễ khi điều khiển từ xa, cải thiện trải nghiệm người dùng xuống dưới 1 giây.
- **Mở rộng khả năng điều khiển thiết bị:** Tích hợp thêm các thiết bị gia dụng như quạt, điều hòa, cửa tự động bằng cách nâng cấp module relay hoặc sử dụng các giao thức điều khiển không dây như Zigbee, tăng tính ứng dụng của hệ thống.
- **Phát triển tính năng thông báo thời gian thực:** Nâng cấp giao diện dashboard trên Firebase để hỗ trợ gửi thông báo qua email hoặc ứng dụng di động khi phát hiện sự cố (như rò rỉ khí gas hoặc xác thực thất bại), tăng tính an toàn.
- **Tích hợp ứng dụng di động:** Xây dựng ứng dụng trên Android/iOS để thay thế hoặc bổ sung cho Web Dashboard, giúp người dùng quản lý hệ thống tiện lợi hơn trên thiết bị di động.
- **Nâng cấp bảo mật và lưu trữ:** Tăng dung lượng lưu trữ vân tay trên AS608 hoặc sử dụng cơ chế mã hóa dữ liệu trên Firebase để tăng cường bảo mật thông tin người dùng.

TÀI LIỆU THAM KHẢO

- [1] Simar Singh, Sourabh Anand, and M. K. Satyarthi, "A Comprehensive Review of Smart Home Automation Systems," *Advances in Computer Science and Information Technology (ACSIT)*, vol. 10, pp. 61-66, 2023.
- [2] A. S. al-Sumaiti, M. H. Ahmed, and M. M. A. Salama, "Smart Home Activities: A Literature Review," *Electric Power Components and Systems*, tập 42, pp. 294-305, 2014.
- [3] Nguyễn Hùng Long và Trịnh Xuân Hùng, "Thiết kế hệ thống nhà thông minh dùng chuẩn không dây Zigbee," Đồ án tốt nghiệp, Khoa Kỹ thuật - Công nghệ, Trường ĐH Bà Rịa - Vũng Tàu, Vũng Tàu, 2022.
- [4] Nguyễn Văn Thắng, Nguyễn Trọng Đức, "Ứng dụng Internet of Things xây dựng ngôi nhà thông," Đề tài NCKH cấp Trường, Trường Đại học Hàng hải, 2016.
- [5] Elsenpeter, R. C., & Velte, T. J., "Build Your Own Smart Home," trong *McGraw-Hill/Osborne*, 2003, pp. 18-19.
- [6] E. Systems, "ESP32 Technical Reference Manual," Espressif System, 2023.
- [7] T. Liu, "Digital-output relative humidity & temperature sensor DHT22," Aosong , China, 2020.
- [8] R. Semiconductor, "BH1750 Ambient Light Sensor IC Datasheet," Rohm Semiconductor, 2022.
- [9] H. Electronics, "MQ-2 Semiconductor Sensor for Combustible Gas," Hanwei Electronics, 2021.
- [10] N. Semiconductors, "MFRC522," NXP Semiconductors, 2010.
- [11] Alientek, "ATK-AS608 指纹模块用户手册," ALIENTEK, China, 2016.
- [12] H. Teachnology, "4-Channel Relay Module Datasheet," Hongfa Teachnology, 2023.
- [13] T. Pro, "MG90S Micro Sensor Motor Datasheet," Tower Pro, 2022.

- [14] N. Semiconductors, “PCF8574/PCF8574A Remote 8-bit I/O expander for I2C-bus,” NXP Semiconductors, 2011.
- [15] S. SYSTECH, “128 x 64 Dot Matrix OLED/PLED Segment/Common Driver with Controller,” Solomon Systech Limited, 2008.
- [16] Bkaii, “Tìm hiểu về truyền thông UART: khái niệm, nguyên lý và ứng dụng,” 5 7 2025. [Trực tuyến]. Available: <https://bkaii.com.vn/tin-tuc/1099-tim-hieu-ve-truyen-thong-uart-khai-niem-nguyen-ly-va-ung-dung> .
- [17] Arduinokit, “Khái niệm cơ bản về chuẩn giao tiếp I2C trong Arduino,” 5 7 2025. [Trực tuyến]. Available: <https://arduinokit.vn/khai-niem-co-ban-ve-chuan-giao-tiep-i2c-trong-arduino/> .
- [18] Arduinokit, “Chuẩn giao tiếp SPI là gì? Giao thức SPI trong Arduino,” 5 7 2025. [Trực tuyến]. Available: <https://arduinokit.vn/chuan-giao-tiep-spi-la-gi/>.
- [19] C. Phong, “Giải thích về các tiêu chuẩn WiFi phổ biến nhất hiện nay,” 5 7 2025. [Trực tuyến]. Available: <https://quantrimang.com/cong-nghe/cac-chuan-wireless-802-11b-802-11a-802-11g-va-802-11n-47723>.
- [20] T. G. Nhuan, “Tìm hiểu về Firebase Realtime Database,” 5 7 2025. [Trực tuyến]. Available: <https://viblo.asia/p/tim-hieu-ve-firebase-realtime-database-Az45bxzVZxY>.
- [21] D. T. T. LAI, “Phần mềm lập trình Arduino IDE là gì,” 5/ 7/ 2025. [Trực tuyến]. Available: <https://dientutuonglai.com/arduino-ide-la-gi.html> .
- [22] A. Software, “EasyEDA – Công Cụ Thiết Kế Vi Mạch,” 5/ 7/ 2025. [Trực tuyến]. Available: <https://atpsoftware.vn/easyeda-cong-cu-thiet-ke-vi-mach/>.
- [23] N. T. Hiếu, “Visual Studio Code là gì? Hướng dẫn sử dụng VSCode cho người mới,” 5/ 7/ 2025. [Trực tuyến]. Available: <https://200lab.io/blog/visual-studio-code-la-gi-huong-dan-su-dung-vscode-cho-nguoi-moi> .
- [24] Hitachi, “HD44780U (LCD-II),” Hitachi, Ltd, Japan.

- [25] Robles, R. J., & Kim, T. H., “Applications, systems and methods in smart home technology: a review,” *International Journal of Advanced Science and Technology*, p. 15, 2010.
- [26] Al-Fuqaha, A., Guizani, M., Mohammadi, M., Aledhari, M., & Ayyash, “Internet of Things: A survey on enabling technologies, protocols, and applications,” *IEEE Communications Surveys & Tutorials*, pp. 2347-2376, 2015.
- [27] Gubbi, J., Buyya, R., Marusic, S., & Palaniswami, “Internet of Things (IoT): A vision, architectural elements, and future directions,” *Future Generation Computer Systems*, pp. 1645-1660, 2013.
- [28] Bkaii, “ “Tìm hiểu về truyền thông UART: khái niệm, nguyên lý và ứng dụng,” 5 7 2025. [Trực tuyến]. Available: <https://bkaii.com.vn/tin-tuc/1099-tim-hieu-ve-truyen-thong-uart-khai-niem-nguyen-ly-va-ung-dung> .
- [29] Đ. T. T. LAI, “Phần mềm lập trình Arduino IDE là gì,” 5/ 7/ 2025. [Trực tuyến]. Available: <https://dientutuonglai.com/arduino-ide-la-gi.html>.